

Книга: **MicroPython** за **VK_RA4M2** чрез демонстрационни примери

Цел

.py VK_RA4M2.

- 1.
- 2.
- 3.

Как да се ползва книгата

examples.

- 1.
- 2.
- 3.
- 4.
- 5.

.py

DO_WRITE, DO_BOOTLOADER, DO_LIGHTSLEEP, DO_DEEPSLEEP SET_DEM

O_TIME,

Съдържание

Глава 0: Първо свързване — от кутията до REPL

-
- - 2
-
-
-
- , -
-

Част I: Първи стъпки с борда и с MicroPython

Глава 1. Първа програма

Глава 2. Първи асинхронни примери

Част II: Езикови основи на MicroPython

Глава 3. Променливи и константи

Глава 4. Оператори, условия и цикли

Глава 5. Enums в MicroPython

Глава 6. Масиви и таблици

Глава 7. Структури в MicroPython

Глава 8. Указатели и еквивалентите им в MicroPython

Глава 9. Функции, callbacks и таблици от функции

Част III: Цифрови и аналогови входове и изходи

Глава 10. Цифров изход

Глава 11. Цифров вход

Глава 12. Аналогов вход и ADC

Глава 13. DAC изход, timed playback и retro synth

Глава 14. PWM

Част IV: Таймери, прекъсвания и време

Глава 15. Закъснения, хардуерни таймери и Timer(-1)

Глава 16. Прекъсвания

Глава 17. Дебаунс на бутон

Глава 18. Четене на бутон: polling, IRQ, Timer(-1), debounce и събития

Глава 19. RTC

Част IVб: asyncio — мост към конкурентно програмиране

(,)

Глава 20. Основи на asyncio

Глава 21. asyncio Event синхронизация

Част V: Серийни интерфейси и двоични протоколи

Глава 22. I2C master и runtime избор на пинове

Глава 23. SoftI2C и I2CTarget

Глава 24. Hardware SPI и SoftSPI

Глава 25. Базова UART комуникация

Глава 26. Работа с външни модули по интерфейси

Глава 27. Байтове, bytearray, struct, memoryview и двоични протоколи

Част VI: Системни функции, памет и организация на проекта

Глава 28. TouchPad: четене, диагностика и cached API

Глава 29. renesas.Flash, /flash и dataflash

Глава 30. Управление на паметта в MicroPython

Глава 31. machine модулът и порт-специфично поведение

Глава 32. boot.py, main.py и жизнен цикъл на програмата

Глава 33. Организация на проект: модули, drivers, config и pins.py

Глава 34. Таблични данни, конфигурации и малки файлови формати

Част VII: Сигнали, измерване, енергия и индикация

Глава 35. Филтриране, калибриране и хистерезис при входни сигнали

Глава 36. Измерване и диагностика с осцилоскоп и logic analyzer

Глава 37. Ниска консумация и събуждане

Глава 38. Динамична 7-сегментна индикация и матрично сканиране

Глава 39. NeoPixel

Глава 40. WS2812 през machine.WS2812 и SCI TX-only

Част VIII: FSM като начин на мислене

Глава 41. Минимална FSM и таблична FSM

Глава 42. FSM с timeout

Глава 43. Conditions vs Events

Глава 44. Guard функции

Глава 45. Приоритети и event queue

Глава 46. Tickless timers и sleep идея

Част IX: Надеждни вградени програми и архитектура на приложението

Глава 47. Обработка на грешки, recovery и fail-safe поведение

Глава 48. Watchdog и самовъзстановяване

Глава 49. Състояние на системата: heartbeat, health checks и debug интерфейс

Глава 50. Капсуловане на периферии като класове

Глава 51. От пример към завършено приложение

Глава 52. RGB_Guardian — казус: игра на WS2812 лента

Част X: Курс по елементарна сигнална обработка

- 1. sample, sample_rate, / , midpoint=2048
- 2. sine, square, triangle, saw
- .
- .
- . RMS power
- . FIR -
- . IIR - / -
- . FFT, , ,
- . audiomixer -
- 10. -

Част XI: Цифрова обработка на аудио сигнали (DSP)

Глава 53. Storm ADC — DTC-управлявано аудио семплиране

Глава 54. FFT — честотен анализ с CMSIS-DSP

Глава 55. MSGEQ7 емуляция — 7-лентов спектрален анализатор

Глава 56. DSP Synthesis — Ретро синтезатор архитектура

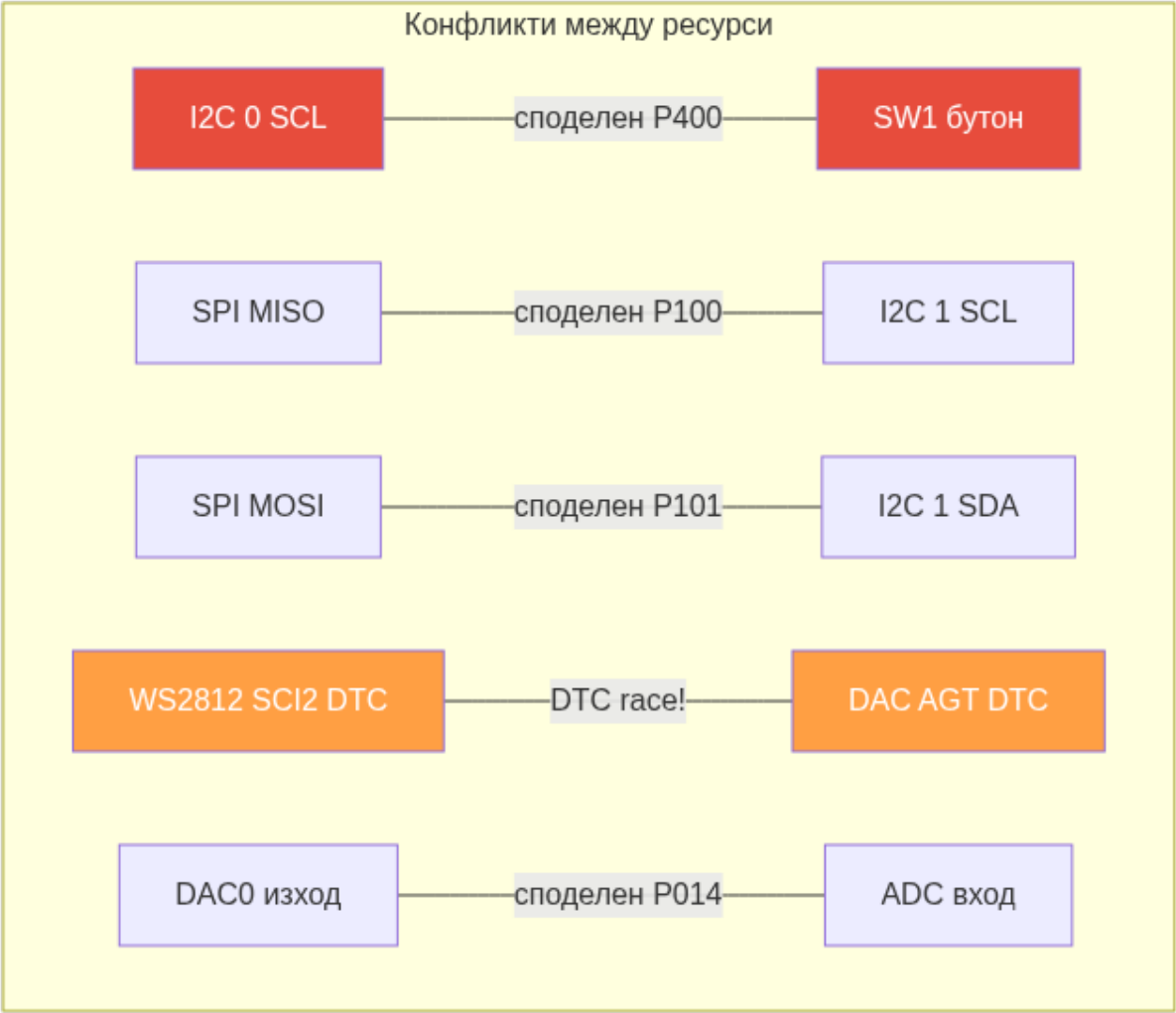
Ресурсна карта на VK_RA4M2

- ```

•
• 1 брой - LED1 = P204
•
• 1 брой - SW1 = P400
•
• USBDP = P914, USBDM = P915, USB_VBUS = P407
•
• 13 броя - P000, P001, P002, P003, P004, P005, P006, P007, P008, P013, P014, P015, P500
•
• 3 броя - ADC.CORE_TEMP, ADC.CORE_VREF, ADC.VREF
•
• 2 броя - P014, P015
•
• 14 броя - P107, P106, P105, P104, P113, P114, P112, P115, P608, P409, P408, P600, P304, P303
•
• 4 инстанции - UART(0), UART(2), UART(7), UART(9)
•
• 2 2 инстанции - I2C(0)=P400/P401, I2C(1)=P100/P101
•
• 2 2 инстанции - I2CTarget(0)=P400/P401, I2CTarget(1)=P100/P101
•
• 1 канал - CS=P103, SCK=P102, MISO=P100, MOSI=P101
•
• 2
•
•
•
• 12 броя - P205, P206, P407, P408, P409, P410, P411, P412, P413, P414, P415, P708
•
• P207 = TSCAP
•
• 6 броя - Timer(1), Timer(2), Timer(3), Timer(4), Timer(5), Timer(6)
•
• Timer(-1)
•
• 1 брой
•
•
• /flash
•
• renesas.Flash
• dataflash 8 KB
•
•
• SPI 1
• Timer(-1)
• SW1 P400, I2C(0).SCL
• LED1 - , 1
• Pin.PULL_UP Pin.PULL_DOWN
• lightsleep deepsleep
• J16 -
• DAC P014/P015 DTC , DTC - DMAC -
• DAC

```

## Таблица на конфликти между ресурси



| Ресурс А | Ресурс Б | Споделен пин/хардуе | Последствие  |
|----------|----------|---------------------|--------------|
| 2 (0).   | 1 ( )    | 00                  | 2 (0) 2 (1). |
| .        | 2 (1).   | 100                 | 2 (1)        |
| .        | 2 (1).   | 101                 | 2 (1).       |
| 2 12 2   |          |                     | ( ).         |
| 0        |          | 0                   | 0            |
| 0        | 01       | 01                  | ,            |
| 1        | 01       | 01                  | .            |

[P ] : , . Мс P

Пълен каталог

(  
").  
.py

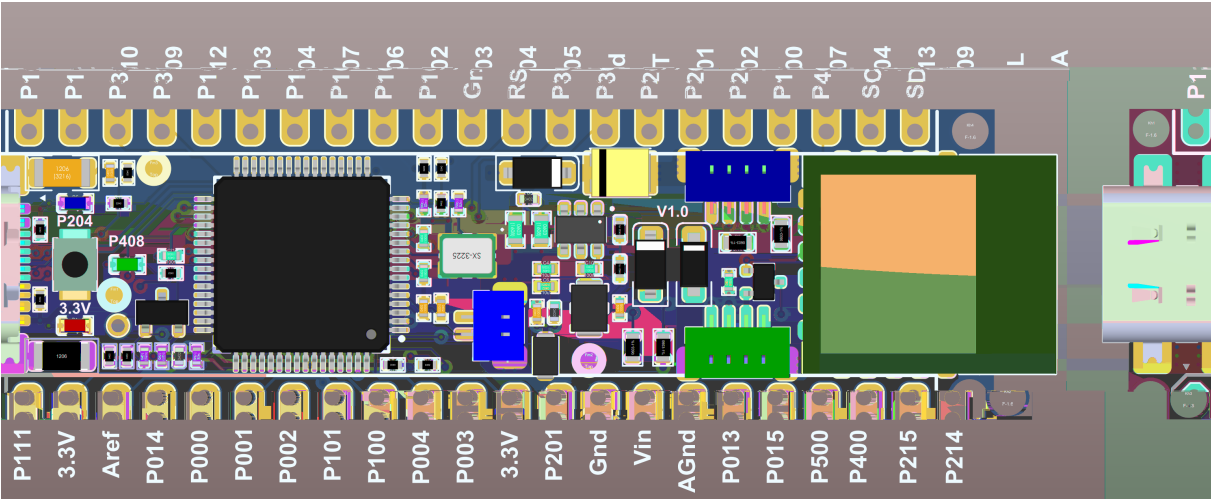
Глава 0: Първо свързване — от кутията до REPL

Какво е в комплекта

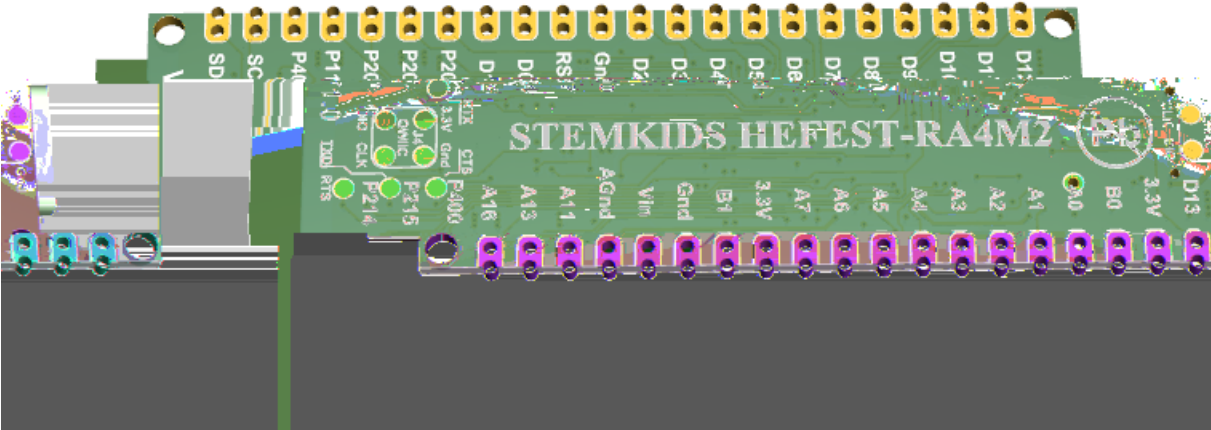
VK\_RA4M2

| Вариант       | Формат | Размер | USB конектор |
|---------------|--------|--------|--------------|
| - 2 ( - 2 )   | -      |        | -            |
| - 2 ( - 2-1 ) |        |        | -            |

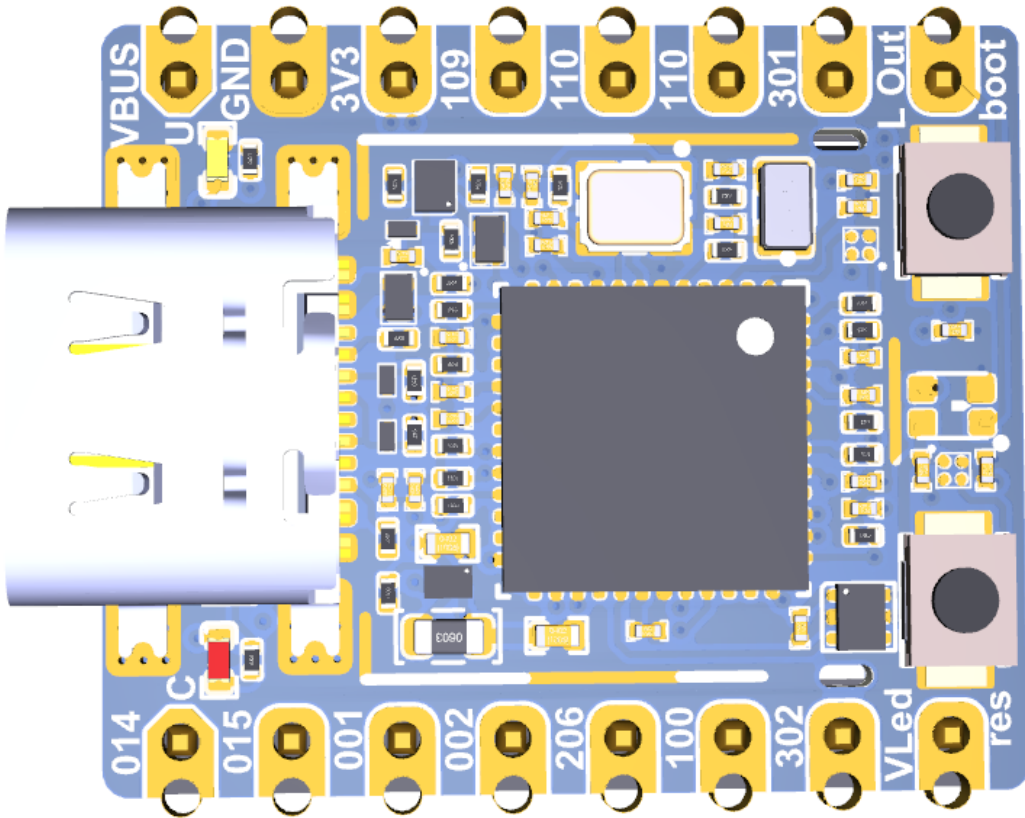
един и същ MCU 2 ( - , 100 , , 12 )



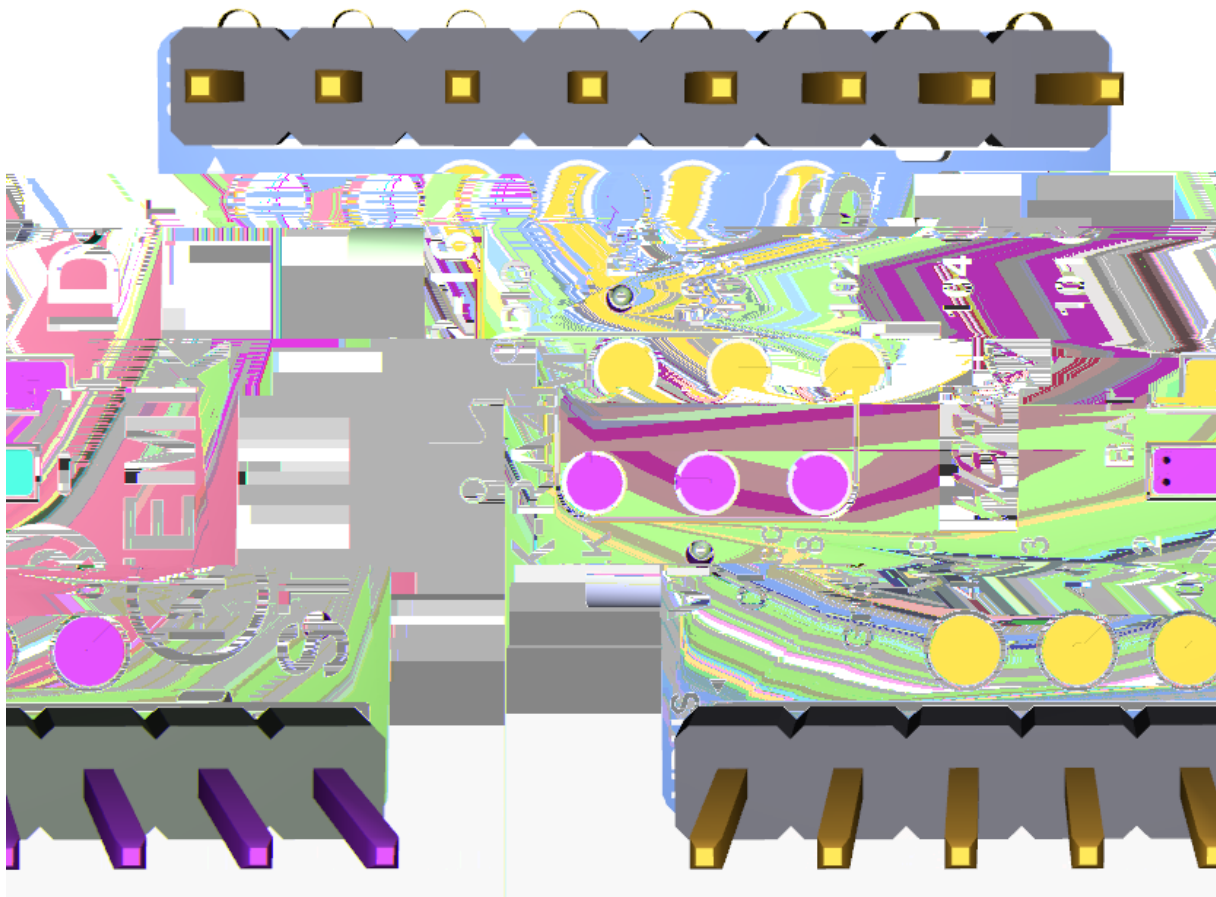
VK-RA4M2 Na



VK-RA4M2 Na



VK-RA4M2 Fe



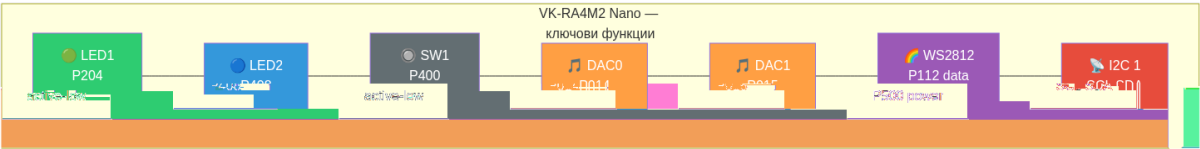
VK-RA4M2 Fe

Какво ви трябва:

- 1 - 2 ( )
- 1 - ( , !)
- / /

[/] USB : da a .

Pinout: VK-RA4M2 Nano (HEFEST-RA4M2)



VK-RA4M2 Na

Горен ред ( )

| 1   | 2  | 3   | 4 | 5 | 6  | 7  | 8   | 9  | 10 | 11 | 12 | 13 | 14  | 15 | 16 | 17 | 18 | 19 | 20 |
|-----|----|-----|---|---|----|----|-----|----|----|----|----|----|-----|----|----|----|----|----|----|
| 110 | 10 | 112 | 0 | 0 | 10 | 10 | 102 | 10 | 10 |    | 01 | 02 | 200 | 20 | 20 | 11 | 0  |    |    |

Долен ред ( )

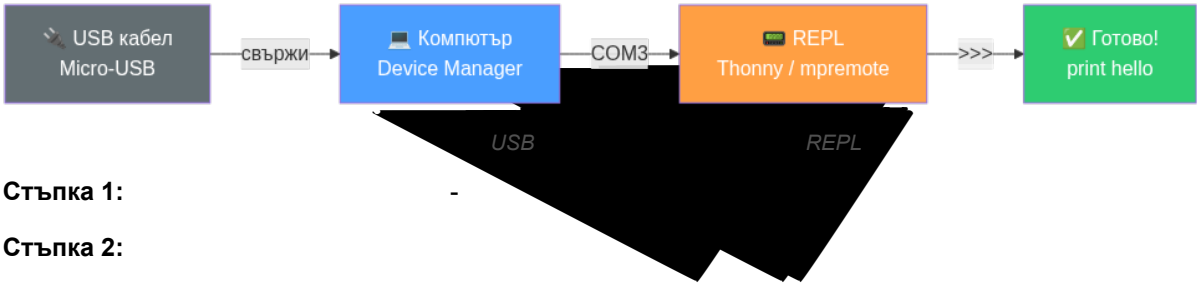
|     |   |   |    |     |     |     |     |    |    |     |    |    |    |    |    |    |    |    |    |
|-----|---|---|----|-----|-----|-----|-----|----|----|-----|----|----|----|----|----|----|----|----|----|
| 1   | 2 | 3 | 4  | 5   | 6   | 7   | 8   | 9  | 10 | 11  | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 111 | . |   | 01 | 000 | 001 | 002 | 101 | 00 | .  | 201 |    |    |    | 01 | 01 | 00 | 00 | 21 | 21 |

| Функция | Пин (Напо)          | Бележка    |
|---------|---------------------|------------|
| 1       | 20                  | - (0)      |
| 2       | 0                   |            |
| 1 ( )   | 00                  | - , 2 (0). |
| 0       | 01                  |            |
| 1       |                     |            |
| 2 (1)   | ( 100)              | 2          |
| 2 (1)   |                     |            |
| 2 12    | 112                 | 2          |
| 2 12    | 00                  |            |
|         | 000 00 , 01 01 , 00 | 1          |
| ( ) /   | 02/ 01              |            |
|         | -                   | -          |

Pinout: VK-RA4M2 Femto (VK-RA4M2-16)

| Страна | Пинове               |
|--------|----------------------|
|        | , 00, 0 , , .        |
|        | , 012, 01 , 01 , . , |

USB свързване и COM порт



Стъпка 1:

Стъпка 2:

- **Windows:** ( & ). ( . COM3).
- **Linux:** `ls /dev/ttyACM*` /dev/ttyACM0.
- **macOS:** `ls /dev/cu.usbmodem*`

[P ]

: VK-RA4M2

CDC-ACM

(

)

W d

10/11

Стъпка 3:

## Зареждане на MicroPython firmware

1. `.hex` 2
2. **Renesas Flash Programmer** ( ) **J-Flash Lite**
- 
- ( 2).
- `.hex`
- " "
- 3.

## Отваряне на REPL

( - - )

### Вариант А: Thonny (препоръчителен за начинаещи)

- 1.
2. ( )
3. >>> !

### Вариант Б: mpremote (препоръчителен за напреднали)

```
pip install mpremote
mpremote connect COM3 repl
```

>>>

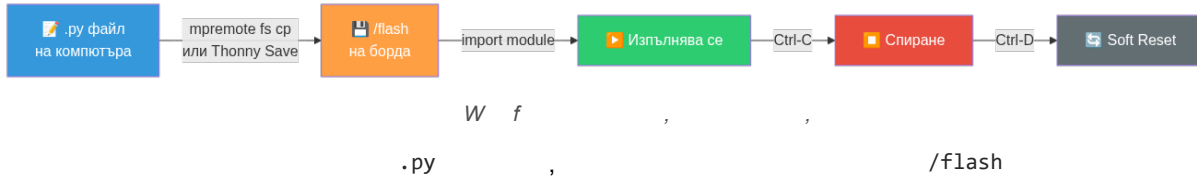
### Вариант В: PuTTY / screen

- **PuTTY (Windows):** , , 11 200 .
- **screen (Linux/macOS):** `screen /dev/ttyACM0 115200`

## Първа проверка

```
>>> print("hello VK_RA4M2!")
hello VK_RA4M2!
>>> from machine import Pin
>>> led = Pin("LED1", Pin.OUT, value=0) # LED светва (active-low)
>>> led.value(1) # LED гасне
```

## Качване на файл на борда



### С mpremote:

```
mpremote connect COM3 fs cp retro_synth.py :/flash/retro_synth.py
```

### С Thonny:

1. .
2. .

### Стартиране на файл:

```
>>> import retro_synth # импортира и изпълнява модула
```

```
mpremote connect COM3 run dac_retro_sfx.py
```

## Reset, Ctrl-C и безопасно спиране

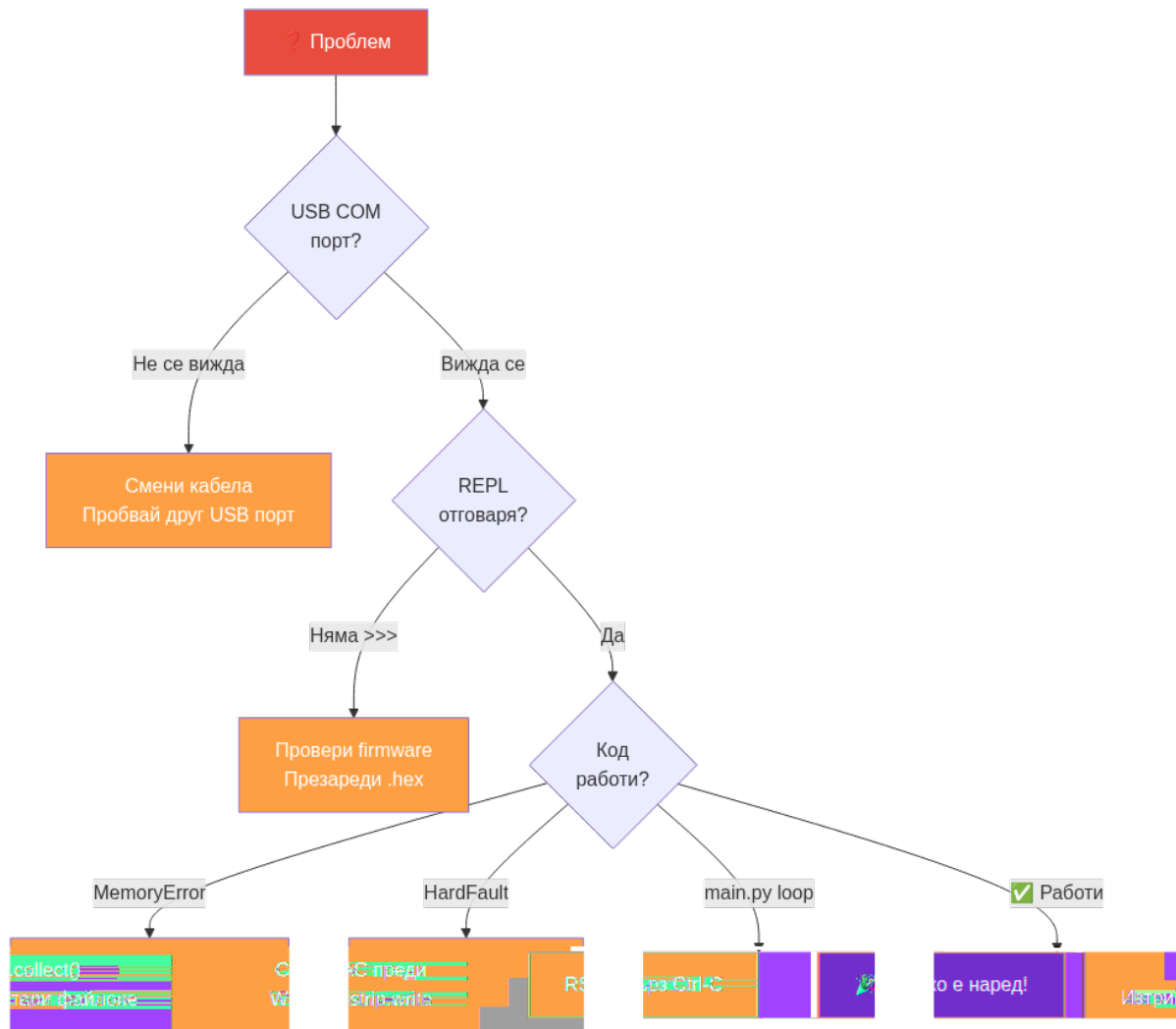
| Действие | Как  | Кога |
|----------|------|------|
|          | -    | , ,  |
| ( )      | -    |      |
|          |      |      |
|          | . () |      |

```
[!] a . b : 1. REPL. 2. RST
. 3. C -C 1-2
a . . 4. , >>>.
: ; . e e("/fa / a . ").
```

## Листване на файлове на борда

```
>>> import os
>>> os.listdir("/flash") # показва всички файлове
>>> os.statvfs("/flash") # показва свободно място
```

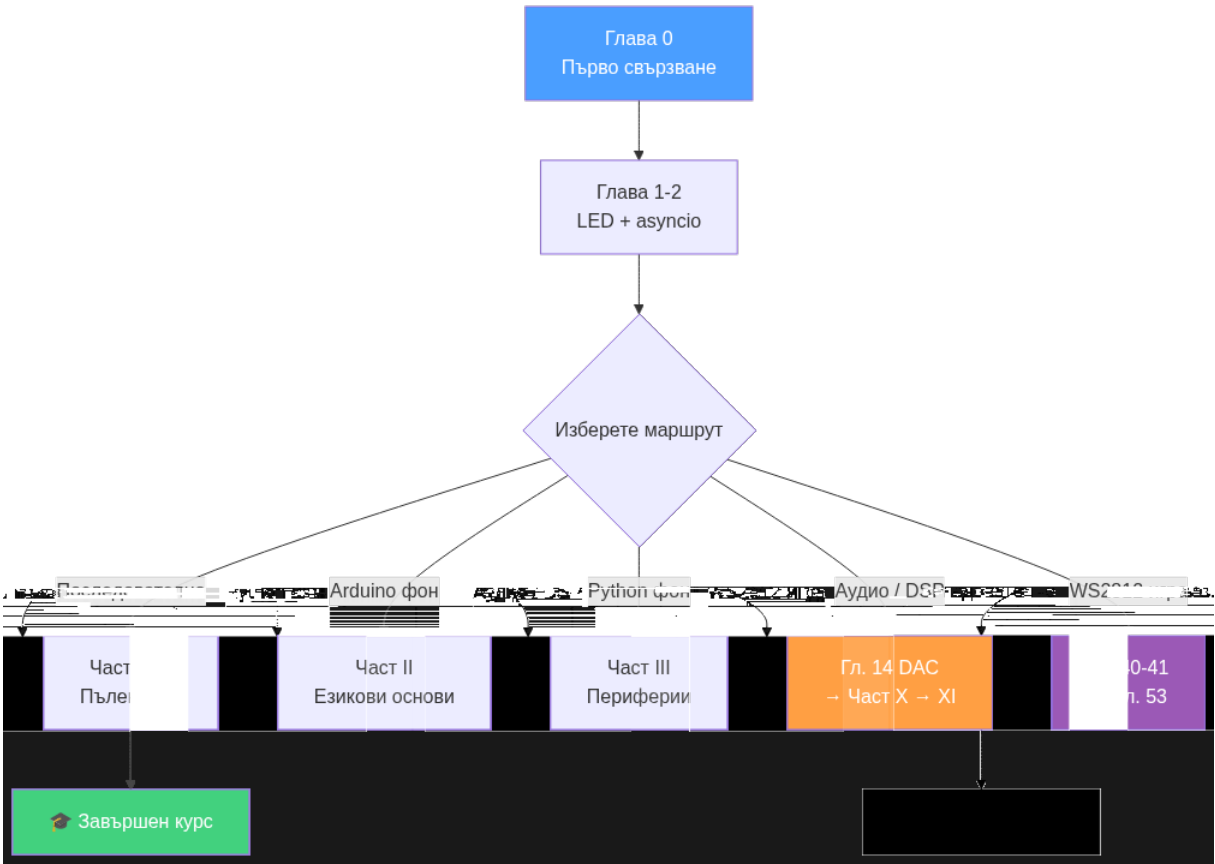
## Системен troubleshooting



T b e f c a

| Симптом | Вероятна причина | Решение     |
|---------|------------------|-------------|
|         |                  |             |
|         |                  |             |
| ( )     |                  |             |
| ,       |                  | , 11 200, 1 |
|         |                  | ()          |
| 2       | /                | (" / / . ") |
| .       |                  | + - ( . - ) |
| 1       | -                | (0) , (1)   |
| + 2 12  |                  | () ( . 2)   |

Навигационна карта



| Ако идвате от... | Препоръчителен старт  |
|------------------|-----------------------|
| /                | 0 1 ( ) ( )           |
| ,                | 0 1                   |
| /                | 0 1 ( ) ( )           |
| 2 12 /           | 0 1 0 1 ( / 2 12) ( ) |
|                  | 0 1 2                 |



# Част I: Първи стъпки с борда и с MicroPython

## Глава 1. Първа програма

[Т] : LED1 = . a e(0) =  
, a e(1) = .

LED1 active-low логика:

value(1) → GPIO = 3.3V → LED = изгасен x

value(0) → GPIO = 0V → LED = светнал v

| led.value(...) | Напрежение | LED |
|----------------|------------|-----|
| 1              | .          |     |
| 0              | 0          |     |

```
from machine import Pin # Импортираме класа Pin, чрез който управляваме GPIO пиновете.
import time # Импортираме модула time, за да правим закъснения между действията.

led = Pin("LED1", Pin.OUT, value=1) # Създаваме изходен обект за LED1 и започваме с изгасен LED.

for blink_index in range(3): # Обхождаме три последователни мигания с for цикъл.
 led.value(0) # Светваме LED1 като подадем активното за платката ниво.
 time.sleep_ms(300) # Изчакаме 300 милисекунди, за да се вижда светването.
 led.value(1) # Гасим LED1, за да завършим едно мигане.
 time.sleep_ms(300) # Изчакаме още 300 милисекунди преди следващото мигане.
```

[Eye] Какво виждаш: 1 . - .

[Lab] Експеримент: 300 100 - . range(3) range(10)  
10 .

[/] : LED1 = P204 ас e- . a e(1) , !

Файл: examples/20\_language\_basics/01\_sample\_program.py

## Резюме

- -
- LED1 active-low value(0) value(1)
- 

## Самопроверка

1. LED1 value(0), value(1)
2. Pin.OUT sleep\_ms()
3. - ,

## Упражнения

1. ( . 100 , 00 ).
2. BLINK\_COUNT DELAY\_MS.
3. print("ON") print("OFF") .

## Ако не работи

- ! . value=1 ( ).
- ! . value(0) value(1) .

## Нататък

## Глава 2. Първи асинхронни примери

[T] : , , . a c

: (a a ), .

### Блокиращ стил (BAD):

```
blink: вкл 500ms чака [X]
blink: изкл 500ms чака [X]
blink: вкл 500ms чака [X]
(counter НИКОГА не работи!)
```

### Asyncio стил (GOOD):

```
t=0ms → blink: вкл LED
t=0ms → counter: started
t=500ms → blink: изкл LED
t=1000ms → counter: 1 секунда
t=1000ms → blink: вкл LED
```

## Задача 1: Мигане на LED

```
import asyncio
from machine import Pin

led = Pin("LED1", Pin.OUT, value=1) # active-low: value=1 → изгасен

async def blink_task():
 while True:
 led.value(0) # [Tip] светваме
 await asyncio.sleep_ms(500) # отстъпваме – друга задача може да работи
 led.value(1) # [X] гасим
 await asyncio.sleep_ms(500) # пак отстъпваме
```

## Задача 2: Броене на секунди

```

async def count_seconds_task():
 seconds = 0
 while True:
 seconds += 1
 print("Изминали секунди:", seconds)
 await asyncio.sleep(1) # чакаме 1 секунда без да блокираме

async def main():
 asyncio.create_task(blink_task()) # стартираме мигането
 asyncio.create_task(count_seconds_task()) # стартираме брояча
 while True:
 await asyncio.sleep(1) # main задачата поддържа event loop-а жив

asyncio.run(main()) # Стартираме event loop-а – без този ред нищо не се случва!

```

[!] : a c . ( a ())

| Ключова дума | Смисъл |
|--------------|--------|
|              | ,      |
|              | ,      |
| .            | -      |

[Eye] Какво виждаш: 1 ,

[Lab] Експеримент: sleep\_ms(500) sleep\_ms(100) - ,  
1 .

[!] : a a , ! e. ee (1)  
a a a c . ee (1) e e - .

- examples/01\_blink\_async.py
- examples/02\_two\_tasks.py

## Резюме

- asyncio
- await ,
- await -

## Самопроверка

1. await
- 2.
- 3.

## Упражнения

- 1.
2. await asyncio.sleep\_ms(500) time.sleep\_ms(500)

3. , 10 .

### Ако не работи

- . `await`  
`asyncio.create_task()`.
- , `asyncio.run()` .

### Нататък

, ,  
 .  
 [P ] : a c ,  
 a c cea e\_a , a e , E e 20 21 ( IV ).

## Част II: Езикови основи на MicroPython

### Глава 3. Променливи и константи

[Т ] : , с ()

| Тип | Синтаксис | Може да се промени | Употреба |
|-----|-----------|--------------------|----------|
|     |           |                    | ,        |
|     | ( )       |                    | , ,      |
|     | 2         | ( )                |          |

```
from micropython import const

BOARD_NAME = "VK_RA4M2" # описателно – може да се промени при нужда
BLINK_COUNT = const(3) # оптимизирано от компилатора – истинска константа
BLINK_DELAY_MS = const(200) # 200ms hardcoded – ясно намерение
uses_asyncio = True # булева – може да варира при runtime
```

[Р ] с ()? Мс Р с (3) 3  
RAM -

[Lab] Опитай: BLINK\_COUNT = const(3), BLINK\_COUNT = 5

- `examples/20_language_basics/02_variables_and_constants.py`

### Резюме

- ,
- `const()` - ,
- , `const()`

### Самопроверка

1. BOARD\_NAME BLINK\_COUNT = const(3)
2. - ,
3. ,

### Упражнения

1. ,
2. , MODE\_BLINK = const(1), if,
3. ,

## Ако не работи

- 
- `const()`

## Нататък

## Глава 4. Оператори, условия и цикли

[P] : ( 2 ). ( 5 ).

## Аритметични оператори

| Оператор | Действие | Пример  | Резултат |
|----------|----------|---------|----------|
| +        |          | +       | 10       |
| -        |          | -       |          |
| *        |          | *       | 21       |
| /        | ( )      | /       | 2. ...   |
| //       |          | //      | 2        |
| %        | ( )      | %       | 1        |
| **       |          | 2 ** 10 | 102      |

```
adc_raw = 3000
adc_max = 4095

voltage = adc_raw * 3.3 / adc_max # 2.42 V – реално деление
percent = adc_raw * 100 // adc_max # 73 – цяло деление, без дробна част
remainder = adc_raw % 256 # 184 – остатък, полезен за кръгови буфери
power_of_two = 1 << 12 # 4096 – но << е битов, не аритметичен (вж. по-долу)
```

[P] : / f a ( MCU). // ( ). e bedded

## Оператори за сравнение

| Оператор | Действие | Пример | Резултат |
|----------|----------|--------|----------|
| !        |          | !      |          |
|          | -        |        |          |
|          | -        |        |          |
|          | -        |        |          |
|          | -        | 10     |          |

```

temperature = 42
threshold_low = 10
threshold_high = 40

is_hot = temperature > threshold_high # True – над горния праг
is_cold = temperature < threshold_low # False – не е под долния
is_exact = temperature == 42 # True – точно 42
is_in_range = (temperature >= threshold_low) and (temperature <= threshold_high) # False – 42 > 40

```

[!] := (a = 5). == (a == 5). -

## Логически оператори

| Оператор | Действие | Пример | Резултат |
|----------|----------|--------|----------|
|          |          |        |          |
|          |          |        |          |
|          |          |        |          |

```

button_pressed = True
light_is_dark = True
door_is_open = False

Лампата светва ако е тъмно И бутонът е натиснат:
lamp_on = light_is_dark and button_pressed # True

Вентилаторът тръгва ако вратата е отворена ИЛИ е горещо:
fan_on = door_is_open or (temperature > 35) # True (42 > 35)

Аларма ако нещо НЕ е наред:
alarm = not (lamp_on and fan_on) # False – всичко е ОК

```

[P] : > a d > . : (a > 0) a d (b < 10) a > 0 a d b < 10.

## Присвояване с операция

| Оператор | Еквивалентно на | Пример |
|----------|-----------------|--------|
| +        | + 1             | + 1    |
| -        | - 1             | - 10   |
| *        | * 2             | * 2    |
| //       | // 2            | // 2   |
| &        | &               | & 0 0  |
|          |                 |        |
|          |                 | 0 01   |
|          |                 |        |
|          |                 |        |

```

count = 0
count += 1 # 1 – най-честият случай: брояч
count += 1 # 2

flags = 0b00000000
flags |= 0x80 # 0b10000000 – задаваме бит 7
flags &= ~0x80 # 0b00000000 – изчистваме бит 7

```

## Условия: if / elif / else

```

left_value = 7
right_value = 3

if left_value < right_value:
 print("По-малка")
elif left_value == right_value:
 print("Равна")
else:
 print("По-голяма") # ← това се изпълнява

```

## Трите форми на цикли:

```

for index in range(3): # 0, 1, 2 – знаем колко пъти
 print(index)

counter = 0
while counter < 3: # докато условие е вярно
 print(counter)
 counter += 1

while True: # do-while: изпълни, после провери
 print("поне веднъж")
 if True:
 break

```

[!] : c /ca e M c P . f/e f/e e  
( 9).

**[Lab] Опитай:** left\_value = right\_value = 5

print(f"{left\_

value} vs {right\_value}”) if, .

- examples/20\_language\_basics/03\_operators\_conditions\_loops.py

## Резюме

- if/elif/else for while
- while True + break ' do-while.
- break .

## Самопроверка

1. ,
2. while True
3. if/elif/else ,

## Упражнения

1. left\_value right\_value , if/el
2. , 5 1, START.
3. while True break, .

## Ако не работи

- ,
- .

## Битови оператори и шестнадесетичен запис

[T ] : (+, -, ==) ,

```
a = 0b11001010 # Двоичен запис: 202 в десетичен вид
b = 0xFF # Шестнадесетичен запис: 255

Основни битови оператори:
print(a & 0xF) # AND маска – взима само долните 4 бита → 0b1010 = 10
print(a | 0x01) # OR – задава бит 0 → 0b11001011 = 203
print(a ^ 0xFF) # XOR – обръща всички битове → 0b00110101 = 53
print(~a & 0xFF) # NOT (инверсия) – обръща битовете → 53
print(a << 2) # Изместване наляво с 2 → умножение по 4
print(a >> 4) # Изместване надясно с 4 → горните 4 бита → 0b1100 = 12
```

| Оператор | Действие | Пример          | Резултат   |
|----------|----------|-----------------|------------|
| &        | ( )      | 0 1100 & 0 1010 | 0 1000     |
|          | ( )      | ( )             | 0 1100     |
|          | ( )      | 0 1100 0 1010   | 0 0110     |
|          | ( )      | 0 0000 & 0      | 0 11111111 |
|          |          | 1               | 0 1000     |
|          |          | 0               | 0 1111 1   |

[P ] : 0b = , 0 = . 0 FF = 255 = 0b11111111.  
e bedded .

```
history = ((history << 1) | button.value()) & 0xFF # Измества историята и добавяме новото четене
if history == 0x00: # 8 последователни 0 → стабилно натиснат
 print("PRESS")
```

## Нататък

## Глава 5. Enums в MicroPython

[T ] : 0, 1, 2 STATE\_RED, STATE\_YELLOW,  
STATE\_GREEN - . E .

Без enum: if state == 2: # Какво означава 2? [?]  
С enum: if state == STATE\_BLINK: # [OK] Веднага ясно!

```
from micropython import const

STATE_OFF = const(0) # LED изключен
STATE_ON = const(1) # LED включен
STATE_BLINK = const(2) # LED мига

Таблица за четимо описание (полезна при print/debug):
STATE_NAMES = {0: "OFF", 1: "ON", 2: "BLINK"}

state = STATE_BLINK
print("Текущо:", STATE_NAMES[state]) # → "Текущо: BLINK"
```

[P ] e .E ? M c P e . c ()  
- - e bedded.

[Lab] Опитай: STATE\_ERROR = const(3) STATE\_NAMES. if/elif .

- examples/20\_language\_basics/04\_enums.py

## Резюме

- 
- STATE\_X = const(N) + dict
- - , - ,

## Самопроверка

1. const() enum
2. , STATE\_BLINK, 2
3. STATE\_OFF, STATE\_ON, STATE\_BLINK

## Упражнения

1. , STATE\_ERROR STATE\_IDLE,
2. if/elif/else,
3. , ,

## Ако не работи

- ,
- dict,

## Нататък

, ,

## Глава 6. Масиви и таблици

[T] : , a a  
b ea a

| Тип      | Пример        | RAM | Когато     |
|----------|---------------|-----|------------|
|          | 100, 200, 00  |     | ,          |
| ( , ...) | ( , 0, 1000)  | 2 - | 1 - ,      |
|          | ( 0 10, 0 20) |     | / 2 , 2 12 |

```
from array import array
```

```
adc_samples = [1200, 1215, 1198, 1222] # list – гъвкав
led_table = [["LED1", "P204"], ["SW1", "P400"]] # вложен list = таблица
pwm_duty_values = array("H", [0, 16384, 32768, 65535]) # 16-бит, 2× по-малко RAM
raw_bytes = bytearray([0x10, 0x20, 0x30, 0x40]) # суров буфер за протоколи
```

[Eye] Визуализация на паметта:

```
list [1200, 1215]: [ptr→1200][ptr→1215] ← всеки елемент е обект
array("H", [1200]): [0x04B0][0x04B0] ← директно 2 байта на елемент
bytearray([0x10]): [0x10] ← 1 байт на елемент
```

**[Lab] Опитай:** `import micropython; micropython.mem_info()`  
`.( micropython`

list array 100  
**Глава**

30.)

- `examples/20_language_basics/05_arrays.py`

## Резюме

- list , .
- array .
- bytearray , .

## Самопроверка

1. list, array, bytearray
2. , ,
3. array("H", ...) bytearray(...)

## Упражнения

1. , .
2. , LED1, SW1, P014, P015, .
3. bytearray .

## Ако не работи

- IndexError.
- array - , "H" 1 - .

## Нататък

## Глава 7. Структури в MicroPython

[ П ] : с C a е, е, е a . Мс Р

| Вариант | Синтаксис | Когато |
|---------|-----------|--------|
|         | 1, 20     | ,      |
|         | (, (, ))  | - ,    |
|         | ...       | +      |

```

from collections import namedtuple

namedtuple – компактна read-only структура
SensorRecord = namedtuple("SensorRecord", ("name", "pin", "unit"))
temp = SensorRecord("Temp", "P000", "°C")
print(temp.name, temp.pin) # достъп по атрибут

dict – гъвкава структура
light_sensor = {
 "name": "Light",
 "adc_pin": "P000",
 "limits": {"dark": 800, "bright": 3000} # структура от структури
}

списък от структури – C-style масив от struct
devices = [
 {"name": "LED1", "pin": "P204", "kind": "output"},
 {"name": "SW1", "pin": "P400", "kind": "input"},
]
for d in devices:
 print(d["name"], "→", d["pin"])

```

*[P ] : ( a ) a ed e ,*  
*dc .*

- `examples/20_language_basics/06_structures.py`

## Резюме

- ' struct dict, namedtuple .
- namedtuple - - dict .
- dict ( ).

## Самопроверка

1. dict, namedtuple dict
2. -
3. , ,

## Упражнения

1. devices
2. light\_sensor , limits["alarm"].
3. , , .

## Ако не работи

- `record["name"]`
- `namedtuple`.
- `record["name"]`

## Класове — бърз старт

[Т] : dc , ( ) ( ) .

RetroSynth, TouchPad, AudioADC.

```
from machine import Pin

class StatusLed:
 def __init__(self, pin_name="LED1"): # Конструктор — извиква се при създаване.
 self.pin = Pin(pin_name, Pin.OUT, value=1) # self.pin = вътрешно състояние на обекта.

 def on(self): # Метод — действие върху обекта.
 self.pin.value(0) # Active-low: 0 = светва.

 def off(self): # Метод — действие върху обекта.
 self.pin.value(1) # 1 = гаси.

led = StatusLed("LED1") # Създаваме обект (инстанция на класа).
led.on() # Извикваме метод — LED светва.
led.off() # LED гасне.
```

| Елемент  | Какво е | Пример |
|----------|---------|--------|
|          | ( )     |        |
| ( , ...) | ()      |        |
|          |         |        |
| . ()     |         |        |

[Р] : 50

( ). , .

## Нататък

## Глава 8. Указатели и еквивалентите им в MicroPython

[Т ] : , . е е .

```
alias_list = adc_list: [100][200][300]
 ↑ ↑
 adc_list alias_list ← ЕДИН обект, ДВЕ имена

adc_copy = list(adc_list): [100][200][300] ← отделно копие
 ↑ ↑
 adc_list adc_copy ← ДВА различни обекта
```

```
adc_list = [100, 200, 300]
alias_list = adc_list # НЕ е копие – второ име към същия обект
alias_list[1] = 999
print(adc_list) # [100, 999, 300] – промяната се вижда!

tx_buffer = bytearray([1, 2, 3, 4])
tx_view = memoryview(tx_buffer) # прозорец – без копиране
tx_view[2] = 77
print(tx_buffer) # bytearray(b'\x01\x02\x03\x04') – промяна!
```

[!] : `adc_c = (adc_ )` . `adc_a a = adc_` !

**[Lab] Опитай:** `print(id(adc_list) == id(alias_list))` True. `print(id(adc_list) == id(list(adc_list)))` False.

- `examples/20_language_basics/07_pointers_equivalents.py`
- `examples/28_machine_misc/02_machine_mem_and_bootloader.py`

## Резюме

- `alias = original` , **ЕДИН** .
- `copy = list(original)` **ДВА** .
- `memoryview` .

## Самопроверка

1. `alias_list = adc_list` ,
2. `memoryview`,
3. `machine.mem32`

## Упражнения

1. ,
2. `bytearray`, `memoryview` ,
3. .

## Ако не работи

- , .
- mem32 .

## Нататък

## Глава 9. Функции, callbacks и таблици от функции

[Т] : , . Ca bac  
:

Директно извикване: add\_values(7, 3) → 10 веднага  
 Callback: run\_callback(print, 42) → print(42) = “42”  
 Function table: ops[1](5) → square(5) = 25

```
def add_values(a, b):
 return a + b

def run_callback(callback, value):
 callback(value) # не знаем коя функция – просто я викаме

def square(value):
 return value * value

def double(value):
 return value * 2

Таблица от функции – C-style function pointer table
operation_table = [square, double]
print(operation_table[0](5)) # square(5) = 25
print(operation_table[1](5)) # double(5) = 10
```

[P] e bedded : Ca bac IRQ a de , , FSM (  
42-47). c -ca e.

**[Lab] Опитай:** run\_callback(print, “hello”) print . run\_callback  
k(square, 7).

- examples/20\_language\_basics/08\_functions.py

## Резюме

- / .
- - .
- .

## Самопроверка

- 1.
- 2.
- 3.

## Упражнения

1. operation\_table , .
2. run\_selected(index, value), .
3. , , .

## Ако не работи

- (square),  
(square(5)).
- . operatio  
n\_table[i].\_\_name\_\_.

## Нататък

,  
.

## Част III: Цифрови и аналогови входове и изходи

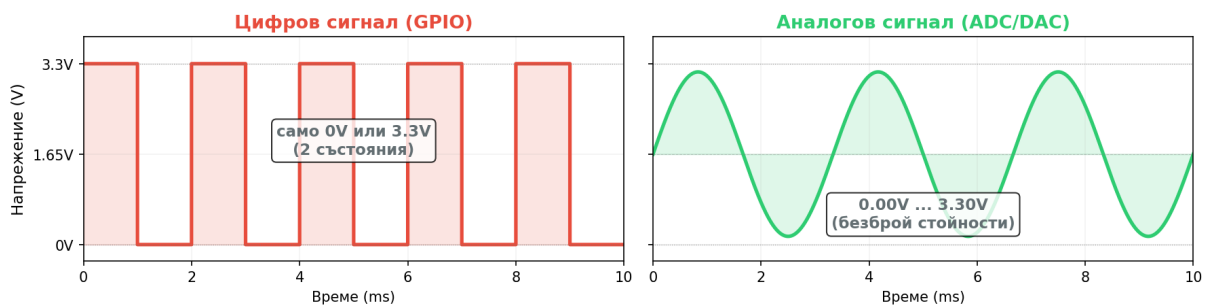
обменят информация с

околния свят.

Какво е информация?

- Цифрова  $0 \quad 1$  (  $\quad / \quad , \quad / \quad$  ).
- Аналогова  $0.00V \dots 3.30V$  (  $\quad , \quad$  ).

Два вида информация: цифрова (0/1) и аналогова (непрекъснат диапазон)



| Блок | Посока | Тип информация | Какво прави | Пример |
|------|--------|----------------|-------------|--------|
|      |        |                | 0 1         | ,      |
|      |        |                | 0 1         | ,      |
|      |        |                |             | ,      |
|      |        |                |             | ,      |
|      |        | (              | ) 0/1       | ,      |

[P ] : GPIO (0/1). ADC DAC  
ADC ( ), DAC ( ). PWM

## Глава 10. Цифров изход

1 P .OUT. - GPIO  
: LED1 ас е- ( . 0 ).

```

from machine import Pin
import time

led = Pin("LED1", Pin.OUT, value=1) # value=1 → изгасен при старт (active-low)
led.value(0) # [Tip] светваме
time.sleep_ms(700)
led.value(1) # [X] гасим

```

[!] : LED1 = P204, ас е- 0 , 1 . GPIO

[Eye] Какво виждаш: 1 00 .

[Lab] Експеримент: 700 100 ( 0 .)

- 0
- 1
- -
- 

- examples/21\_digital\_io/01\_digital\_output.py
- examples/01\_blink\_async.py

## Резюме

- Pin.OUT , value(0) value(1) .
- LED1 - , 0 , 1 .
- ,

## Самопроверка

1. LED1 value(0), value(1)
2. Pin.OUT
3. , - ,

## Упражнения

1. blink\_n(count, on\_ms, off\_ms), 1 .
2. print("LED ON") print("LED OFF"), .
3. .

## Ако не работи

- 1 Pin("LED1", Pin.OUT)
- -
- , .
- .

## Нататък

## Глава 11. Цифров вход

[Т ] : PULL\_UP . PULL\_UP  
1 ( ). 0.

Без PULL\_UP: P400 — BTN — GND → floating = шум x  
С PULL\_UP: 3.3V —[R]— P400 — BTN — GND  
|ненатиснат: 1 (pull-up печели)  
|натиснат: 0 (GND печели)

| Състояние | button.value() | Смисъл |
|-----------|----------------|--------|
|           | 1              |        |
|           | 0              |        |

```
from machine import Pin

button = Pin("SW1", Pin.IN, Pin.PULL_UP)
print(button.value()) # 1 = пуснат, 0 = натиснат
```

[Eye] Какво виждаш: 1

0

[Lab] Опитай:

while True: print(button.value()).

1

[!] : SW1 = P400, ac e- . deb ce  
17.

0-

- 
- 
- PULL\_UP
- PULL\_DOWN

- examples/21\_digital\_io/02\_digital\_input.py

## Резюме

- Pin.IN , Pin.PULL\_UP
- SW1 , 0, 1.
-

## Самопроверка

1. `Pin.PULL_UP,`
2. `button.value() == 0` `SW1`
3. `print(button.value())`

## Упражнения

1. `PRESSED` `RELEASED,`
2. `1` `,`
3. `,`

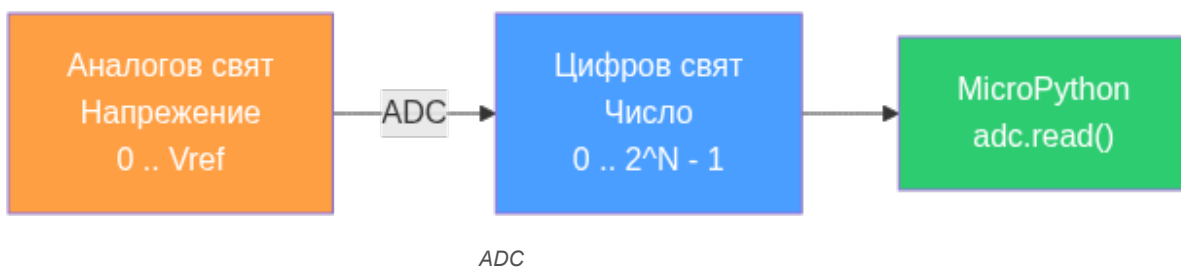
## Ако не работи

- `Pin.PULL_UP`
- `SW1` `-` `0.`
- `,`

## Нататък

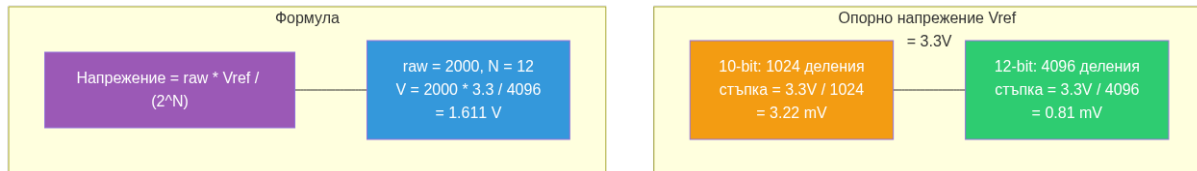
## Глава 12. Аналогов вход и ADC

( - - ) аналоговия цифровия



## Опорно напрежение и разрядност

- Опорно напрежение (Vref) `VK_RA4M2` **3.3V.**
- Разрядност (N бита) `10` `2 10` **1024**
- `12` `2 12` **4096**



ADC

## Формулата

$$\text{Напрежение (V)} = \frac{\text{raw} \times \text{Vref}}{2^N}$$

- raw (adc.read()), 0 2^N - 1
- Vref ( 2 3.3V)
- 2^N ( 12- 4096)

$$[P] : a / 2^N \quad ( 0.0 \quad 1.0).$$

$$a = 4095 \quad 4095 \quad 3.3 / 4096 \quad 3.3V ( a \quad 2^N \quad Vef).$$

Пример: , 10 (102 ), adc.read() raw = 512

$$V = 512 \times 3.3 / 1024 = 1.65V \quad (\text{точно средата на диапазона})$$

Пример 2: , raw = 768

$$V = 768 \times 3.3 / 1024 = 2.475V$$

Пример 3: 12- ( 2) raw = 2000

$$V = 2000 \times 3.3 / 4096 = 1.611V$$

$$[P] : a / 2^N \quad , \quad ADC : = Vef / 2^N$$

$$10\text{-b} : 3.3V / 1024 = 3.22 \text{ V} \quad 1.000V \quad 1.002V. \quad 12\text{-b} : 3.3V / 4096 = 0.81 \text{ V}$$

| Резолюция | Стъпки (2^N) | Стъпка при Vref=3.3V | Употреба |
|-----------|--------------|----------------------|----------|
| -         | 2            | 12.                  | ,        |
| 10-       | 102          | .2                   | /        |
| 12-       | 0            | 0. 1                 | 2        |

```
Преобразуване на raw ADC стойност в напрежение:
VREF = 3.3
BITS = 12
raw = adc.read() # число 0..4095
voltage = raw * VREF / (1 << BITS) # 1 << 12 = 4096
print(f"{raw} → {voltage:.3f} V") # напр. "2000 → 1.611 V"
```

```
from machine import ADC

temperature_adc = ADC(ADC.CORE_TEMP) # вътрешен сензор – без окабеляване
vref_adc = ADC(ADC.CORE_VREF) # вътрешно референтно напрежение

print(temperature_adc.read()) # число 0-4095 (при 12-bit)
print(vref_adc.read())
```

**[Eye] Какво виждаш:**

2000-2 00 CORE\_TEMP

**[Lab] Опитай:**

CORE\_TEMP

20- 0

```
[/] : b = ADC
```

RA4M2

- ADC12, 12 bit, 12 / 10 / 8 bit.
- ADC(pin\_or\_channel, bits=8), ADC(..., bits=10), ADC(..., bits=12).
- bits

```
from machine import ADC # Импортираме ADC за избор на резолюция при създаване на обекта.
```

```
adc_fast = ADC("P000", bits=8) # Създаваме ADC обект с 8-битова резолюция за бързо четене.
```

```
adc_balanced = ADC("P001", bits=10) # Създаваме ADC обект с 10-битова резолюция за баланс бързина/точност.
```

```
adc_precise = ADC("P002", bits=12) # Създаваме ADC обект с 12-битова резолюция за максимална точност.
```

```
print(adc_fast.read(), adc_balanced.read(), adc_precise.read()) # Четем и трите канала и показваме резулта...
```

VK\_RA4M2

- ADCLK = 50 MHz / 0.31 us
- 8 bit, 0.35 us 10 bit, 0.39 us 12 bit
- - , / ADSSTR, 0.41 us, 0.45 us, 0.49 us
- , ADC.read()
- , time.ticks\_us()
- ADC.read() examples/23\_timing/07\_adc\_timing\_measure.py
- 8 / 10 / 12 bit

P000, P001, P002, P003, P004, P005, P006, P007, P008, P013, P014, P015, P500

- `examples/22_analog/01_analog_input_adc.py`
- `examples/23_timing/07_adc_timing_measure.py`

## Резюме

- `ADC.read()`
- `ADC.CORE_TEMP` `ADC.CORE_VREF`
- `bits=8/10/12`
- `ADC.read()`

## Самопроверка

1. `ADC.CORE_TEMP` `P000`
2. `bits=`
3. `ADC.read()`

## Упражнения

1. `bits=8, bits=10 bits=12`
- 2.
3. `ADC.read()` `time.ticks_us()`

## Ако не работи

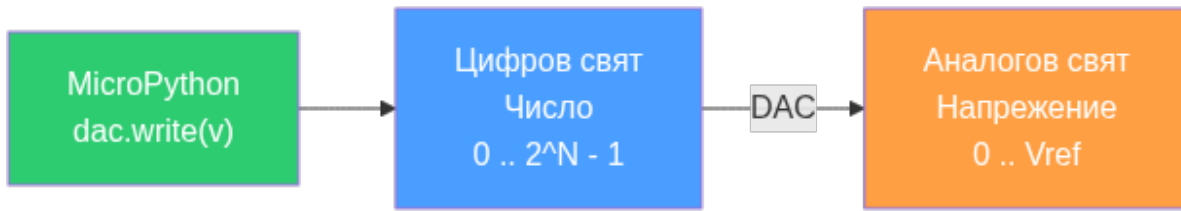
- 
- 
- 

## Нататък

ADC

## Глава 13. DAC изход, timed playback и retro synth

( - - ) обратното на ADC



DAC

## Опорно напрежение и разрядност (DAC)

опорно напрежение (Vref)    разрядност (N бита)

- Vref , . 2 3.3V.
- Разрядност . 12- 4096 нива.

## Формулата (обратна на ADC)

$$\text{Напрежение (V)} = \frac{\text{value} \times V_{\text{ref}}}{2^N}$$

- value , (dac.write(value)), 0 2^N - 1
- Vref (3.3V)
- 2^N (4096 12- )

[P ] : a e / 2^N . V ef

a e = 0 0 3.3 / 4096 = 0V ( ). a e = 2048 2048

3.3 / 4096 = 1.65V ( " " ). a e = 4095 4095 3.3 / 4096 3.3V ( ).

| value | Напрежение | Смисъл за аудио |
|-------|------------|-----------------|
| 0     | 0.0        |                 |
| 20    | 1.         | ( )             |
| 0     | .          |                 |

```

Обратната формула – от желано напрежение към DAC стойност:
VREF = 3.3
BITS = 12
desired_voltage = 1.0
value = int(desired_voltage * (1 << BITS) / VREF) # 1.0 × 4096 / 3.3 = 1241
dac.write(value) # → 1.0V на пина

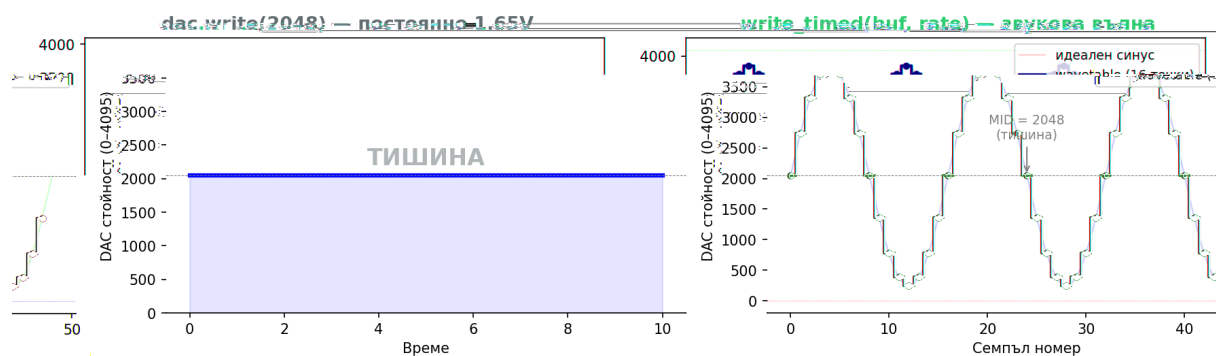
```

## От статично ниво до звук

фиксирана скорост

масив от числа с

## DAC: статично ниво vs звукова вълна



DAC:

• Ляво: `dac.write(2048)`

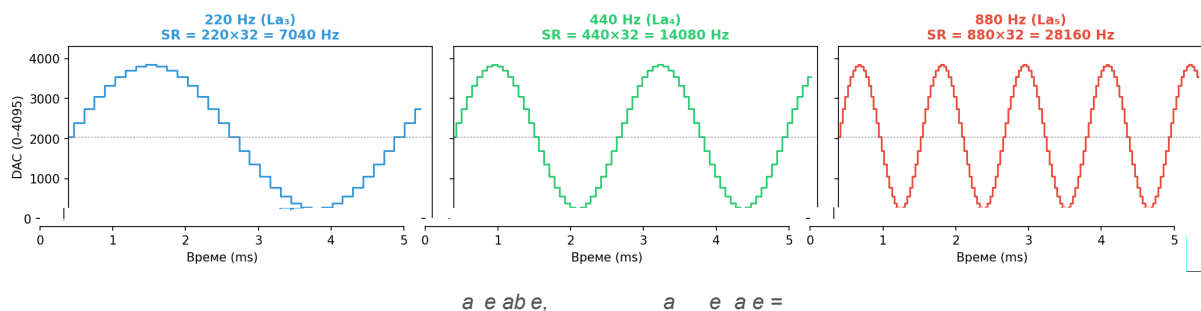
1.

• Дясно: 1

!

## Sample rate — колко бързо DAC сменя стойностите

Една и съща wavetable, различен sample rate = различна честота



## Sample rate

$$\text{sample\_rate} = \text{честота\_на\_тона} \times \text{семпли\_за\_един\_период}$$

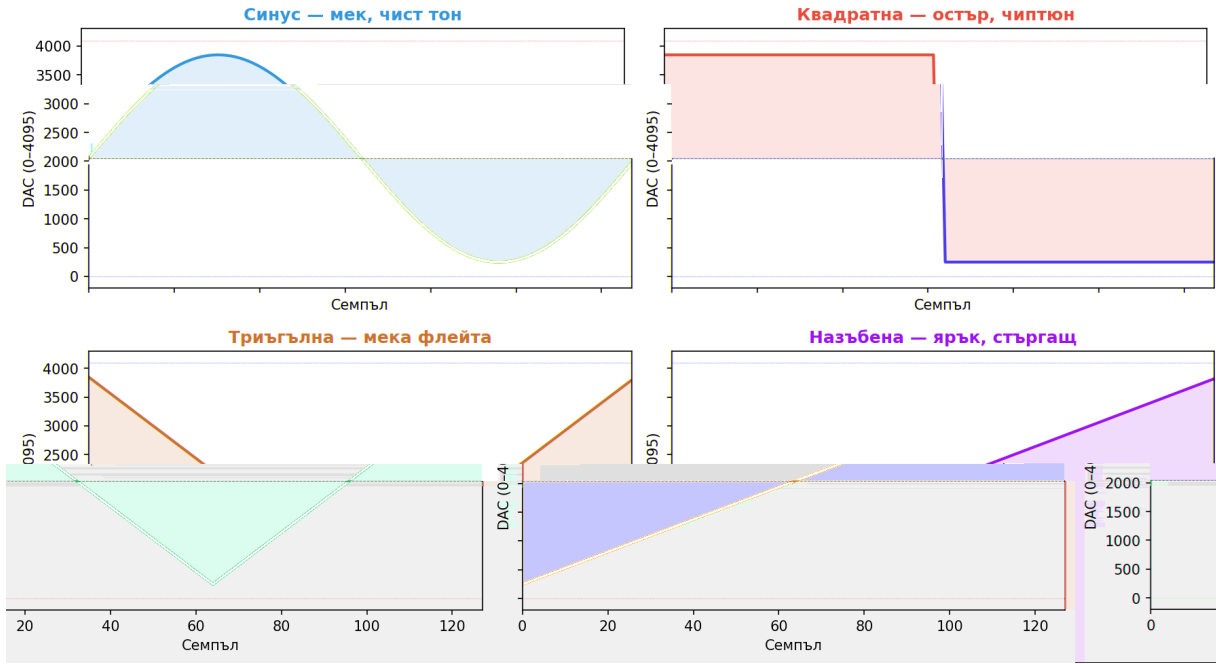
Пример: 0 ( ). - 12

$$\text{sample\_rate} = 440 \times 128 = 56\,320 \text{ Hz}$$

56 320 пъти в секунда. ( ) ,

## Четири вълнови форми от числа

Четири вълнови форми — 128 семпла, 12-bit DAC



128 , 12-b DAC

(0 0 ) ,

( 20 ).

| Форма | Звук  | Как се изчислява | Хармоници  |
|-------|-------|------------------|------------|
|       | ,     | . ( )            |            |
|       | , " " | 0                | (1, , ...) |
|       |       | -                | ,          |
|       | ,     | +                |            |

| Метод       | Кора | Кой движи данните |
|-------------|------|-------------------|
| . ( )       |      | ( )               |
| . ( , , , ) | 2    |                   |
| . ( , , , ) | 102  |                   |

Hardware DAC pipeline: AGT → DTC → DAC → P014



На d a e DAC e e: AGT DTC DAC P014

- ( , , )
- 
- 
- coin, jump, laser, explosion
- 

2

- P014 = DA0 = J19-5 = A4
- P015 = DA1 = J19-6 = A5
- J16 - не е
- ,

Статичен DAC изход

```
from machine import DAC, Pin # Импортираме DAC и Pin за аналогов изход.

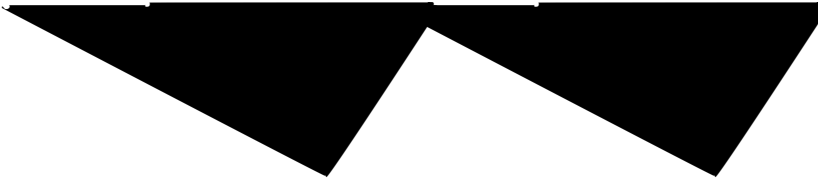
dac = DAC(Pin("P014")) # Създаваме DAC обект върху DA0 (J19-5 / A4).
dac.write(2048) # Записваме средна стойност (2048 от 4095 = ~1.65 V при 3.3 V AVCC0).
dac.write(4095) # Максимална стойност = ~3.3 V.
dac.write(0) # Минимална стойност = 0 V.
```

Timed playback — основна идея

- DAC.write\_timed(...)
- AGT
- DTC DMAC
- - ,

|       | DTC | DMAC |
|-------|-----|------|
| ( - ) |     |      |
| ( )   | 2   | 102  |

DMAC



```

from array import array # Импортираме array за ефективен буфер.
from machine import DAC, Pin # Импортираме DAC и Pin.
import time # Импортираме time за sleep.

wave = array("H", [2048, 3072, 4095, 3072, 2048, 1024, 0, 1024]) # 12-bit DAC стойности за един период.
dac = DAC(Pin("P014")) # Ползваме DA0 на J19-5 / A4.

dac.write_timed(
 wave,
 440 * len(wave), # Един период на 440 Hz = sample rate 3520 Hz.
 mode=DAC.CIRCULAR, # Безкраен loop.
 transfer=DAC.TRANSFER_DTC, # Ползваме DTC (до 256 семпла).
)

time.sleep_ms(500) # Свирим 500 ms.
dac.stop() # Спираме timed playback.

```

### Пример: chirp\_20k\_10k.py — непрекъснат 5 kHz синус (wavetable loop)

```

[P] : c _20_10 . , 5 H
 DTC c c a , ee (c).
 c , a _ ee () e _ . (56).

```

```

import gc # Импортираме gc за управление на garbage collector.
import math # Импортираме math за sin() функцията.
from array import array # Импортираме array за ефективен буфер с unsigned short стойности.
from machine import DAC, Pin # Импортираме DAC и Pin за аналогов изход.

DAC_PIN = "P014" # Избираме DA0 – J19-5 / A4.
MID = 2048 # Средна точка на 12-bit DAC (0..4095).
AMP = 1800 # Амплитуда на синусоидата.
FREQ = 5000 # Целева честота на синуса: 5 kHz.
TABLE_LEN = 16 # Брой семпли за един период.
SAMPLE_RATE = FREQ * TABLE_LEN # Изчисляваме sample rate: 5000 * 16 = 80 000 Hz.

buf = array("H", [0] * TABLE_LEN) # Създаваме буфер от 16 unsigned short стойности.
for i in range(TABLE_LEN): # Запълваме буфера с един период на синусоидата.
 v = MID + int(AMP * math.sin(2.0 * math.pi * i / TABLE_LEN)) # Изчисляваме DAC стойност.
 if v < 0: # Ограничаваме отдолу до 0.
 v = 0
 elif v > 4095: # Ограничаваме отгоре до 4095.
 v = 4095
 buf[i] = v # Записваме в буфера.

dac = DAC(Pin(DAC_PIN)) # Създаваме DAC обект.
dac.write(MID) # Задаваме начално ниво в средата.

gc.collect() # Принуждаваме garbage collection преди timed playback.
gc.disable() # Забраняваме GC, за да не прекъсва DMA трансфера (вж. бележката по-долу).
print("5 kHz sine, DTC circular (Ctrl-C to stop)...") # Информираме потребителя.
dac.write_timed(buf, SAMPLE_RATE, mode=DAC.CIRCULAR, transfer=DAC.TRANSFER_DTC) # Стартираме безкраен DTC ...

try:
 while True: # Чакаме безкрайно – хардуерът свири самостоятелно.
 pass
except KeyboardInterrupt: # Ctrl-C спира цикъла.
 pass

dac.stop() # Спираме timed playback.
dac.write(MID) # Връщаме изхода в средата.
gc.enable() # Разрешаваме GC отново.
print("Stopped.") # Информираме, че е спрял.

```

examples/chirp\_20k\_10k.py

### Пример: dac\_dual\_dtc\_dmac.py — двуканален DAC: DTC vs DMAC circular

```

import gc # Управление на garbage collector.
import math # Математически функции за синус.
from array import array # Ефективен буфер за DAC семпли.
from machine import DAC, Pin # DAC и Pin класове.

MID = 2048 # Средна точка на 12-bit DAC.
AMP = 1800 # Амплитуда.
FREQ = 5000 # Целева честота: 5 kHz.
TBL = 32 # Брой семпли за един период – 32.
SR = FREQ * TBL # Sample rate: 5000 * 32 = 160 kHz.

buf = array("H", [0] * TBL) # Буфер за 32 семпла.
for i in range(TBL): # Запълваме с един период синус.
 v = MID + int(AMP * math.sin(2.0 * math.pi * i / TBL)) # Стойност за текущия семпл.
 buf[i] = max(0, min(4095, v)) # Ограничаваме в 0..4095 с max/min.

print("5 kHz sine, {} samples @ {} Hz".format(TBL, SR)) # Отпечатваме параметрите.

dac0 = DAC(Pin("P014")) # DA0 на J19-5 / A4.
dac1 = DAC(Pin("P015")) # DA1 на J19-6 / A5.
dac0.write(MID) # Начално ниво CH0.
dac1.write(MID) # Начално ниво CH1.

gc.collect() # Почистваме паметта.
gc.disable() # Забраняваме GC по време на playback.

print("CH0=DTC circular | CH1=DMAC circular (Ctrl-C to stop)") # Инфо за двата канала.

dac0.write_timed(buf, SR, mode=DAC.CIRCULAR, transfer=DAC.TRANSFER_DTC) # CH0: DTC repeat до 256 семпла, ...
dac1.write_timed(buf, SR, mode=DAC.CIRCULAR, transfer=DAC.TRANSFER_DMAC) # CH1: DMAC repeat до 1024, ISR r...

try:
 while True: # Чакаме – и двата канала свирят самостоятелно.
 pass
except KeyboardInterrupt: # Ctrl-C спира.
 pass

dac0.stop() # Спираме CH0.
dac1.stop() # Спираме CH1.
dac0.write(MID) # Връщаме CH0 в средата.
dac1.write(MID) # Връщаме CH1 в средата.
gc.enable() # Разрешаваме GC.
print("Stopped.") # Готово.

```

examples/dac\_dual\_dtc\_dmac.py

```
[P] c.d a b e() DMA/DTC a b a c ? M c P
Ga b a e C e c (GC)
GC CPU , DTC/DMAC
 GC
1. c.c e c() GC , e a - 2. c.d a b e()
 GC. 3. DMA a b a c . 4. c.e a b e() GC
a b a c . c 30 ().
```

### Пример: retro\_synth.py — библиотека за ретро звуци

```
[P] Re S : Re S (7),
 DTC/DAC : . a _ e("A4", 200, " a e")
La 200 . a e a b e (),
 DTC dac. e_ ed()
 56.
```

, **helper** модул,  
RetroSynth.

```
MID = 2048 # Средна точка на 12-bit DAC.
DEFAULT_AMP = 1400 # Амплитуда по подразбиране.
DEFAULT_TABLE_LEN = 128 # Семпли за един период.
DEFAULT_NOISE_BURST_LEN = 2048 # Дължина на noise burst буфера.
```

- clamp12(value) 0.. 0 .
- midi\_to\_freq(midi\_note) 440 \* 2^((n-69)/12).
- note\_to\_freq(note\_or\_freq) " ", " # ", 0, "1000 "
- semitone\_freq(base, semitones)
- make\_square(length, amp, duty\_num, duty\_den)
- make\_triangle(length, amp)
- make\_saw(length, amp)
- make\_noise\_burst(length, amp)

RetroSynth

```
synth = RetroSynth("P014") # Създаваме synth обект на DA0.
synth.play_note("A4", 200, "square") # Свирим нота A4 за 200 ms, квадратна вълна.
synth.play_sweep("A3", "A6", 300, "saw") # Sweep от A3 до A6 за 300 ms.
synth.play_arpeggio("C4", (0, 4, 7), step_ms=60) # Арпеджо C-E-G.
synth.play_melody([("\E5", 110), ("\B4", 110)]) # Поредица от ноти.
synth.coin() # Звук "монета" (B5 + E6).
synth.jump() # Звук "скок" (sweep нагоре).
synth.laser() # Звук "лазер" (sweep надолу).
synth.explosion() # Звук "експлозия" (noise burst).
synth.stop() # Спира всичко.
```

examples/retro\_synth.py

## Пример: dac\_retro\_sfx.py — четири game звука

```
import time # Импортираме time за паузи между звуците.
from retro_synth import RetroSynth # Импортираме helper модула.

DAC_PIN = "P014" # DA0 на J19-5 / A4.
synth = RetroSynth(DAC_PIN) # Създаваме synth обект.

def demo(): # Дефинираме демо функцията.
 print("DAC retro SFX demo on {} (J19-5 / A4 / DA0)".format(DAC_PIN)) # Печатаме пин информация.
 print("coin -> jump -> laser -> explosion") # Печатаме последователността.
 synth.coin() # Звук "монета": две бързи ноти B5 и E6.
 time.sleep_ms(120) # Пауза 120 ms.
 synth.jump() # Звук "скок": sweep от A3 до F5.
 time.sleep_ms(120) # Пауза 120 ms.
 synth.laser() # Звук "лазер": sweep от A6 до A3.
 time.sleep_ms(120) # Пауза 120 ms.
 synth.explosion() # Звук "експлозия": LFSR noise burst.
 synth.stop() # Спираме DAC изхода.
 print("Done.") # Готово.

demo() # Пускаме демото.
```

examples/dac\_retro\_sfx.py

## Пример: dac\_retro\_music.py — мелодия и арпеджо

```

from retro_synth import RetroSynth # Импортираме helper модула.

DAC_PIN = "P014" # DA0 на J19-5 / A4.
synth = RetroSynth(DAC_PIN) # Създаваме synth обект.

MELODY = [# Дефинираме мелодия като списък от (нота, ms, вълна).
 ("E5", 110, "pulse25"), # Нота E5 за 110 ms с 25% duty pulse.
 ("B4", 110, "pulse25"), # Нота B4 за 110 ms.
 ("C5", 110, "pulse25"), # Нота C5 за 110 ms.
 ("D5", 110, "pulse25"), # Нота D5 за 110 ms.
 ("C5", 110, "pulse25"), # Нота C5 за 110 ms.
 ("B4", 110, "pulse25"), # Нота B4 за 110 ms.
 ("A4", 180, "pulse25"), # Нота A4 за 180 ms (по-дълга).
 (None, 40, "pulse25"), # Пауза 40 ms (None = тишина).
 ("A4", 110, "pulse25"), # Нота A4 за 110 ms.
 ("C5", 110, "pulse25"), # Нота C5 за 110 ms.
 ("E5", 180, "pulse25"), # Нота E5 за 180 ms.
 ("D5", 110, "pulse25"), # Нота D5 за 110 ms.
 ("C5", 110, "pulse25"), # Нота C5 за 110 ms.
 ("B4", 220, "pulse25"), # Нота B4 за 220 ms (финална).
]

def demo(): # Дефинираме демо функцията.
 print("Retro music demo on {} (J19-5 / A4 / DA0)".format(DAC_PIN)) # Печатаме пин информация.
 print("melody -> arpeggio bed -> coin") # Печатаме последователността.
 synth.play_melody(MELODY, gap_ms=12) # Свири мелодията с 12 ms пауза между нотите.
 synth.play_arpeggio("A3", (0, 4, 7, 12), step_ms=55, cycles=6, wave="triangle", gap_ms=5) # Арпеджо A-...
 synth.coin() # Финален "coin" звук.
 synth.stop() # Спираме DAC изхода.
 print("Done.") # Готово.

demo() # Пускаме демото.

```

examples/dac\_retro\_music.py

## Пример: test\_dac\_dmac\_dtc.py — автоматичен тест на DTC и DMAC режими

1. DTC circular 12 - , 00 , .
2. DTC one-shot 2 - , , .
3. DMAC one-shot 20 - 12 , , .

```

def wait_until_idle(dac, timeout_ms=2500): # Чака DAC да спре да свири.
 start = time.time() # Запомняме началото.
 deadline = time.time() + timeout_ms # Крайно време.
 while dac.playing(): # Докато playback е активен...
 if time.time() - start >= timeout_ms: # Ако таймаутът изтече...
 raise RuntimeError("timeout waiting for timed DAC to stop") # Грешка.
 time.sleep(10) # Чакаме по 10 ms.
 return time.time() - start # Връщаме elapsed ms.

```

ALL PASS.

RuntimeError

examples/test\_dac\_dmac\_dtc.py

### Пример: dac\_firmware\_probe.py — probe за firmware валидация

test\_dac\_dmac\_dtc.py, **firmware debugging**

- , - .
- PASS: .
- RuntimeError .
- , J16 - , .

examples/dac\_firmware\_probe.py

### Пример: dac\_dmac\_diag.py — регистрова диагностика на DMAC

**напреднали**

mem32[] / mem8[]

```
ICU_BASE = 0x40006000 # Interrupt Controller Unit базов адрес.
DMAC_BASE = 0x40005000 # DMAC базов адрес.
DMA_BASE = 0x40005200 # DMA Common Register базов адрес.
BUS_BASE = 0x40003000 # Bus Error базов адрес.
```

- DMAST
  - DMECHR ( ).
  - BUS3ERRSTAT / DMACDTCERRSTAT
  - IELSR[46] 0 .
  - DELSR[0..7] .
  - DMCRA, DTE, DMREQ, DMSTS .
- , 0 2 0 ,

examples/dac\_dmac\_diag.py

### Нататък

(-1)",

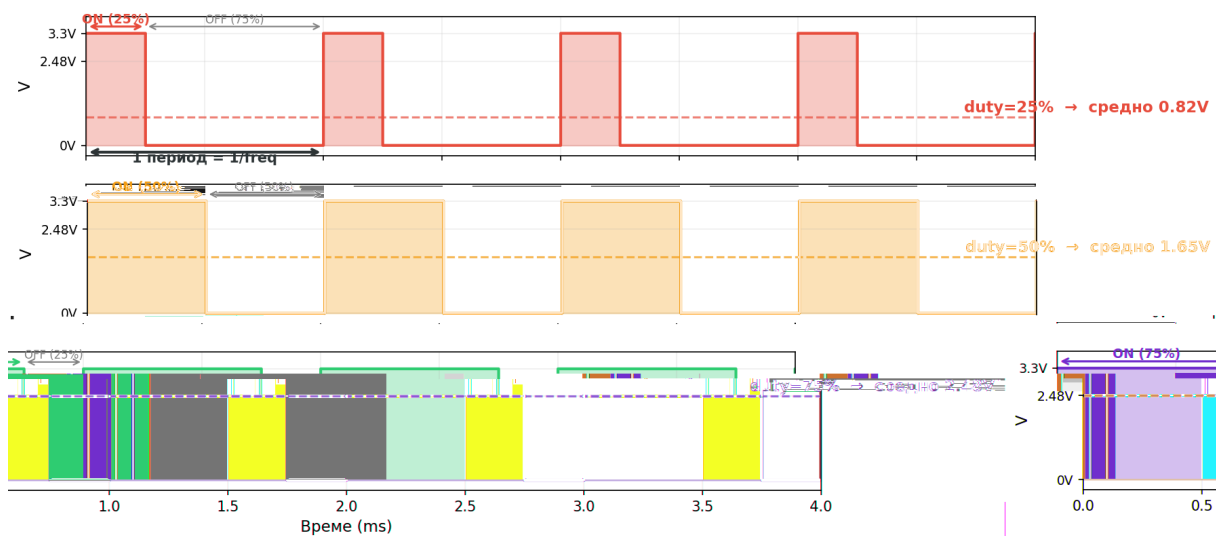
## Част IV: Таймери, прекъсвания и време

### Глава 14. PWM

[T] : PWM (d e)

50% d

PWM: duty cycle определя средното напрежение  
freq = 1000 Hz (период = 1 ms)

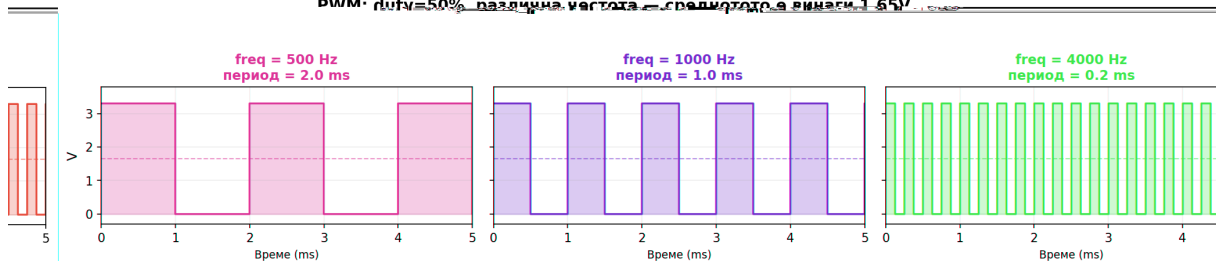


PWM d c c e 25%, 50%, 75%

(1).

| Параметър | Смисъл | Пример |
|-----------|--------|--------|
|           |        | 1000 1 |
|           | %      | 0 0%   |

PWM: duty=50% различна честота = средното е винаги 1.65V



PWM

0%

(1.).

```
from machine import Pin, PWM

pwm = PWM(Pin("P107"), freq=1000, duty=50) # 1kHz, 50% → средно 1.65V
```

[!]  
: GPT P106 P107 GPT . fe ,  
. d .

```
pwm_a = PWM(Pin("P107"), freq=1000, duty=25)
pwm_b = PWM(Pin("P106"), freq=1000, duty=75) # споделен freq с pwm_a
pwm_a.freq(2000) # ← промяна засяга и pwm_b!
```

[Eye] Какво виждаш: 10 ( )

[Lab] Опитай: duty 10 0 10

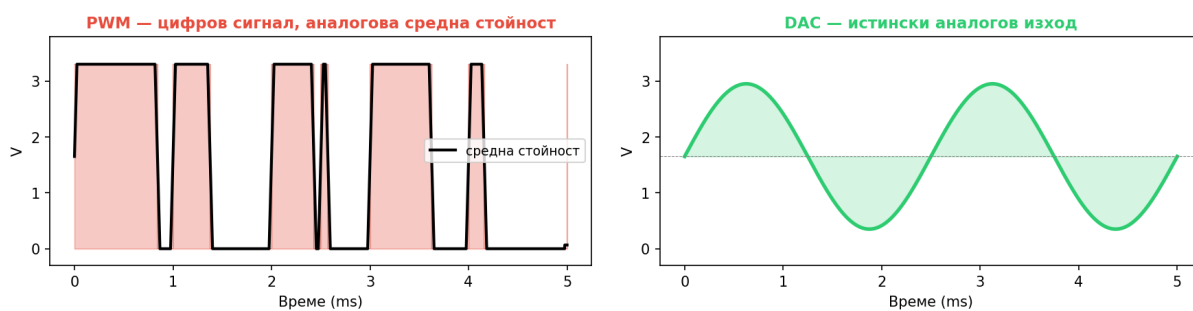
- 
- 
- 
- 

freq() /

- examples/22\_analog/02\_pwm\_output.py
- examples/03\_pwm\_ab\_same\_channel.py

## PWM vs DAC — каква е разликата?

**PWM имитира аналогово ниво, DAC го генерира директно**



PWM DAC

имитира

(0 . ). генерира директно

0

(Гл

ава 13)

[P ] : PWM, DAC, ,  
RC ( + )

## Резюме

- PWM GPT, ,
- A/B GPT duty, freq().
- PWM , , DAC
- , , , print().

## Самопроверка

1. pwm\_a.freq() pwm\_b, GPT
2. freq duty
3. print() ,

## Упражнения

1. duty
2. GPT , duty
3. 500 Hz, 1000 Hz 2000 Hz

## Ако не работи

- 
- GPT, freq(),
- 

## Нататък

(-1) ([Глава 15](#)),

## Глава 15. Закъснения, хардуерни таймери и Timer(-1)

[T ] : е. ee \_ (300)  
T е (1)

| Метод  | Блокира CPU? | Прецизност | Когато |
|--------|--------------|------------|--------|
| . ( )  |              |            |        |
| (1.. ) |              |            |        |
| (-1)   |              |            |        |
| . ( )  |              |            |        |

```
import time # Импортираме time за блокиращото закъснение.
time.sleep_ms(300) # Това закъснение блокира изпълнението на останалия код.
```

```
from machine import Timer # Импортираме Timer за хардуерния таймер.

timer = Timer(1) # Използваме един от наличните хардуерни machine.Timer канали от Timer(1) до Timer(6).
timer.init(freq=4, callback=timer_callback, hard=False) # Стартираме Timer(1) на 4 Hz с callback извън har...
hard=False (по подразбиране): callback-ът се изпълнява през Python scheduler-а – безопасно за print(), ал...
hard=True: callback-ът се изпълнява директно в хардуерния IRQ контекст – минимална латентност, но НЕ алок...
не ползвайте print() и не извиквайте сложни Python обекти вътре. Подходящо само за екстремно кратки и кри...
```

```
from machine import Pin, Timer # Импортираме Pin и Timer за демонстрацията на софтуерния таймер.
import time # Импортираме time, за да оставим таймера да работи кратко време.

led = Pin("LED1", Pin.OUT) # Вземаме LED1 като видим индикатор за сработване на callback-а.
soft_timer = Timer(-1) # Създаваме софтуерен таймер чрез идентификатор -1.
soft_state = {"count": 0} # Пазим броя сработвания в mutable структура.
started = False # С този флаг ще следим дали таймерът е стартирал успешно.

def soft_timer_callback(timer_object): # Това е функцията, която ще се извиква при всяко сработване на соф...
 soft_state["count"] = soft_state["count"] + 1 # Увеличаваме брояча при всяко събитие.
 led.value(0 if led.value() else 1) # Превключваме LED, за да виждаме че callback-ът се изпълнява.

if not started: # Ако още не сме стартирали таймера, пробваме втория стил с честота.
 try: # Влизаме в защитен блок за втория опит.
 soft_timer.init(freq=4, callback=soft_timer_callback, hard=False) # Пробваме инициализация с често...
 started = True # Отбелязваме, че вторият стил е успял.
 print("Timer(-1) тръгна с freq API.") # Печатаме кой резервен стил е сработил.
 except Exception as second_error: # Ако и вторият стил не тръгне, само информираме потребителя.
 print("freq API също не тръгна:", second_error) # Показваме причината и за втория неуспех.
```

- Timer(1) Timer(6)
- Timer(-1)
- asyncio

time.ticks\_us()

- time.ticks\_us() time.ticks\_diff()
- time.ticks\_us() mp\_hal\_ticks\_us(),

- `ADC.read()` ,

- `examples/23_timing/01_delays_and_timers.py`
- `examples/23_timing/04_software_timer_minus1.py`
- `examples/23_timing/05_timer_compare_hw_sw.py`
- `examples/23_timing/07_adc_timing_measure.py`
- `examples/23_timing/13_hardware_timer_periodic_oneshot.py`
- `examples/23_timing/14_hardware_timer_output_compare.py`
- `examples/23_timing/15_hardware_timer_input_capture.py`
- `examples/23_timing/16_hardware_timer_event_count.py`

## Подглава 15.1. Хардуерни таймери

`Timer(1) Timer(-1)`

`VK_RA4M2`

- `Timer(1) Timer(6)`
- `Timer(-1)`
- `Timer.PERIODIC Timer.ONE_SHOT`
- `freq(), period() counter()`
- `output compare, input capture event count`
- `event count N , Timer.callba`
- `ck() period=N`

```
from machine import Timer # Импортираме Timer, за да използваме хардуерните таймери.
```

```
periodic_timer = Timer(1) # Вземаме Timer(1) за периодичен режим.
```

```
oneshot_timer = Timer(2) # Вземаме Timer(2) за еднократен режим.
```

```
periodic_timer.init(mode=Timer.PERIODIC, freq=2, callback=periodic_callback, hard=False) # Пускаме периоди...
```

```
print(periodic_timer.period()) # Показваме периода в сурови AGT counts.
```

```
print(periodic_timer.counter()) # Показваме текущия суров counter.
```

```
oneshot_timer.init(mode=Timer.ONE_SHOT, freq=2, callback=oneshot_callback, hard=False) # Пускаме еднократе...
```

- `examples/23_timing/13_hardware_timer_periodic_oneshot.py`

```

from machine import Timer # Импортираме Timer, защото output compare работи през хардуерния таймер.

timer = Timer(1) # Използваме Timer(1) като базов таймер за compare събитията.
timer.init(mode=Timer.PERIODIC, freq=10, callback=cycle_callback, hard=False) # Пускаме бавен периодичен т...

period_counts = timer.period() # Вземаме периода в сурови counts.
channel_a = timer.channel(1, mode=Timer.OC, compare=period_counts // 4, callback=compare_a_callback) # Нас...
channel_b = timer.channel(2, mode=Timer.OC, compare=(period_counts * 3) // 4, callback=compare_b_callback) ...

```

- examples/23\_timing/14\_hardware\_timer\_output\_compare.py

```

from machine import Pin, PWM, Timer # Импортираме Pin, PWM и Timer за генерация и измерване на тестов сигнал.

signal_pwm = PWM(Pin("P107"), freq=1000, duty=50) # Генерираме тестов квадратен сигнал 1 kHz на P107.
timer = Timer(1) # Използваме Timer(1) за input capture измерването.

timer.init(mode=Timer.PERIODIC, freq=1_000_000, hard=False) # Пускаме Timer(1) на 1 MHz, за да е 1 count п...
period_channel = timer.channel(0, mode=Timer.IC, pin=Pin("P100"), measure=Timer.IC_PERIOD, edge=Timer.RISIN...

```

- examples/23\_timing/15\_hardware\_timer\_input\_capture.py

N

```

from machine import Pin, PWM, Timer # Импортираме Pin, PWM и Timer за броене на входни импулси.

signal_pwm = PWM(Pin("P107"), freq=200, duty=50) # Генерираме входен тестов сигнал 200 Hz на P107.
timer = Timer(1) # Използваме Timer(1) като хардуерен брояч на събития.

timer.init(mode=Timer.PERIODIC, period=20, callback=every_n_events_callback, hard=False) # Искаме callback...
counter_channel = timer.channel(0, mode=Timer.IC, pin=Pin("P100"), measure=Timer.IC_EVENT_COUNT, edge=Timer...
print(counter_channel.capture()) # Четем частичния брой импулси в текущия прозорец.

```

- Timer.IC\_EVENT\_COUNT
  - capture()
  - Timer.callback()
- N

- examples/23\_timing/16\_hardware\_timer\_event\_count.py

## Подглава 15.2. Практически checklist за AGT тестове

- examples/AGT\_TEST\_CHECKLIST.md

- ```

•                               examples/      /flash/examples
•                               print()
•                               ,
•                               ,

1.      examples/23_timing/13_hardware_timer_periodic_oneshot.py
2.      examples/23_timing/14_hardware_timer_output_compare.py
3.      P107 -> P100
4.      examples/23_timing/15_hardware_timer_input_capture.py
5.      P107 -> P100
6.      examples/23_timing/16_hardware_timer_event_count.py

1      -

• Timer(1)
• Timer(2)
• Timer(1).counter()
•      -

2

• Cycle-end callback count
• Compare A callback count
• Compare B callback count
•      ,      -      0

•      Timer(1)      pin=Pin("P500")      channel(1)
•      Timer(1)      pin=Pin("P501")      channel(2)
•      1/4  3/4

•      P107 -> P100
•      1 kHz / 50% duty      Измерен период      1000
•      Измерена high ширина      500
•      0,      -

•      P107 -> P100
•      200 Hz
•      20
•      1.1 s      10      11
• Частичен брой в текущия прозорец      0  19
• Приблизителен общ брой импулси      220

•      Timer.IC_EVENT_COUNT
• capture()
•      Timer.callback()

```

- P107 -> P100
- PWM Timer
- deinit()
- periodic , compare -
- compare , capture event count , -

1. examples/23_timing/13_hardware_timer_periodic_oneshot.py
2. examples/23_timing/15_hardware_timer_input_capture.py P107 -> P100
3. examples/23_timing/16_hardware_timer_event_count.py

Резюме

- time.sleep_ms() - , .
- Timer(n) , .
- Timer(-1) , .
- , .

Самопроверка

1. time.sleep_ms(), Timer(n) Timer(-1)
2. - sleep_ms()
- 3.

Упражнения

1. 1 sleep_ms(), .
2. 4 Hz 2 Hz 8 Hz .
3. examples/23_timing/05_timer_compare_hw_sw.py Timer(1)
) Timer(-1).

Ако не работи

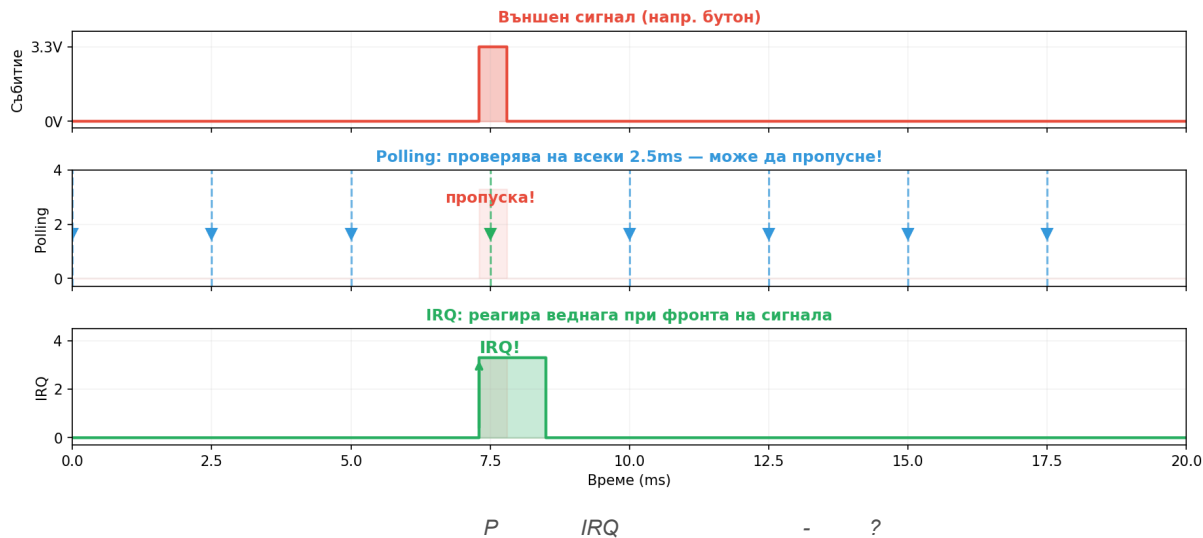
- freq
- .
- - .

Нататък

,

Глава 16. Прекъсвания

[T] : P CPU 5 " ?" IRQ



[!] : IRQ a de . (, I2C,)

IRQ

```
from machine import Pin, disable_irq, enable_irq # Импортираме Pin, disable_irq и enable_irq за демонстрац...

button = Pin("SW1", Pin.IN, Pin.PULL_UP) # Настройваме SW1 като вход с pull-up.

def button_handler(pin_object): # Това е функцията, която ще се вика при прекъсване от бутона.
    print("IRQ") # Печатаме текст при всяко прекъсване.

button.irq(handler=button_handler, trigger=Pin.IRQ_FALLING, hard=False) # Регистрираме обработчик за падащ...
```

```
irq_state = disable_irq() # Временно забраняваме прекъсванията, за да вземем консистентно копие на брояча.
snapshot = irq_state_data["count"] # Копираме брояча, докато прекъсванията са спрени.
enable_irq(irq_state) # Възстановяваме предишното състояние на прекъсванията.
```

Софтуерни (soft) и хардуерни (hard) прекъсвания

hard=

Параметър	Контекст на изп	Какво може	Какво НЕ може
-----------	-----------------	------------	---------------

```

from machine import Pin

press_count = 0

def on_press(pin):
    global press_count
    press_count += 1
    print(f"Натискане #{press_count}")      # print() е OK в soft IRQ
    # Можете да четете I2C, пишете в UART, алокирате памет

button = Pin("SW1", Pin.IN, Pin.PULL_UP)
button.irq(handler=on_press, trigger=Pin.IRQ_FALLING, hard=False)

```

Hard IRQ (порт-специфичен за VK_RA4M2)

```

from machine import Pin

irq_flag = bytearray(1)  # pre-allocated – не алокирай вътре в hard IRQ!

def on_press_hard(pin):
    irq_flag[0] = 1      # само запис в съществуващ буфер – OK
    # print() тук → CRASH!
    # list.append() тук → CRASH!
    # I2C.readfrom() тук → CRASH!

button = Pin("SW1", Pin.IN, Pin.PULL_UP)
button.irq(handler=on_press_hard, trigger=Pin.IRQ_FALLING, hard=True)

while True:
    if irq_flag[0]:
        irq_flag[0] = 0
        print("Натиснат (hard IRQ)")      # тежката работа – в main loop

```

[P] : a d=Fa e . a d=T e
100 . - eed e c de afe

- time.ticks_us()
- , ,
-
- examples/23_timing/08_interrupt_latency_measure.py
- examples/23_timing/02_interrupts.py
- examples/23_timing/08_interrupt_latency_measure.py
- examples/28_machine_misc/02_machine_mem_and_bootloader.py

Резюме

- ,
- -

- , ,

Самопроверка

- 1.
2. -
3. `disable_irq()` `enable_irq()`

Упражнения

1. , ,
2. .
3. , .

Ако не работи

-
-

Нататък

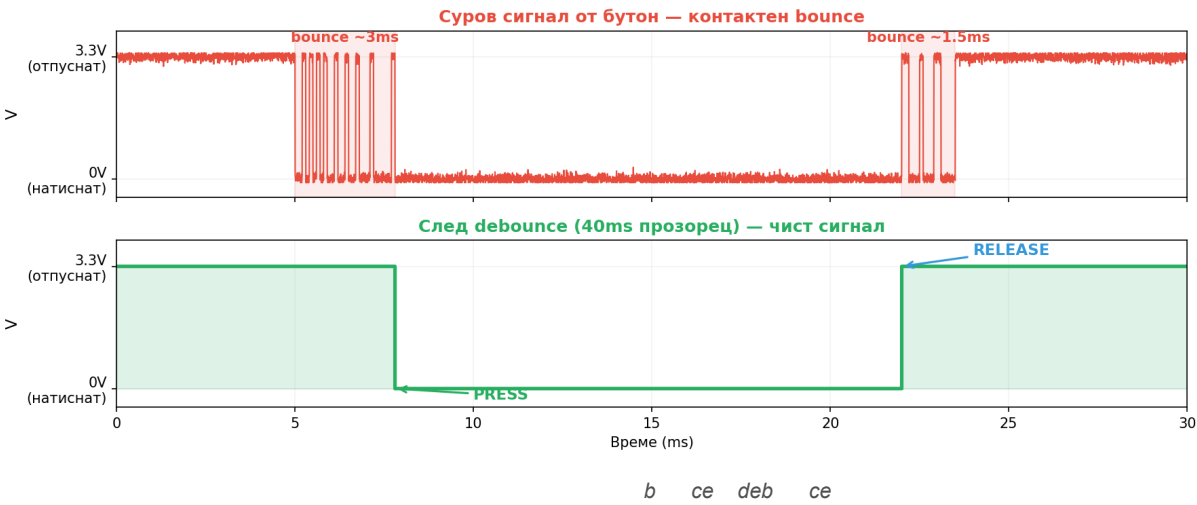
,

Глава 17. Дебаунс на бутон

16 , *IRQ*
b *ce.* :

[*T*] :
 1-20 . *deb ce* 5-50 . *Deb ce*

Дебаунс: от шумен механичен сигнал до чисто логическо събитие



Подход	Как работи	Когато
	0	
		,

```
if current_value != last_stable_value: # Проверяваме дали е настъпила промяна спрямо последното стабилно с...
    now_ms = time.ticks_ms() # Вземаме текущото време само при промяна.
    if time.ticks_diff(now_ms, last_change_ms) >= 40: # Приемаме промяната за валидна само ако е минало по...
        last_change_ms = now_ms # Обновяваме момента на последната приета промяна.
        last_stable_value = current_value # Приемаме новата стойност за стабилна.
```

uasyncio

```
if stable_value == 0 and stable_count == 4: # Ако видим четири поредни проби с 0, приемаме валидно натискане.
    async_presses += 1 # Увеличаваме броя на валидните асинхронни натискания.
```

-
-

• examples/23_timing/03_button_debounce.py

Резюме

-
-
-

Самопроверка

- 1.
- 2.

3. ,

Упражнения

1. 40 ms 10 ms 80 ms
2. ,
3. uasyncio

Ако не работи

-
-

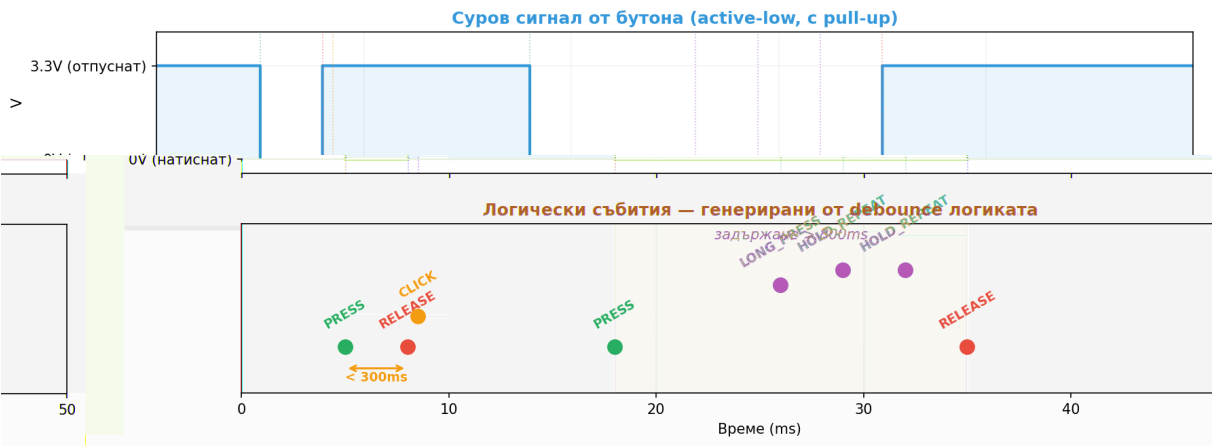
Нататък

polling, , Timer(-1), ,

Глава 18. Четене на бутон: polling, IRQ, Timer(-1), debounce и събития

[T] : b . a e() 0 1.

: " , " 2 , " .



Събитие	Условие
	1 0
	0 1
	+ 00
	00

VK_RA4M2.

SW1



- PRESS
- RELEASE
- CLICK
- LONG_PRESS
- HOLD_START
- HOLD_REPEAT

button.value()

polling

```
raw_value = button.value() # Четем суровата моментна стойност на бутона.
if raw_value != last_raw_value: # Ако видим ново сурово ниво, започваме нов debounce прозорец.
    last_raw_value = raw_value # Запомняме новата сурова стойност.
    raw_change_ms = now_ms # Запомняме кога е започнала промяната.
if raw_value != stable_value and time.ticks_diff(now_ms, raw_change_ms) >= DEBOUNCE_MS: # Ако новото ниво ...
    stable_value = raw_value # Обновяваме стабилното състояние на бутона.
    if stable_value == 0: # Ако стабилното състояние е 0, имаме валидно натискане.
        print("EVENT: PRESS") # Печатаме събитие за натискане.
    else: # Ако стабилното състояние е 1, имаме валидно отпускане.
        print("EVENT: RELEASE") # Печатаме събитие за отпускане.
```

-
-
-

- polling,

IRQ_FALLING | IRQ_RISING

```
def button_irq_handler(pin_object): # Това е краткият IRQ обработчик за суровите фронтове на бутона.
    irq_state_data["pending"] = 1 # Маркираме, че главният цикъл трябва да довърши логическата обработка.
    irq_state_data["deadline_ms"] = time.ticks_add(time.ticks_ms(), DEBOUNCE_MS) # Отлагаме окончателното ...

if pending and time.ticks_diff(now_ms, deadline_ms) >= 0: # Ако има чакаща работа и debounce прозорецът е ...
    stable_candidate = button.value() # Четем реалното ниво след краткото изчакване.
    if stable_candidate != previous_stable_value: # Ако стабилното ниво реално е ново, генерираме логическ...
        print("EVENT: PRESS" if stable_candidate == 0 else "EVENT: RELEASE") # Печатаме кое логическо съби...
```

-

-
-
-

PRESS RELEASE,

-

Pin.irq(..., fast=True) @micropython.asm_thumb,

```

•
•
•
[!]
:
ARM T b a e b ,
c e .add e f ISR
P301 P302 ( SW1 = P400),
- GND. 19.
P301 P302, fast ASM IRQ, bytearray +

```

```

from array import array # Импортираме array, за да държим shared 32-bit стойности в RAM.
from machine import Pin # Импортираме Pin, за да вържем бутоните към external IRQ линиите.
from ctypes import addressof # Импортираме addressof, за да дадем на ASM кода адресите на shared буферите.
import machine # Импортираме machine, за да ползваме disable_irq/enable_irq около consumer логиката.
import micropython # Импортираме micropython, защото fast IRQ handler-ите са @micropython.asm_thumb.
import time # Импортираме time, за да направим release debounce извън IRQ.

events = bytearray(16) # Това е ring buffer-ът за кратките събития от прекъсването.
event_values = array("i", [0] * 16) # Тук пазим snapshot на стойността за всяко събитие.
state = array("i", [0, 0, 0, 0, 0, 0]) # Държим wr_idx, rd_idx, value, lost_events, plus_lock и minus_lock.

queue_addr = addressof(events) # Взимаме адреса на ring buffer-а за директен byte достъп от ASM.
values_addr = addressof(event_values) # Взимаме адреса на snapshot буфера за директен word достъп от ASM.
state_addr = addressof(state) # Взимаме адреса на state блока за директен word достъп от ASM.

exec(build_button_isr_source("plus_isr", +1, 1), globals(), globals()) # Генерираме fast ASM ISR за бутона...
exec(build_button_isr_source("minus_isr", -1, 2), globals(), globals()) # Генерираме fast ASM ISR за бутон...

plus_button = Pin("P301", Pin.IN, Pin.PULL_UP) # Настройваме P301 като вход с pull-up за бутона "+".
minus_button = Pin("P302", Pin.IN, Pin.PULL_UP) # Настройваме P302 като вход с pull-up за бутона "-".

plus_button.irq(trigger=Pin.IRQ_FALLING, handler=plus_isr, fast=True) # Връзваме fast IRQ на падащ фронт з...
minus_button.irq(trigger=Pin.IRQ_FALLING, handler=minus_isr, fast=True) # Връзваме fast IRQ на падащ фронт...

while True: # Главният цикъл остава извън IRQ и само довършва debounce и печата събитията.
    handle_release_debounce() # Re-arm-ваме бутона едва след стабилно отпускане.
    irq_state = machine.disable_irq() # Кратко спираме IRQ, за да вземем консистентен snapshot на опашката.
    event_code, event_value, lost_events = pop_event() # Вземаме едно събитие и snapshot стойността му от ...
    machine.enable_irq(irq_state) # Връщаме IRQ веднага след кратката критична секция.
    if event_code == 1: # Ако кодът е 1, печатаме събитие за бутона "+".
        print("PLUS value =", event_value) # Показваме snapshot стойността за натискане на бутона "+".
    elif event_code == 2: # Ако кодът е 2, печатаме събитие за бутона "-".
        print("MINUS value =", event_value) # Показваме snapshot стойността за натискане на бутона "-".
    if lost_events: # Ако ring buffer-ът е бил пълен, показваме, че има изпуснати събития.
        print("Queue overflow, lost events =", lost_events) # Печатаме диагностиката за overflow извън IRQ.
    time.sleep_ms(2) # Оставяме кратък sleep, за да е четим изходът и да работи release debounce логиката.

```

```

•
•
•
,
,
+

```

```

• , -
- . - build_button_isr_source(...) @micropyth
on.asm_thumb

```

- examples/23_timing/17_fast_irq_two_buttons_queue.py

```

Timer(-1)

```

```

Timer(-1)

```

```

soft_timer.init(period=SAMPLE_MS, mode=Timer.PERIODIC, callback=button_sample_callback) # Стартираме Timer...
if timer_state["history"] == 0x00 and timer_state["stable_value"] != 0: # Ако последните осем проби са 0, ...
    timer_state["event_code"] = EVENT_PRESS # Записваме PRESS събитие за основния цикъл.
elif timer_state["stable_value"] == 0 and not timer_state["hold_started"] and time.ticks_diff(now_ms, timer...
    timer_state["event_code"] = EVENT_HOLD_START # Записваме HOLD_START събитие.
elif timer_state["stable_value"] == 0 and timer_state["hold_started"] and time.ticks_diff(now_ms, timer_sta...
    timer_state["event_code"] = EVENT_HOLD_REPEAT # Записваме HOLD_REPEAT събитие.

```

- PRESS 1 -> 0
- RELEASE 0 -> 1
- CLICK
- LONG_PRESS
- HOLD_START
- HOLD_REPEAT

N

```

history = ((history << 1) | button.value()) & 0xFF # Премества историята наляво и добавяме новата проба ...
if history == 0x00 and stable_value != 0: # Ако последните осем проби са натиснати, приемаме стабилно PRES...
    print("EVENT: PRESS") # Печатаме събитие за натискане.
elif history == 0xFF and stable_value != 1: # Ако последните осем проби са отпуснати, приемаме стабилно RE...
    print("EVENT: RELEASE") # Печатаме събитие за отпускане.

```

history

- 0x00
- 0xFF

- - , polling
- ,
- / , Timer(-1)
- ,

- `examples/23_timing/09_button_polling_events.py`
- `examples/23_timing/10_button_irq_deferred.py`
- `examples/23_timing/17_fast_irq_two_buttons_queue.py`
- `examples/23_timing/11_button_soft_timer_hold.py`
- `examples/23_timing/12_button_shift_debounce.py`
- `examples/23_timing/02_interrupts.py`
- `examples/23_timing/03_button_debounce.py`
- `examples/23_timing/04_software_timer_minus1.py`

Резюме

-
- polling, , fast ASM IRQ , Timer(-1)
- , PRESS, RELEASE HOLD ,

Самопроверка

1. PRESS
2. Timer(-1) - polling
3. fast ASM IRQ - ,
4. 0x00 0xFF

Упражнения

1. 09_button_polling_events.py, LONG_PRESS CLICK
2. CLICK 10_button_irq_deferred.py, PRESS RELEASE
3. 17_fast_irq_two_buttons_queue.py P301 P302
4. 12_button_shift_debounce.py

Ако не работи

- PRESS RELEASE
- LONG_PRESS HOLD_REPEAT

Нататък

Глава 19. RTC

```
[T] : e. c _ () . RTC
, e e ( ).
```

	time.ticks_ms()	RTC
		()
	,	,

```
from machine import RTC # Импортираме RTC, за да работим с часовника в реално време.
```

```
rtc = RTC() # Създаваме обект за достъп до вградения RTC модул.
print(rtc.info()) # Печатаме диагностичната информация за стартиране на RTC.
print(rtc.calibration()) # Печатаме текущата калибрационна стойност.
print(rtc.datetime()) # Четем текущата дата и час като осемелементен tuple.
```

-
-
-

- `examples/23_timing/06_rtc_basic.py`

Резюме

- RTC
- `rtc.datetime()`
-
- `print()`.

Самопроверка

1. RTC `Timer(1)`
2. - `rtc.datetime()`
- 3.

Упражнения

1. `rtc.datetime()`
2. YYYY-MM-DD HH:MM:SS - `rtc.datetime()`.
- 3.

Ако не работи

-
- `rtc.datetime()`.

•	.	.
---	---	---

Нататък

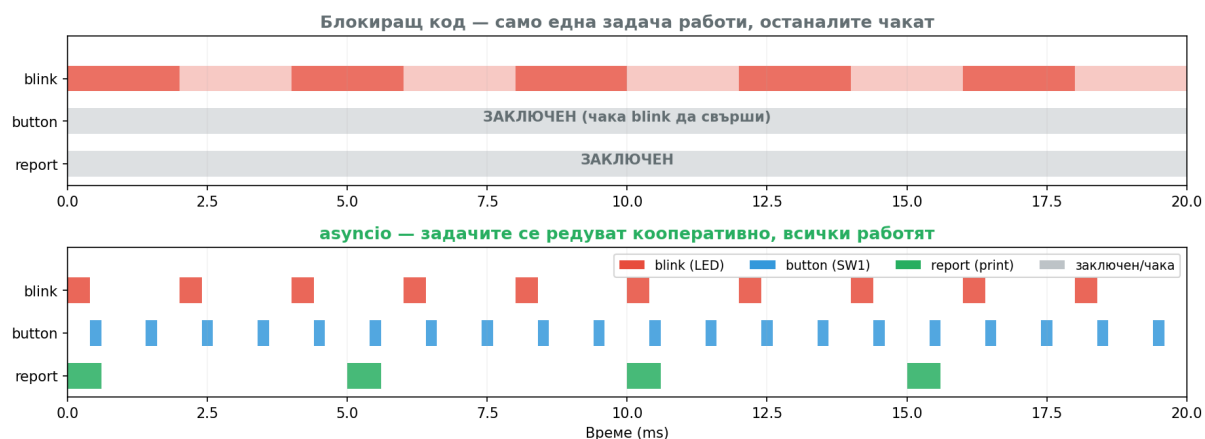
2 , ,

Част IVб: asyncio — мост към конкурентно програмиране

ако имаме няколко задачи, които трябва да се изпълняват едновременно, тогава asyncio е идеалният инструмент. asyncio е библиотека за асинхронно програмиране, която позволява на програмата да изпълнява няколко задачи едновременно, без да се блокира. asyncio е базирана на asyncio loop, който е обект, който управлява асинхронните задачи. asyncio е базирана на asyncio loop, който е обект, който управлява асинхронните задачи.

Глава 20. Основи на asyncio

В тази глава ще разгледаме основите на asyncio. asyncio е библиотека за асинхронно програмиране, която позволява на програмата да изпълнява няколко задачи едновременно, без да се блокира. asyncio е базирана на asyncio loop, който е обект, който управлява асинхронните задачи. asyncio е базирана на asyncio loop, който е обект, който управлява асинхронните задачи.



а с

```
import asyncio # Импортираме asyncio за кооперативна многозадачност.
from machine import Pin # Импортираме Pin за реален хардуерен достъп.

LED_ON = 0 # Active-low: 0 = светва LED1.
LED_OFF = 1 # Active-low: 1 = гаси LED1.

led = Pin("LED1", Pin.OUT, value=LED_OFF) # LED1 = P204, започваме изгасен.
button = Pin("SW1", Pin.IN, Pin.PULL_UP) # SW1 = P400, вътрешен pull-up.

async def blink_task(period_ms): # Асинхронна задача за мигане с зададен период.
    while True: # Безкрайно мигане.
        led.value(LED_ON) # Светваме LED1.
        await asyncio.sleep_ms(period_ms) # Отстъпваме управлението за period_ms.
        led.value(LED_OFF) # Гасим LED1.
        await asyncio.sleep_ms(period_ms) # Отстъпваме управлението отново.
```

- `create_task`
- `asyncio.gather`
- `examples/33_asyncio/01_asyncio_basics.py`

Резюме

- `asyncio` е библиотека за асинхронно програмиране в Python.
- `asyncio` е базирана на `asyncio` и `asyncio`.
- `asyncio` е базирана на `asyncio` и `asyncio`.

Самопроверка

1. `create_task`
2. `asyncio`
- 3.

Упражнения

1. `asyncio`
2. `asyncio`
3. `asyncio`

Ако не работи

- `await`
- `await`

Нататък

Глава 21. asyncio Event синхронизация

[Т] : а с .Е е ()

button_producer: — натискане → event.set() →

led_consumer: ————— await event.wait() —→ мига LED
(кооперативно спи докато чака)

```

import asyncio # Импортираме asyncio за кооперативна многозадачност.
from machine import Pin # Импортираме Pin за реален хардуерен достъп.

LED_ON = 0 # Active-low: 0 = светва LED1.
LED_OFF = 1 # Active-low: 1 = гаси LED1.

led = Pin("LED1", Pin.OUT, value=LED_OFF) # LED1 = P204, започваме изгасен.
button = Pin("SW1", Pin.IN, Pin.PULL_UP) # SW1 = P400, вътрешен pull-up.
button_event = asyncio.Event() # Създаваме Event обект за синхронизация между задачи.

async def button_producer(): # Тази задача следи SW1 и сигнализира при натискане.
    last_value = 1 # Последна стойност на бутона (ненатиснат).
    current_value = button.value() # Четем SW1.
    if current_value == 0 and last_value == 1: # Нов натиск (falling edge).
        button_event.set() # Сигнализираме Event-а.

async def led_consumer(): # Тази задача чака Event и мига LED1.
    await button_event.wait() # Чакаме Event-а (блокираме кооперативно).
    button_event.clear() # Нулираме Event-а за следващо използване.

```

- `asyncio.Event`
- -
- -
- `examples/33_asyncio/02_asyncio_event_flag.py`

Резюме

- `asyncio.Event`
- -
- ,

Самопроверка

1. `asyncio.Event`
2. -
3. ,

Упражнения

1. .
2. ,
3. `set()`.

Ако не работи

- - `clear()`.
- .

Нататък

Част V

",

asyncio

2 , .

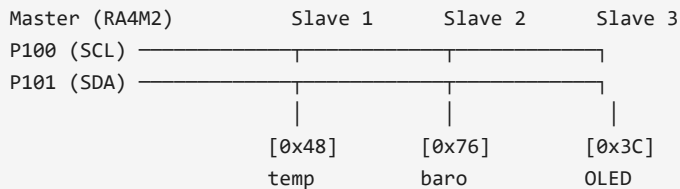
Част V: Сериини интерфейси и двоични протоколи

I2C, SPI, UART,

- .

Глава 22. I2C master и runtime избор на пинове

[T] : I2C (2) (а е)
(7-b). Ма е " 0 48, " 0 48



[!] : I2C(0) P400 = SW1! I2C(1) P100/P101

```
from machine import I2C # Импортираме I2C за работа като master върху наличния хардуерен контролер.

i2c = I2C(1, freq=100000) # Създаваме I2C(1) на 100 kHz върху бордовите пинове P100 и P101.
print(i2c.scan()) # Сканираме шината за устройства с 7-битов адрес.
```

```
from machine import I2C, Pin # Импортираме I2C и Pin, за да покажем runtime избиране на SCL и SDA.

i2c = I2C(1, freq=400000, scl=Pin("P100"), sda=Pin("P101")) # Създаваме I2C(1) на 400 kHz с изрично зададе...
```

- I2C(0) P400/P401
- P400 SW1

- examples/24_i2c/01_i2c_master_basic.py
- examples/24_i2c/02_i2c_runtime_pins.py

Резюме

- I2C , - , .
- scan() - , .

- `I2C(0)` SW1.

Самопроверка

1. `scan()` - I2C
2. `scan()` - I2C(0)
3. SW1

Упражнения

1. `scan()` 100 kHz, 400 kHz, `scl` `sda`
2. SCL, SDA, master, device address.
- 3.

Ако не работи

- `scan()`
- `scan()`
- `scan()`

Нататък

2 2 , 2

Глава 23. SoftI2C и I2CTarget

```
[P ] P .OPEN_DRAIN I2C: I2C SCL SDA e -d a
- ( 4.7 VCC)
S f I2C P .OPEN_DRAIN; I2C
```

2

```
from machine import SoftI2C, Pin # Импортираме SoftI2C и Pin за софтуерна I2C шина върху GPIO.

soft_i2c = SoftI2C(scl=Pin("P105", Pin.OPEN_DRAIN), sda=Pin("P104", Pin.OPEN_DRAIN), freq=100000) # Създав...
```

2

```
from machine import I2CTarget, Pin # Импортираме I2CTarget и Pin за работа в target или slave режим.

memory_buffer = bytearray(range(16)) # Подготвяме малък memory-backed буфер, който target-ът ще обслужва.
target = I2CTarget(1, addr=0x42, mem=memory_buffer, mem_addrsize=8, scl=Pin("P100"), sda=Pin("P101")) # Съ...
```

- 2
- -
- examples/24_i2c/03_softi2c_basic.py
- examples/24_i2c/04_i2ctarget_memory.py

Резюме

- SoftI2C , 2
-
- I2CTarget , ,
- - ,

Самопроверка

1. SoftI2C I2C
2. I2CTarget,
3. -

Упражнения

1. - .
2. addr=0x42
3. SoftI2C I2C.

Ако не работи

- , / -
- -
-

Нататък

, 2

Глава 24. Hardware SPI и SoftSPI

[Т] : SPI (MOSI/MISO) (SCK). I2C, SPI f -d e . a e CS (C Se ec)

	I2C	SPI
	2 (+)	+ (+ + +)
	-	
	00	0
	,	, ,

```
from machine import SPI, Pin # Импортираме SPI и Pin за достъп до хардуерния SPI канал.  
  
spi = SPI(0, baudrate=500000, polarity=0, phase=0, bits=8, sck=Pin("P102"), mosi=Pin("P101"), miso=Pin("P10...))
```

```
from machine import SoftSPI, Pin # Импортираме SoftSPI и Pin за софтуерен SPI върху GPIO.  
  
soft_spi = SoftSPI(baudrate=100000, polarity=0, phase=0, sck=Pin("P105"), mosi=Pin("P104"), miso=Pin("P106"...))
```

SPI 1 .

- examples/25_spi/01_spi_basic.py
- examples/25_spi/02_softspi_basic.py

Резюме

- SPI - 2 , SCK, MOSI, MISO CS.
- SPI , SoftSPI
- SPI (polarity phase) ,

Самопроверка

1. MOSI, MISO, SCK CS
2. - SoftSPI, SPI
3. polarity/phase ,

Упражнения

1. , - , .
2. baudrate .

3. polarity/phase

Ако не работи

- Проверете полярността и фазата на сигнала. Ако е необходимо, променете `polarity` и `phase` в конфигурацията.
- Проверете дали е свързан MOSI и MISO.
- Проверете дали е свързан CS.

Нататък

(Глава 25),

Глава 25. Базова UART комуникация

[Т] : UART
а е / а е. TX RX
а е

RA4M2 (UART9) Компютър / модул
P602 (TX) → RX
P601 (RX) ← TX
GND → GND

[P] VK_RA4M2: UART(9) TX=P602, RX=P601
I2C/SPI.

```
from machine import UART # Импортираме UART класа за серийна комуникация.
import time # Импортираме time за кратко закъснение.

uart = UART(9, 115200) # Създаваме UART(9) на 115200 baud (TX=P602, RX=P601).
message = "Здравей от VK_RA4M2!\r\n" # Подготвяме съобщение за изпращане.
bytes_sent = uart.write(message) # Изпращаме съобщението по TX пина.

time.sleep_ms(100) # Кратко закъснение, за да пристигнат данни (ако има loopback).
available = uart.any() # Проверяваме дали има налични байтове за четене.
if available > 0: # Ако има данни за четене:
    received = uart.read(available) # Четем наличните байтове от RX буфера.
    print("Получено:", received) # Показваме получените данни.
```

- UART(0) - TX=P411, RX=P410
- UART(2) - TX=P302, RX=P301
- UART(7) - TX=P401, RX=P402, CTS=P403
- UART(9) - TX=P602, RX=P601, CTS=P603

-
-
- ()

- `examples/32_uart/01_uart_basic.py`

Резюме

- -
- ,
- -

Самопроверка

- 1.
- 2.
3. `any()` `read()`

Упражнения

- 1.
2. ,
3. `\r\n`.

Ако не работи

-
-

Нататък

•

Глава 26. Работа с външни модули по интерфейси

[P] : `b ea a , b e -`
27

I2C, SPI UART

- 1.
2. I2C, SPI UART
3. CS, RST, INT, BUSY EN
4. - ,

VK_RA4M2

- I2C(0) P400 SW1
- I2C(1) P100/P101 hardware SPI
- UART(9) - , 2

, , .

-

- I2C -
- SPI - , CS -
- UART , ,

1. GND

2.

3.

4.

5.

6. ,

I2C

```
from machine import I2C # Импортираме I2C за базов probe на външна шина.
```

```
i2c = I2C(1, freq=100000) # Създаваме I2C(1) на безопасни 100 kHz.
```

```
print(i2c.scan()) # Сканираме шината за адреси на свързани I2C модули.
```

SPI

```
from machine import SPI, Pin # Импортираме SPI и Pin за frame-базиран обмен с външен модул.
```

```
cs = Pin("P103", Pin.OUT, value=1) # Държим CS неактивен преди началото на транзакцията.
```

```
spi = SPI(0, baudrate=500000, polarity=0, phase=0, bits=8, sck=Pin("P102"), mosi=Pin("P101"), miso=Pin("P10..."))
```

```
cs.value(0) # Активираме target модула чрез CS.
```

```
spi.write_readinto(b"\x9F\x00\x00\x00", bytearray(4)) # Изпращаме команда и четем едновременно отговора.
```

```
cs.value(1) # Освобождаваме модула след края на frame-a.
```

UART /

```
from machine import UART # Импортираме UART за сериен обмен с външен модул.
```

```
uart = UART(9, 115200) # Създаваме UART(9) с често срещана начална скорост.
```

```
uart.write(b"AT\r\n") # Изпращаме примерна текстова команда към модула.
```

```
print(uart.read()) # Четем каквото е пристигнало към този момент.
```

- I2C scan()
- SPI CS
- UART baud rate, TX/RX

- datasheet-

- I2C
- SPI polarity, phase, CS -
- UART baud rate,

- examples/34_external_modules/01_i2c_module_probe.py
- examples/34_external_modules/02_spi_module_transaction.py
- examples/34_external_modules/03_uart_module_request_response.py

Резюме

- I2C, SPI UART -
- , -
- VK_RA4M2 .

Самопроверка

1. ,
2. SPI I2C
3. UART(9) - I2C(1) SPI

Упражнения

1. power, pins, protocol, probe.
2. 01_i2c_module_probe.py, ,
3. 03_uart_module_request_response.py,

Ако не работи

- GND, ,
- , baud rate, SPI mode,
- RST, CS, BUSY, INT

Нататък

Глава 27. Байтове, bytearray, struct, memoryview и двоични протоколи

- UART /
- SPI
- I2C
-

```
import struct

frame = bytearray(6)
frame[0] = 0xAA
struct.pack_into("<H", frame, 1, 1023)
frame[3] = 7

checksum = 0
for b in memoryview(frame)[:1]:
    checksum ^= b

frame[-1] = checksum
print(frame)
```

- bytearray
- struct
- memoryview
-

- examples/20_language_basics/05_arrays.py
- examples/20_language_basics/07_pointers_equivalents.py
- examples/24_i2c/04_i2ctarget_memory.py
- examples/32_uart/01_uart_basic.py
- examples/34_external_modules/02_spi_module_transaction.py
- examples/34_external_modules/03_uart_module_request_response.py

Резюме

-
- bytearray, struct memoryview
-

Самопроверка

1. bytes bytearray
2. struct.pack_into()
3. memoryview -

Упражнения

1. header, payload length checksum.
2. struct.unpack_from().
- 3.

Ако не работи

- struct
-

Нататък

Част VI: Системни функции, памет и организация на проекта

, machine,

Глава 28. TouchPad: четене, диагностика и cached API

```
from machine import Pin, TouchPad # Импортираме Pin и TouchPad за работа с CTSU вход.

touch = TouchPad(Pin("P205")) # Създаваме TouchPad върху валиден touch пин P205.
touch.config(500) # Задаваме праг 500 за метода value().
print(touch.read()) # Четем необработената CTSU стойност.
print(touch.value()) # Преобразуваме стойността към 0 или 1 според прага.
print(touch.read_value()) # Взимаме едновременно count, value и последна FSP грешка.
```

-
-
-

- examples/26_touchpad/01_touchpad_basic.py

Резюме

- TouchPad.read() , value() 0 1, read_value()

- config()

-

Самопроверка

1. read()
2. config(500)
3. read_value() read() + value()

Упражнения

1. read() 20
2. value() 1
3. (P205, P206, P408)

Ако не работи

- `read()` връща `0`, ако `touch.sample_rate(20)` не е стартиран.
- `value()` връща `1`, ако `touch.read_cached()` не е стартиран.

Диагностика, `cached API` и асинхронно следене

```
read() # Връща 0, ако touch.sample_rate(20) не е стартиран.
```

```
TouchPad.sample_rate(20) # Стартираме глобалния cooperative sampler на 20 пълни scan-а в секунда.
touch.start() # Стартираме non-blocking scan, ако не е в ход такъв.
TouchPad.service() # Придвижваме вътрешния sampler една стъпка от VM контекст.
print(touch.ready(), touch.read_cached(), touch.value_cached()) # Показваме cached състоянието.
print(touch.diagnose(8)) # Печатаме резултат от кратък диагностичен цикъл.
```

```
TouchPad.sample_rate(50) # Стартираме sampler на 50 scan-а в секунда.

async def touch_task(): # Асинхронна задача за следене на TouchPad.
    while True: # Безкрайно следене.
        if tp.ready(): # Проверяваме дали пробата е готова.
            print(tp.read_cached(), tp.value_cached(), tp.age_ms()) # Печатаме cached стойностите.
            await asyncio.sleep_ms(20) # Сканираме на 20 ms.
```

- `examples/26_touchpad/02_touchpad_diagnostics.py`
- `examples/04_touchpad_async.py`

Резюме

- `sample_rate()` задава честота на сканиране, която се използва от `read()`.
- `read_cached()` и `value_cached()` връщат последно сканирано състояние.
- `diagnose()` връща резултат от кратък диагностичен цикъл.

Самопроверка

1. `read()` връща `0`, ако `touch.sample_rate(20)` не е стартиран.
2. `sample_rate(50)` задава честота на сканиране, която се използва от `read()`.
3. `age_ms()` връща време в милисекунди, от което `read_cached()` е стартиран.

Упражнения

1. `sample_rate(20)` `read_cached()` 0
2. `diagnose(8)` -
3. , 1 , `value_cached()`.

Ако не работи

- `read_cached()` `(age_ms())` . , `TouchPad.service()` `sample_rate()` .
- `diagnose()` -

Нататък

`renesas.Flash, /flash dataflash ,`

Глава 29. renesas.Flash, /flash и dataflash

[Т] : RAM , . Fa
- , e e . da afa (8KB)
 , .

Памет	Размер	Жива след reset?	Употреба
()	12		,
/ ()			, .
			,

[!] : Fa e (100,000 c de fa).

`renesas.Flash`

```
import os # Импортираме os, за да покажем съдържанието на файловата система /flash.
import renesas # Импортираме renesas модула, който излага Flash block device.

flash = renesas.Flash() # Създаваме block device обект върху вътрешната flash памет.
sector = bytearray(16) # Подготвяме малък буфер за четене на първите байтове от блок 0.
flash.readblocks(0, sector) # Четем първите 16 байта от блок 0 в буфера.
print(os.listdir("/flash")) # Показваме кои файлове има във вътрешната файлова система.
```

`dataflash`

```
import dataflash # Импортираме dataflash модула за достъп до отделната Data Flash област.

print(dataflash.size()) # Печатаме общия размер на Data Flash областта.
print(dataflash.block_size()) # Печатаме размера на erase блока.
print(dataflash.write_size()) # Печатаме минималния размер за запис.
print(dataflash.read(0, 16)) # Четем първите 16 байта от Data Flash.
```

dataflash

LED1

```
from machine import Pin # Импортираме Pin за управление на LED1.
import dataflash # Импортираме dataflash за устойчиво съхранение на състояние.

led = Pin("LED1", Pin.OUT, value=1) # Използваме board alias-а LED1, започваме изгасен.
state = dataflash.read(0, 1)[0] # Връщаме стойността на първия байт.

if state == 0x00: # Ако предишният старт е записал 0x00, сега правим erase към 0xFF.
    dataflash.erase_block(0) # Изтриваме блок 0, всички байтове стават 0xFF.
    blinks_per_sec = 2 # Бавно мигане означава erase.
else: # Иначе приемаме, че байтът е 0xFF и записваме 0x00.
    dataflash.write(0, bytes([0x00])) # Записваме 0x00 в първия байт.
    blinks_per_sec = 10 # Бързо мигане означава write.
```

dataflash

VK_RA4M2

LED1,

P204

- 0xFF erase_block(0)
- 0x00 write(0, bytes([0x00]))

-
-
-

VK_RA4M2

- 384 KB 128 KB
- 290 KB / , 94 KB FLASH_FS

/flash

- 8 KB dataflash
- /flash , FAT 512 bytes
- - 94 KB,

- examples/27_storage/01_flash_blockdev.py
- examples/27_storage/02_dataflash_basic.py
- examples/27_storage/03_dataflash_state_led.py

Команда	Действие
. ()	
. ()	
. ()	
. ()	
. ()	

[P] : c.d abe() 14 (DAC a bac).

```
import gc # Импортираме gc за контрол и измерване на garbage-collected heap-a.
import micropython # Импортираме micropython за mem_info и други runtime helper-и.

gc.collect() # Пускаме GC в предвидим момент, за да започнем от по-чисто състояние.
print(gc.mem_free(), gc.mem_alloc()) # Печатаме свободния и заетия heap.
micropython.mem_info() # Печатаме обобщена информация за heap-a.
```

```
buffer = bytearray(32) # Създаваме един буфер, който ще използваме.
window = memoryview(buffer)[8:12] # Вземаме прозорец към част от буфера без копиране (вж. Глава 8 за основ...
window[0] = 123 # Променяме първия байт през memoryview.
print(list(buffer[8:12])) # Показваме, че оригиналният буфер е променен на място.
```

```
, -
```

- -
- ,
-
-

VK_RA4M2

- - - _heap_start _heap_end
- - 8 KB
- /flash dataflash ,

```
- ,
```

- gc.collect()
- gc.mem_free() gc.mem_alloc()
- gc.threshold() -
- micropython.mem_info() micropython.mem_info(1) -
- micropython.alloc_emergency_exception_buf() - /
- micropython.heap_lock() micropython.heap_unlock() ,

```
, -
```

- bytearray ,
- memoryview,

- `gc.collect()`
- `machine.mem8/mem16/mem32`
- `memoryview`
- `07_pointers_equivalents.py`
- `04_i2ctarget_memory.py`
- `01_flash_blockdev.py` `02_dataflash_basic.py`
- `examples/28_machine_misc/04_memory_management.py`
- `examples/20_language_basics/07_pointers_equivalents.py`
- `examples/24_i2c/04_i2ctarget_memory.py`
- `examples/27_storage/01_flash_blockdev.py`
- `examples/27_storage/02_dataflash_basic.py`

Резюме

- `gc.collect()`
- `machine.mem8/mem16/mem32`
- `memoryview`
- `07_pointers_equivalents.py`
- `04_i2ctarget_memory.py`
- `01_flash_blockdev.py` `02_dataflash_basic.py`
- `examples/28_machine_misc/04_memory_management.py`
- `examples/20_language_basics/07_pointers_equivalents.py`
- `examples/24_i2c/04_i2ctarget_memory.py`
- `examples/27_storage/01_flash_blockdev.py`
- `examples/27_storage/02_dataflash_basic.py`

Самопроверка

1. `gc.collect()`
2. `machine.mem8/mem16/mem32`
3. `memoryview`

Упражнения

1. `gc.collect()`
2. `machine.mem8/mem16/mem32`
3. `memoryview`

Ако не работи

- `gc.collect()`
- `machine.mem8/mem16/mem32`
- `memoryview`

Нататък

- `gc.collect()`
- `machine.mem8/mem16/mem32`
- `memoryview`

Глава 31. machine модулът и порт-специфично поведение

```
import machine # Импортираме machine, за да покажем системните функции на порта.

uid = machine.unique_id() # Четем уникалния 128-битов идентификатор на MCU.
uid_hex = "".join("{:02x}".format(byte_value) for byte_value in uid) # Преобразуваме байтовете до четим he...

print("machine.freq() =", machine.freq()) # Показваме текущата честота на ядрото.
print("machine.unique_id() =", uid_hex) # Печатаме уникалния идентификатор в hex вид.
print("machine.reset_cause() =", machine.reset_cause()) # Печатаме причината за последния reset.
machine.info() # Печатаме подробна системна информация от самия порт.
```

```
irq_state = machine.disable_irq() # Временно забраняваме прекъсванията и пазим предишното състояние.
machine.enable_irq(irq_state) # Възстановяваме прекъсванията веднага след демонстрацията.
print(hasattr(machine, "mem32")) # Показваме дали портът излага 32-битов memory access.
```

```
DO_LIGHTSLEEP = False # Оставяме lightsleep изключен по подразбиране.
DO_DEEPSLEEP = False # Оставяме deepsleep изключен по подразбиране.

print("Извикваме кратък machine.lightsleep(20).") # Подготвяме потребителя за кратък sleep.
machine.lightsleep(20) # Извикваме кратък lightsleep за 20 ms.

if DO_LIGHTSLEEP: # Само ако е включено, пробваме по-дълъг lightsleep.
    machine.lightsleep(100) # Извикваме lightsleep за 100 ms.
```

- machine.info
- machine.freq
- machine.unique_id
- machine.reset_cause
- disable_irq enable_irq
- mem8, mem16, mem32
- machine.bootloader
- sleep, lightsleep, deepsleep

- machine.freq() 100 MHz
- 50 MHz -
- - ,
- - 15.4 mA , 6.1 mA
- 4.4 mA
- - 0.7 uA 5 ... 16 uA

- 12-bit / 0.8 mA,

- `examples/28_machine_misc/01_machine_info_and_id.py`
- `examples/28_machine_misc/02_machine_mem_and_bootloader.py`
- `examples/28_machine_misc/03_sleep_modes.py`

Резюме

- `machine` , ,
- `machine.info()`, `freq()`, `unique_id()` `reset_cause()` ,
- `disable_irq()`, `mem32`, `bootloader`, `lightsleep()` `deepsleep()` ,
- , -

Самопроверка

1. `machine.info()` `machine.unique_id()`
2. `disable_irq()` `enable_irq(irq_state)`
3. `DO_LIGHTSLEEP`, `DO_DEEPSLEEP`

Упражнения

1. , `freq`, `unique_id`, `reset_cause` `machine.i`
`nfo()`.
2. `machine.lightsleep(20)` `time.ticks_ms()`
3. `bootloader()` -

Ако не работи

- `bootloader`, `deepsleep` -
- - `mem8/mem16/mem32`
- , `en`
`able_irq()` ,

Нататък

- ,

Глава 32. boot.py, main.py и жизнен цикъл на програмата

-
-
-
-
-

```
# boot.py
from machine import Pin

status_led = Pin("LED1", Pin.OUT, value=1)

def early_init():
    status_led.value(0)
    status_led.value(1)

early_init()
```

```
# main.py
import app

app.run()
```

```
# app.py
def run():
    print("Стартира приложението.")
    # Тук вече може да се създадат обекти, задачи и основен loop.
```

- boot.py
- main.py
-

boot.py,

- examples/index.py
- examples/28_machine_misc/01_machine_info_and_id.py
- examples/28_machine_misc/03_sleep_modes.py

Резюме

- - .
- `boot.py` , .
- `main.py` .

Самопроверка

1. `boot.py`
2. , `main.py` `app.run()`
3. -

Упражнения

1. - `boot.py`, `main.py` `app.py`.
2. `LED1`, , .
3. , `main.py` .

Ако не работи

- `boot.py` `main`
- `.py` `app.py`.
- `print()`

Нататък

Глава 33. Организация на проект: модули, `drivers`, `config` и `pins.py`

```
/flash/
  boot.py
  main.py
  app.py
  config.py
  pins.py
  drivers/
  services/
```

- `pins.py` -
- `config.py`
- `drivers/`
- `services/` -
- `app.py`

```
# pins.py
LED_STATUS = "LED1"
BUTTON_OK = "P400"
DAC_OUT = "P014"
```

```
# config.py
BUTTON_SAMPLE_MS = 10
AUDIO_WAVE = "pulse25"
```

```
# app.py
from machine import Pin
import pins
import config

status_led = Pin(pins.LED_STATUS, Pin.OUT, value=1)
print("Button sample period =", config.BUTTON_SAMPLE_MS)
```

, .

- examples/34_external_modules/01_i2c_module_probe.py
- examples/retro_synth.py

Резюме

- , , .
- pins.py config.py .
- - .

Самопроверка

1. pins.py -
2. drivers/ services/
3. app.py ,

Упражнения

1. pins.py.
2. config.py.
3. , , , .

Ако не работи

- pins.py,
- - ,

Нататък

• ,

Глава 34. Таблични данни, конфигурации и малки файлови формати

-

-
-
- -
-
-

JSON

```
import json

cfg = {
    "button_sample_ms": 10,
    "audio_wave": "triangle",
    "melody_tempo": 120,
}

with open("/flash/config.json", "w") as f:
    json.dump(cfg, f)
```

,

-
-
- -

- examples/27_storage/01_flash_blockdev.py
- examples/27_storage/02_dataflash_basic.py
- examples/27_storage/03_dataflash_state_led.py
- examples/retro_synth.py

Резюме

-
- JSON
- -

Самопроверка

1. ,
2. -
- 3.

Упражнения

1. , - JSON .
2. - .
3. flash .

Ако не работи

- .
- . , .

Нататък

Част VII: Сигнали, измерване, енергия и индикация

Глава 35. Филтриране, калибриране и хистерезис при входни сигнали

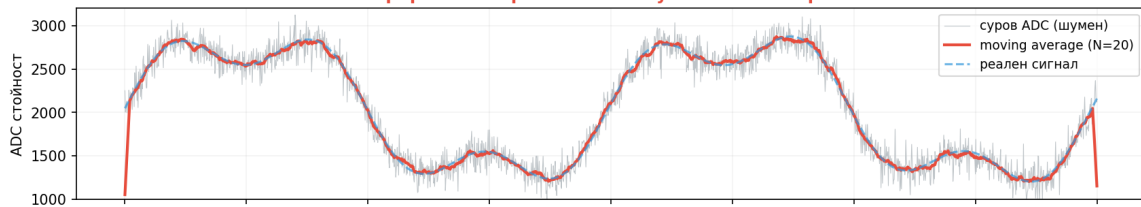
```
ADC.read()
```

-
-
-
-
-
-
-
-

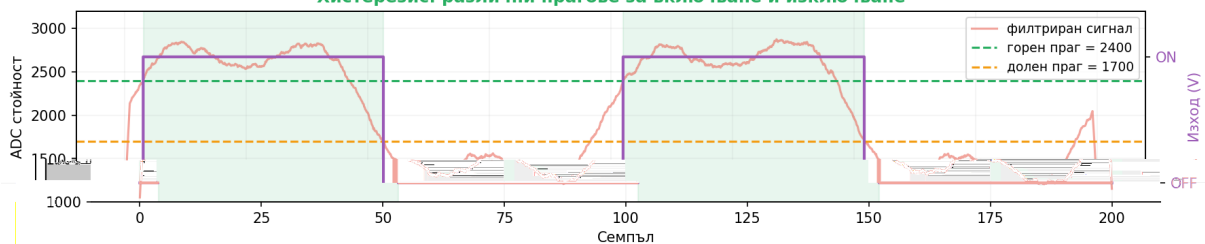
```
def moving_average(samples):  
    return sum(samples) // len(samples)  
  
def hysteresis_state(value, state, low_th=1800, high_th=2200):  
    if state == 0 and value >= high_th:  
        return 1  
    if state == 1 and value <= low_th:  
        return 0  
    return state
```

Глава 35: Филтриране, калибриране и хистерезис

Филтриране: отстраняване на шум от ADC измерването



Хистерезис: различни прагове за включване и изключване



-
-
-

- `examples/22_analog/01_analog_input_adc.py`
- `examples/23_timing/07_adc_timing_measure.py`
- `examples/26_touchpad/01_touchpad_basic.py`
- `examples/26_touchpad/02_touchpad_diagnostics.py`

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
- 2.
3. 0..100%.

Ако не работи

-
-

Нататък

Глава 36. Измерване и диагностика с осцилоскоп и logic analyzer

, , `print()` .

-
- -
- `ticks_us()`

```

from machine import Pin

probe = Pin("P204", Pin.OUT, value=1)

def mark():
    probe.value(0)
    probe.value(1)

```

-
-
-
-

- PWM
- UART
- SPI
- I2C
- DAC
-

- examples/23_timing/07_adc_timing_measure.py
- examples/23_timing/08_interrupt_latency_measure.py
- examples/23_timing/13_hardware_timer_periodic_oneshot.py
- examples/32_uart/01_uart_basic.py
- examples/dac_firmware_probe.py
- examples/dac_dmac_diag.py

Резюме

-
- ticks_us()
-

Самопроверка

1. print()
- 2.
3. -

Упражнения

- 1.
- 2.
- 3.

Самопроверка

1. - lightsleep, deepsleep
2. RTC -
- 3.

Упражнения

1. lightsleep.
2. , .
3. .

Ако не работи

- - .
- - - lightsleep .

Нататък

Глава 38. Динамична 7-сегментна индикация и матрично сканиране

```
from machine import Pin # Импортираме Pin, за да покажем базовото сканиране на външен дисплей и бутони.
import time # Импортираме time за краткото динамично опресняване.

segment_pins = [Pin(name, Pin.OUT) for name in ("P105", "P104", "P113", "P114", "P115", "P304", "P303")] #...
digit_pins = [Pin(name, Pin.OUT) for name in ("P608", "P409")] # Това са двата общи извода за две цифри.
patterns = {"1": (0, 1, 1, 0, 0, 0, 0), "2": (1, 1, 0, 1, 1, 0, 1)} # Това са сегментните шаблони за две п...

for refresh_step in range(10): # Повтаряме кратък опреснителен цикъл десет пъти.
    for digit_index, symbol in enumerate(("1", "2")): # Показваме символ 1 на първата цифра и 2 на втората...
        for digit_pin in digit_pins: # Първо изключваме всички цифри, за да няма ghosting.
            digit_pin.off() # Изключваме текущата цифра.
        for segment_pin, level in zip(segment_pins, patterns[symbol]): # Зареждаме нужния сегментен шаблон...
            segment_pin.value(level) # Настройваме сегмента според шаблона.
        digit_pins[digit_index].on() # Включваме само текущата цифра за кратко време.
        time.sleep_ms(5) # Държим цифрата включена кратко, за да се получи динамична индикация.
```

```

row_pins = [Pin(name, Pin.OUT) for name in ("P408", "P600")] # Това са редовете на примерна 2x2 клавиатурн...
col_pins = [Pin(name, Pin.IN, Pin.PULL_UP) for name in ("P301", "P302")] # Това са колоните на примерната ...
pressed_keys = [] # Подготвяме списък за натиснати клавиши от матрицата.

for row_index, row_pin in enumerate(row_pins): # Сканираме всеки ред поотделно.
    for reset_row in row_pins: # Първо изключваме всички редове.
        reset_row.off() # Изключваме реда.
    row_pin.on() # Активираме текущия ред.
    for col_index, col_pin in enumerate(col_pins): # Проверяваме всяка колона в активния ред.
        if col_pin.value() == 0: # Ако колоната падне на 0, приемаме че бутонът е натиснат.
            pressed_keys.append((row_index, col_index)) # Записваме координатите на натиснатия бутон.

```

- [examples/29_display_scan/01_dynamic_7seg_and_keypad.py](#)

Резюме

- , - , .
- , .
- .

Самопроверка

- 1.
- 2.
3. ,

Упражнения

- 1.
- 2.
3. 1 .

Ако не работи

- .
- , .

Нататък

- , ,

Глава 39. NeoPixel

NeoPixel

еорixel.

- P500
 - P112 DIN
 - 1 -
 - P500
 - - -
 -
 - , NeoPixel
- 1.
 - 2.
 3. write(),

```
from machine import Pin # Импортираме Pin за power-enable и data линията.
import neopixel # Импортираме neopixel драйвера за RGB модула.
```

```
vcc = Pin("P500", Pin.OUT, value=1) # Включваме P500, защото без него модулет не е активен в този setup.
np = neopixel.NeoPixel(Pin("P112"), 1) # Създаваме NeoPixel обект за един пиксел върху P112.
np[0] = (40, 0, 0) # Зареждаме червено в буфера на първия пиксел.
np.write() # Изпращаме буфера и реално сменяме цвета на модула.
```

- (R, G, B)
- write()
- , 40,
- , NeoPixel

- DIN P112
- P500
-
-

- examples/neopixel.py
- examples/neopixel_test.py

Резюме

- `NeoPixel`
- `power`, `write()`.
-

Самопроверка

1. `NeoPixel`
2. `write()` `np[0] = (...)`
3. `P500`, `-`

Упражнения

1. `2` `4`
2. `,`
3. `SW1` `-`

Ако не работи

- `P500` `DIN`
- `,`
- `P500`

Нататък

- `WS2812` `machine.WS2812`,
- `-` `-`

Глава 40. WS2812 през machine.WS2812 и SCI TX-only

- `[T]` `: WS2812` `LED-` `da a`
- `24` `,` `c c`

RA4M2 (P112, SCI2 TX) LED[0] LED[1] LED[2] ...
 → [GRB 24b] → [GRB 24b] → [GRB 24b] →
 (пази) (пази) (пази)

Метод	Изисква	Горна граница	Употреба
<code>+</code> <code>()</code>			
<code>2 12 ()</code>			2

[!] DTC : WS2812 SCI2 DAC AGT DTC. DAC . e() (52).

WS2812B

neopixel machine.bitstream(), machine.WS2812. SCI TXD/MOSI

- DIN
- SCK MISO
- write(), NeoPixel

SCI , 3-bit

- SCI 2.857143 MHz
- 5-bit 2 12
- LOW -
- LOW latch/reset
- DTC , IRQ

, -

- write()
-

audrate, b

- SCI0 - P101, P205, P411
- SCI1 - P213, P401, P501
- SCI2 - P102, P112, P302
- SCI9 - P109, P203, P409, P602

, P112 ,

SCI2 ,

- machine.WS2812
- UART(2)
- SPI(2) SCI2

SCI

```

from machine import Pin # Импортираме Pin за захранването и data линията.
from machine import WS2812 # Импортираме специализирания драйвер за WS2812.
import time # Импортираме time за кратко изчакване след включване на захранването.

vcc = Pin("P500", Pin.OUT, value=1) # Включваме P500, защото без него модулт не е активен в този setup.
time.sleep_ms(100) # Изчакваме power-enable линията и модулт да се стабилизира.
strip = WS2812(pixel_count=1, pin=Pin("P109"), channels=3) # Създаваме драйверен обект върху алтернативния...
strip[0] = (40, 0, 0) # Зареждаме червено в буфера на първия пиксел.
strip.write() # Изпращаме буфера към модула през SCI TX-only backend-a.

```

```

NeoPixel, write().

```

```

, - , - .

```

- - WS2812
- SCI TX/MOSI
- ,
- P500
- , P500
-

1. pixel_count=1.
2. red, green, blue, white, off.
3. .

```

, - .

```

```

VK_RA4M2, machine.WS2812

```

```

.

```

- examples/ws2812_sci.py
- examples/ws2812_sci_test.py
- examples/ws2812_sci_chase_145.py
- examples/ws2812_sci_p101.py
- examples/ws2812_sci_p109.py

Резюме

- WS2812 machine , SCI TX/MOSI
- ,
- SCI SCI WS2812 UART SP

I.

Самопроверка

1. `neopixel.NeoPixel machine.WS2812`
2. `DIN -> избрания TXD/MOSI pin,`
SCI
3. `P109 UART(9) machine.WS2812`

Упражнения

1. `P112, P109,`
2. `pixel_count 2 4`
3. `SCI0 SCI1`
4. `examples/ws2812_sci_chase_145.py 1 + 12*12 , P500`

Ако не работи

- `P500 DIN`
- `SCI UART SP`
- `I`
- `P500.`
- `P500`
- `neopixel , SCI TX/MOSI`

Нататък

" ,

Част VIII: FSM като начин на мислене

Глава 41. Минимална FSM и таблична FSM

9 с -са е. FSM
: (а е, е е), а е.

[T] : FSM (F e S a e Mac e)
(BUTTON, TIMEOUT)

Светофар FSM:

RED —(TIMER)—> GREEN —(TIMER)—> YELLOW —(TIMER)—> RED
↑
(цикличен loop)

Двата стила:

```
# 1. Минимален if/elif (до 3-4 състояния):
if state == "OFF":
    if event == "BUTTON": state = "ON"

# 2. Таблица (5+ състояния, по-четима):
transitions = {
    ("OFF", "BUTTON"): "ON",
    ("ON", "BUTTON"): "OFF",
    ("ON", "TIMEOUT"): "OFF",
}
state = transitions.get((state, event), state)
```

[P] : 3 f/e f. 4+ . + FSM.

```
STATE_OFF = "OFF" # Това е първото състояние на лампата.
STATE_ON = "ON" # Това е второто състояние на лампата.
EVENT_BUTTON = "BUTTON" # Това е събитието от бутона.

state = STATE_OFF # Започваме с изключена лампа.

def handle_event(current_state, event_name): # Дефинираме преходната функция на машината на състояния.
    if current_state == STATE_OFF and event_name == EVENT_BUTTON: # Ако сме в OFF и бутонът е натиснат, ми...
        return STATE_ON # Връщаме новото състояние ON.
    if current_state == STATE_ON and event_name == EVENT_BUTTON: # Ако сме в ON и бутонът е натиснат, мина...
        return STATE_OFF # Връщаме новото състояние OFF.
    return current_state # Ако няма преход, оставаме в същото състояние.
```

```

state = "OFF" # Започваме в състояние OFF.

transition_table = {("OFF", "BUTTON"): "ON", ("ON", "BUTTON"): "OFF"} # В таблицата описваме преходите кат...

event_sequence = ["BUTTON", "BUTTON", "BUTTON", "BUTTON"] # Симулираме поредица от събития.

```

```

OFF --BUTTON--> ON
ON  --BUTTON--> OFF

```

- `examples/30_fsm/01_fsm_lamp_button.py`
- `examples/30_fsm/02_fsm_table.py`

Резюме

-
-
-

Самопроверка

- 1.
2. - if
3. -

Упражнения

- 1.
2. -
- 3.

Ако не работи

-
-

Нататък

,

.

Глава 42. FSM с timeout

```
state = "OFF" # Започваме с изключена лампа.

transition_table = {("OFF", "BUTTON"): "ON", ("ON", "TIMEOUT"): "OFF", ("ON", "BUTTON"): "OFF"} # Описваме...

event_sequence = ["BUTTON", "TIMEOUT", "BUTTON", "BUTTON"] # Симулираме бутон и timeout събития.
```

+ + + .

- `examples/30_fsm/03_fsm_timeout.py`

Резюме

- `time.time()` връща текущото време в секунди, липсата на дробна част означава, че времето е цяло число.
- `time.time()` връща текущото време в секунди, липсата на дробна част означава, че времето е цяло число.
- `time.time()` връща текущото време в секунди, липсата на дробна част означава, че времето е цяло число.

Самопроверка

1. `time.time()` връща текущото време в секунди, липсата на дробна част означава, че времето е цяло число.
2. `time.time()` връща текущото време в секунди, липсата на дробна част означава, че времето е цяло число.
3. `time.time()` връща текущото време в секунди, липсата на дробна част означава, че времето е цяло число.

Упражнения

1. `time.time()` връща текущото време в секунди, липсата на дробна част означава, че времето е цяло число.
2. `time.time()` връща текущото време в секунди, липсата на дробна част означава, че времето е цяло число.
3. `time.time()` връща текущото време в секунди, липсата на дробна част означава, че времето е цяло число.

Ако не работи

- `time.time()` връща текущото време в секунди, липсата на дробна част означава, че времето е цяло число.
- `time.time()` връща текущото време в секунди, липсата на дробна част означава, че времето е цяло число.

Нататък

, -

Глава 43. Conditions vs Events

```
light_level = 600 # Това е моментното ниво от светлинния сензор в симулацията.
dark_threshold = 800 # Това е прагът, под който приемаме че е тъмно.
event_name = "BUTTON" # Това е дискретното събитие от бутон.

condition_is_dark = light_level < dark_threshold # Това е condition, която може да е вярна дълго време.

if condition_is_dark and event_name == "BUTTON": # Комбинираме условие и събитие в общо решение.
    print("Лампата може да се включи, защото е тъмно и има бутонно събитие.") # Печатаме положителния изход.
else: # Ако комбинацията не е изпълнена, не включваме лампата.
    print("Няма достатъчно условия за включване.") # Печатаме отрицателния изход.
```

-
-

• examples/30_fsm/04_conditions_vs_events.py

Резюме

• **Condition** () light_level < 800. **Event** BUTTON

• ,

• ,

().

Самопроверка

1. , .
2. ,
3. (light_level < 800) (BECAME_DARK)

Упражнения

1. (temperature > 30) .
2. DARK_ENTERED **прехода** ,
3. (, ,)

Ако не работи

- , already_handled.
 - - - **едно**
- временно** - , False.

Нататък

Глава 44. Guard функции

```
context = {"light": 500, "pir": True} # Подготвяме контекст със стойности от сензори.
state = "OFF" # Започваме в състояние OFF.
event_name = "MOTION" # Симулираме събитие за засечено движение.

def is_dark(data): # Това е първата guard функция.
    return data["light"] < 800 # Връща True само ако е достатъчно тъмно.

def has_motion(data): # Това е втората guard функция.
    return data["pir"] is True # Връща True само ако PIR сензорът вижда движение.

if state == "OFF" and event_name == "MOTION" and is_dark(context) and has_motion(context): # Комбинираме е...
    state = "ON" # Минаваме в ON само ако всички проверки са успешни.
```

- `examples/30_fsm/05_guard_functions.py`

Резюме

- **допълнителна проверка** ,
- , - False.
- -
- ,
- **ЧИСТИ** () True/False.

Самопроверка

- 1.
2. , (.)
3. ,

Упражнения

1. `is_nighttime(data),`
2. `is_dark()` `config,`
3. `print()`

Ако не работи

- , - ,
- and () or ().

Нататък

, ,

Глава 45. Приоритети и event queue

```
pending_events = ["BUTTON", "ERROR", "TIMEOUT"] # Симулираме едновременно чакащи събития.
priority_order = {"ERROR": 0, "BUTTON": 1, "TIMEOUT": 2} # По-малкото число означава по-висок приоритет.

pending_events.sort(key=lambda event_name: priority_order[event_name]) # Подреждаме събитията по приоритет.
```

```
event_queue = [] # Създаваме проста event queue като обикновен Python list.

event_queue.append(("BUTTON", {"source": "SW1"})) # Добавяме събитие от бутон.
event_queue.append(("PIR", {"source": "pir_sensor"})) # Добавяме събитие от PIR сензор.
event_queue.append(("TIMEOUT", {"ms": 30000})) # Добавяме timeout събитие.

while event_queue: # Обработваме опашката, докато има чакащи събития.
    event_name, payload = event_queue.pop(0) # Взимаме най-старото събитие от началото на опашката. Забеле...
    print("Обработваме", event_name, "с payload", payload) # Печатаме кое събитие се обработва сега.
```

-
-
-
-

- examples/30_fsm/06_priorities.py
- examples/30_fsm/07_event_queue.py

Резюме

- , приоритетът ERROR
- BUTTON TIMEOUT.
- (list collections.deque) генерирането ()
- обработката ().
- , ,

Самопроверка

1. , ERROR BUTTON ,
2. -
3. list.pop(0) collections.deque.popleft()

Упражнения

1. SENSOR_ALERT ERROR BUTTON ,
2. dropped_events ,
3. list collections.deque((), 16) -

Ако не работи

- , sort() **преди**
- , priority_order
- - -

Нататък

Глава 46. Tickless timers и sleep идея

```
import machine # Импортираме machine за кратък lightsleep между събитията.
import time # Импортираме time за работа с ticks_ms и ticks_diff.

tasks = [("lamp_timeout", 120), ("fan_timeout", 450), ("display_refresh", 40)] # Подготвяме три задачи с p...
start_ms = time.ticks_ms() # Запомняме началния момент.
next_deadline_ms = min(deadline for _, deadline in tasks) # Избираме най-близкия срок като tickless цел.
sleep_ms = max(0, next_deadline_ms - time.ticks_diff(time.ticks_ms(), start_ms)) # Изчисляваме колко време...
machine.lightsleep(sleep_ms) # Спим точно до най-близкото планирано събитие с lightsleep.
elapsed_ms = time.ticks_diff(time.ticks_ms(), start_ms) # Изчисляваме изминалото време след съня.
ready_tasks = [task_name for task_name, deadline in tasks if deadline <= elapsed_ms] # Избираме всички зад...
```

- -
 -
 -
- examples/30_fsm/08_tickless_sleep.py
 - examples/28_machine_misc/03_sleep_modes.py

Част IX: Надеждни вградени програми и архитектура на приложението

Глава 47. Обработка на грешки, recovery и fail-safe поведение

[7] : . I2C

Без retry: I2C грешка → crash x
 С retry: I2C грешка → изчакай 50ms → опитай отново × 3 → ако неуспешно → log + safe state v

Стратегия	Когато	Пример
		2 ,
-		,
		()

-
-
-
-
- UART

```
import sys
import time

def read_with_retry(reader, retries=3, delay_ms=50):
    for attempt in range(retries):
        try:
            return reader()
        except OSError as e:
            sys.print_exception(e) # Печата пълен traceback – стандартен начин за дебъг в MicroPython.
            time.sleep_ms(delay_ms)
    return None
```

[P] . _e ce (e) Mc P acebac
(e), , . _e ce (e)

-

-
-
- ,

- - ,
.

- examples/34_external_modules/01_i2c_module_probe.py
- examples/34_external_modules/02_spi_module_transaction.py
- examples/34_external_modules/03_uart_module_request_response.py
- examples/27_storage/03_dataflash_state_led.py

Резюме

-
- , , - .
- , .

Самопроверка

1. -
- 2.
3. ,

Упражнения

1. 2 .
2. last_error .
3. .

Ако не работи

-
- enter_safe_state()

Нататък

,

.

Глава 48. Watchdog и самовъзстановяване

- `renesas-ra` WDT

```
import time

last_alive_ms = time.time()

def mark_alive():
    global last_alive_ms
    last_alive_ms = time.time()

def looks_stuck(timeout_ms=2000):
    return time.time() - last_alive_ms > timeout_ms
```

- `renesas-ra` WDT

- `examples/01_blink_async.py`
- `examples/02_two_tasks.py`
- `examples/33_asyncio/01_asyncio_basics.py`

Резюме

- `renesas-ra`

Самопроверка

1. `renesas-ra`
2. `renesas-ra`
3. `renesas-ra`

Упражнения

1. `last_alive_ms`
2. ,
3. `WDT.feed()`, -

Ако не работи

- , -
- -
-

Нататък

-
-

Глава 49. Състояние на системата: heartbeat, health checks и debug интерфейс

- ,
- .

-
-
-
-

```
health = {
    "loops": 0,
    "last_error": None,
    "audio_active": False,
}

def heartbeat_tick():
    health["loops"] += 1
```

-
-
-
-

- `examples/01_blink_async.py`
- `examples/02_two_tasks.py`
- `examples/32_uart/01_uart_basic.py`

- `examples/dac_firmware_probe.py`

Резюме

-
- , `last_error` ,
- ,

Самопроверка

- 1.
2. -
3. `last_error` -

Упражнения

1. `health` .
2. .
3. , .

Ако не работи

-
- - .
- .

Нататък

,

Глава 50. Капсуловане на периферии като класове

7

`S a` `Led` .

-
-
-
-

```

from machine import Pin

class StatusLed:
    def __init__(self, pin_name="LED1"):
        self.pin = Pin(pin_name, Pin.OUT, value=1)

    def on(self):
        self.pin.value(0)

    def off(self):
        self.pin.value(1)

```

retro_synth.py

- examples/retro_synth.py
- examples/26_touchpad/02_touchpad_diagnostics.py

Резюме

-
-
-

Самопроверка

- 1.
- 2.
- 3.

Упражнения

- 1.
2. is_active().
- 3.

Ако не работи

-
-

Нататък

Глава 51. От пример към завършено приложение

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.

```
def init_board():
    pass

def init_services():
    pass

def run_forever():
    while True:
        pass

def main():
    init_board()
    init_services()
    run_forever()

main()
```

-
-
-
-

- examples/33_asyncio/01_asyncio_basics.py
- examples/retro_synth.py
- examples/dac_retro_music.py

Резюме

-
-
-

Самопроверка

- 1. -
- 2.
- 3. , ,

Упражнения

- 1. config, pins, app.
- 2. health , last_error .
- 3. .

Ако не работи

- . ,
- .

Нататък

Глава 52 (), , 2 12, , .

Глава 52. RGB_Guardian — казус: игра на WS2812 лента

Какво е RGB_Guardian

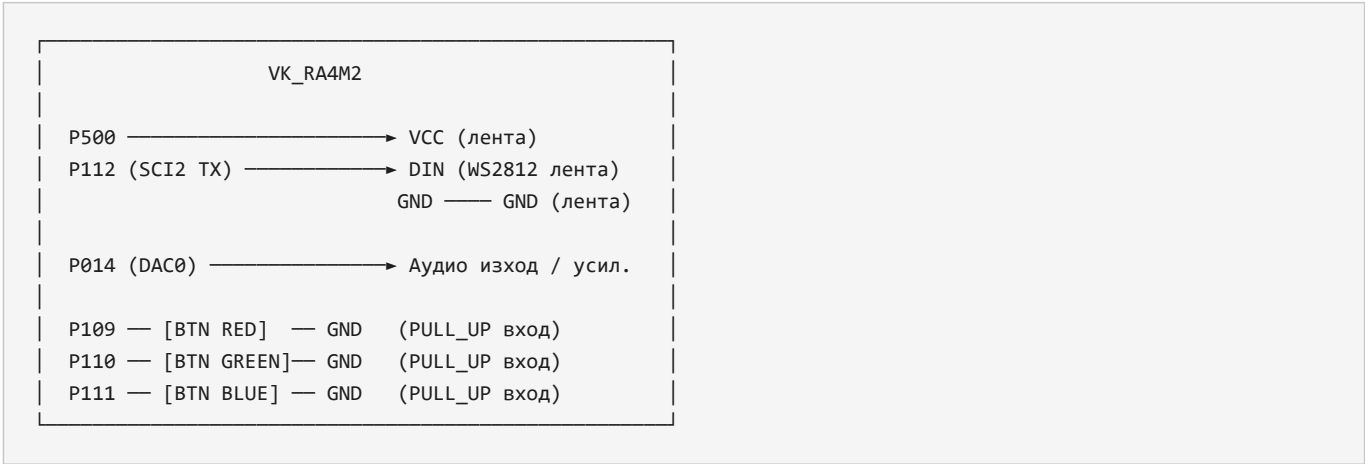
1- , , , 2 12 . , ,

наведнъж

Техника	Откъде идва
2 12 -	0
/	
(, ,)	1-
-	0
+ /	1 -1
	20-21

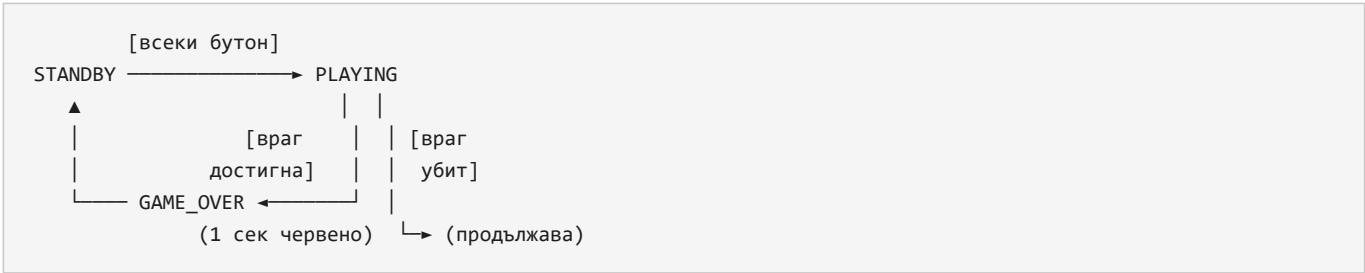
Хардуер





Сигнал	Пин	Тип	Описание
2 12	112	2	00 ,
2 12	00		,
	01	0	12- , ,
	10		()
	110		
	111		

Архитектура на играта



Game loop при всеки тик:

Enemy spawn → Bullet move → Collision check → WS2812 render + RetroSynth SFX

```
[!]  
: DTC ace c d RA4M2 ace c d DTC  
SCI2 (WS2812) AGT (DAC). e()  
Ha dFa : () DAC e(): ` def ():  
() # DAC/AGT DTC ace e() # SCI2 DTC`
```

Три слоя памет без runtime allocation:

```
# Pre-allocated bytearray pools – fixed size, zero runtime allocation
b_pos    = bytearray(MAX_BULLETS) # позиция на куршум
b_color  = bytearray(MAX_BULLETS) # цвят: 0=червен 1=зелен 2=син
b_active = bytearray(MAX_BULLETS) # 1=в полет, 0=свободен слот

e_pos    = bytearray(ENEMY_COUNT) # позиция на враг (255=убит)
e_color  = bytearray(ENEMY_COUNT) # цвят на враг
e_active = bytearray(ENEMY_COUNT) # 1=жив, 0=убит/неактивен
```

Структура	Тип	Размер	Алтернативата
,		10	00+
,		0	00+
			()
,	...	1	

Sentinel стойности: `e_pos[j] = 255` `b_active[i] = 0`

Звукови ефекти

Събитие	Метод	Описание
	<code>(" ", " ", 0, " ",)</code>	
	<code>()</code>	
	<code>(" ", " ", 0, " ",)</code>	
	<code>()</code>	

Ключова идея: collision detection без списъци

```
# Проверка за попадение: куршум и враг на един и същи LED
for b in range(MAX_BULLETS):
    if not b_active[b]:
        continue
    for e in range(ENEMY_COUNT):
        if e_active[e] and e_pos[e] == b_pos[b] and e_color[e] == b_color[b]:
            e_active[e] = 0 # враг убит
            b_active[b] = 0 # куршум изчезва
            synth.coin()    # звуков ефект при попадение
```

Файл

- `examples/RGB_Guardian.py`

Ключови зависимости

- `examples/retro_synth.py`
- `machine.WS2812` - ()
- P109, P110, P111 2 12 P112

Резюме

-
- - , 2 12
- , append/remove

Самопроверка

1. bytearray list
- 2.
- 3.

Упражнения

1. ()
2. " 0 (1-)
3. ENEMY_COUNT LED_COUNT - 5 -

Ако не работи

- PIN_WS_POWER = "P500" WS2812
- RetroSynth OSError. retro_synth.py /flash
- ENEMY_COUNT LED_COUNT.

Нататък

Част X

Каталог на всички примери

```

• examples/01_blink_async.py          LED1
• examples/02_two_tasks.py             asyncio
• examples/03_pwm_ab_same_channel.py
• examples/04_touchpad_async.py
• examples/dac_dmac_diag.py            -
• examples/dac_firmware_probe.py       DTC  DMAC  -
• examples/dac_retro_music.py
• examples/dac_retro_sfx.py            coin, jump, laser, explosion
• examples/neopixel.py                P500          P112
• examples/neopixel_test.py
• examples/retro_synth.py              ,  ,
• examples/ws2812_sci.py               2 12          machine.WS2812  SCI2          P112
• examples/ws2812_sci_test.py          2 12          machine.WS2812
• examples/ws2812_sci_chase_145.py          1 + 12*12  2 12
• examples/ws2812_sci_p101.py          2 12          machine.WS2812          SCI0  P101
• examples/ws2812_sci_p109.py          2 12          machine.WS2812          SCI9  P
109
• examples/20_language_basics/01_sample_program.py
• examples/20_language_basics/02_variables_and_constants.py
• examples/20_language_basics/03_operators_conditions_loops.py
• examples/20_language_basics/04_enums.py  -          const()
• examples/20_language_basics/05_arrays.py  ,          , array, bytearray
• examples/20_language_basics/06_structures.py dict, namedtuple,
• examples/20_language_basics/07_pointers_equivalents.py  , memoryview, machine.mem32
• examples/20_language_basics/08_functions.py  ,
• examples/21_digital_io/01_digital_output.py          LED1
• examples/21_digital_io/02_digital_input.py          SW1
• examples/22_analog/01_analog_input_adc.py
• examples/22_analog/02_pwm_output.py
• examples/23_timing/01_delays_and_timers.py          ,          , uasyncio
• examples/23_timing/02_interrupts.py
• examples/23_timing/03_button_debounce.py          uasyncio
• examples/23_timing/04_software_timer_minus1.py          Timer(-1)
• examples/23_timing/05_timer_compare_hw_sw.py          Timer(1)  Timer(-1)
• examples/23_timing/06_rtc_basic.py
• examples/23_timing/07_adc_timing_measure.py          ADC.read()  time.ticks_us()
• examples/23_timing/08_interrupt_latency_measure.py
• examples/23_timing/09_button_polling_events.py  polling          PRESS, RELEASE, CLICK  LONG_PR
ESS
• examples/23_timing/10_button_irq_deferred.py
• examples/23_timing/11_button_soft_timer_hold.py  Timer(-1)          PRESS, RELEASE, HOLD_START  HOLD_REPEA
T
• examples/23_timing/12_button_shift_debounce.py  -

```

- examples/23_timing/13_hardware_timer_periodic_oneshot.py Timer.PERIODIC, Timer.ONE_SHOT, perio
- d() counter()
- examples/23_timing/14_hardware_timer_output_compare.py output compare - channel(1) channel(2
-)
- examples/23_timing/15_hardware_timer_input_capture.py input capture pulse width
- examples/23_timing/16_hardware_timer_event_count.py event count N
- examples/23_timing/17_fast_irq_two_buttons_queue.py fast ASM IRQ,
- examples/AGT_TEST_CHECKLIST.md
- examples/24_i2c/01_i2c_master_basic.py 2
- examples/24_i2c/02_i2c_runtime_pins.py 2
- examples/24_i2c/03_softi2c_basic.py 2
- examples/24_i2c/04_i2ctarget_memory.py 2 -
- examples/25_spi/01_spi_basic.py
- examples/25_spi/02_softspi_basic.py
- examples/26_touchpad/01_touchpad_basic.py
- examples/26_touchpad/02_touchpad_diagnostics.py ,
- examples/27_storage/01_flash_blockdev.py renesas.Flash /flash
- examples/27_storage/02_dataflash_basic.py dataflash
- examples/27_storage/03_dataflash_state_led.py dataflash
- examples/28_machine_misc/01_machine_info_and_id.py machine.info, freq, unique_id
- examples/28_machine_misc/02_machine_mem_and_bootloader.py disable_irq, enable_irq, mem32, bootloader
- examples/28_machine_misc/03_sleep_modes.py sleep, lightsleep, deepsleep
- examples/28_machine_misc/04_memory_management.py gc, micropython.mem_info, bytearray, memoryview
- examples/29_display_scan/01_dynamic_7seg_and_keypad.py
- examples/30_fsm/01_fsm_lamp_button.py
- examples/30_fsm/02_fsm_table.py
- examples/30_fsm/03_fsm_timeout.py
- examples/30_fsm/04_conditions_vs_events.py
- examples/30_fsm/05_guard_functions.py
- examples/30_fsm/06_priorities.py
- examples/30_fsm/07_event_queue.py
- examples/30_fsm/08_tickless_sleep.py
- examples/32_uart/01_uart_basic.py
- examples/34_external_modules/01_i2c_module_probe.py 2
- examples/34_external_modules/02_spi_module_transaction.py CS
- examples/34_external_modules/03_uart_module_request_response.py /
- examples/33_asyncio/01_asyncio_basics.py
- examples/33_asyncio/02_asyncio_event_flag.py
- examples/index.py

Част X: Курс по елементарна сигнална обработка

P_j . 12- , :
 $[d \text{ } _f \text{ } _d / \text{ } _a4 \text{ } 2_ \text{ } a_ \text{ } ce \text{ } _c \text{ } e_ \text{ } d \text{ } _b . \text{ } d$ 12 (0 11), 19
 PNG , .
 DSP ? a e a e,
 DC/AC, c , a e a b e FFT .
 S ADC, CMSIS-DSP MSGEQ7 XI.

Блок	Роля
. + /	01
.	
	-
(-)	, , , ,

Учебен маршрут

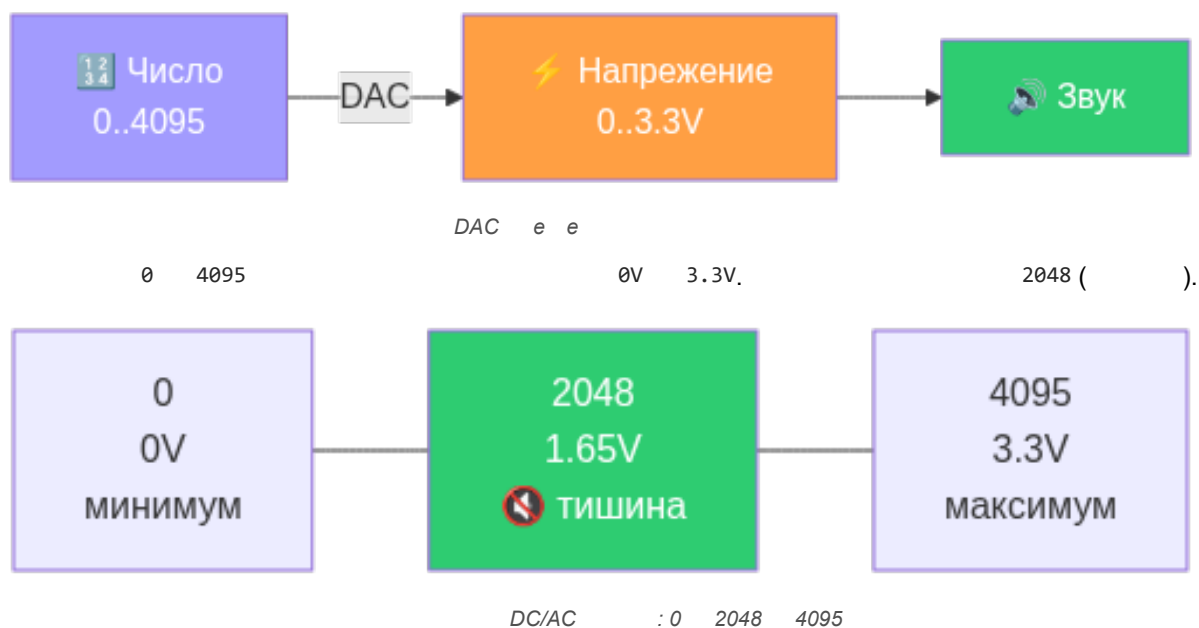
Урок 0-1
DAC основи

```
graph TD; A[Урок 0-1  
DAC основи] --> B[Урок 2-3  
Вълни и генериране]; B --> C[Урок 4  
Clipping / Aliasing]; C --> D[ ];
```

Урок 2-3
Вълни и генериране

Урок 4
Clipping / Aliasing

Как работи DAC pipeline



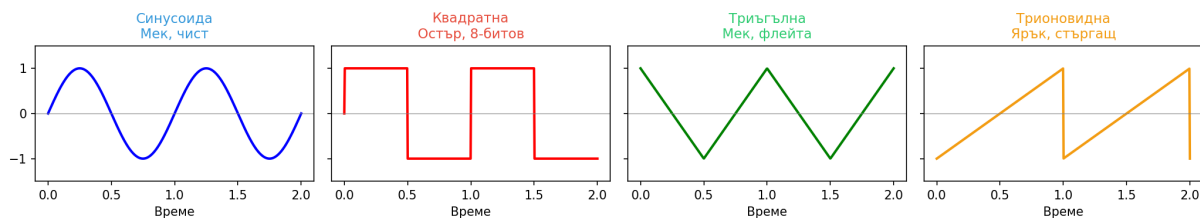
Как работи курсът

1. Аналогия
2. Код
3. Наблюдение
4. Експеримент

1

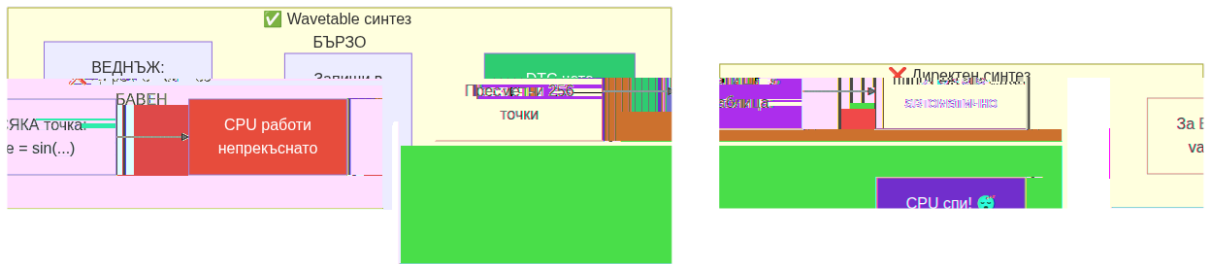
Вълновите форми

Вълнови форми



```
from retro_synth import RetroSynth
synth = RetroSynth("P014")
synth.play_note("A4", 500, "square") # Квадратна – "чиптюн"
synth.play_note("A4", 500, "triangle") # Триъгълна – "мека флейта"
synth.play_note("A4", 500, "saw") # Трионовидна – "остър тембър"
```

Wavetable синтез — защо е бързо

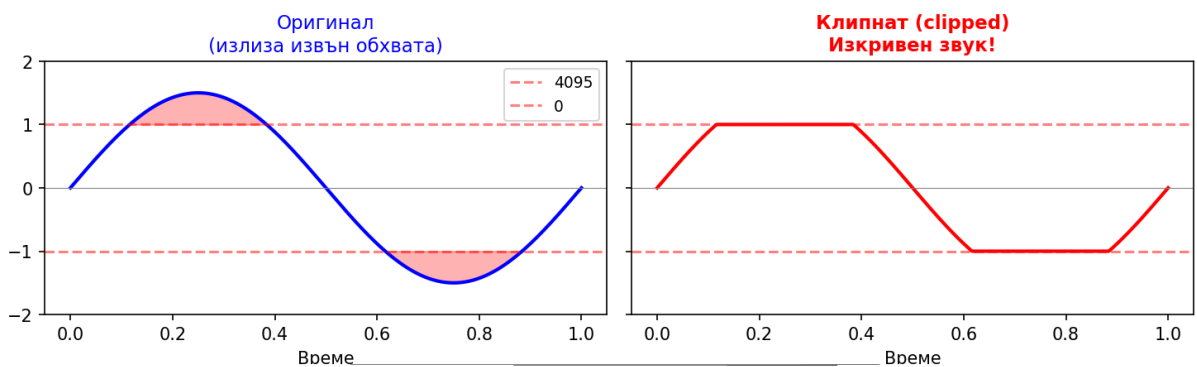


Wa e ab e

веднъж,

Клипване

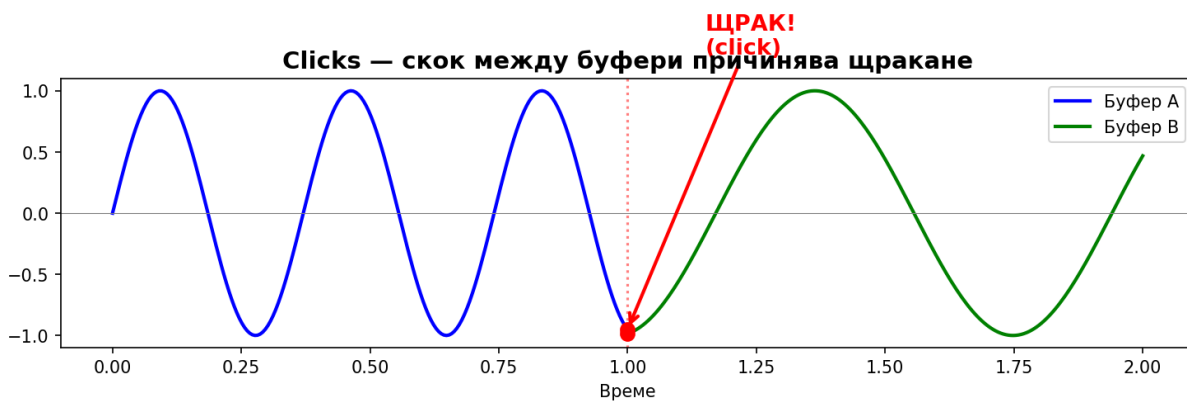
Клипване (Clipping)



c ed

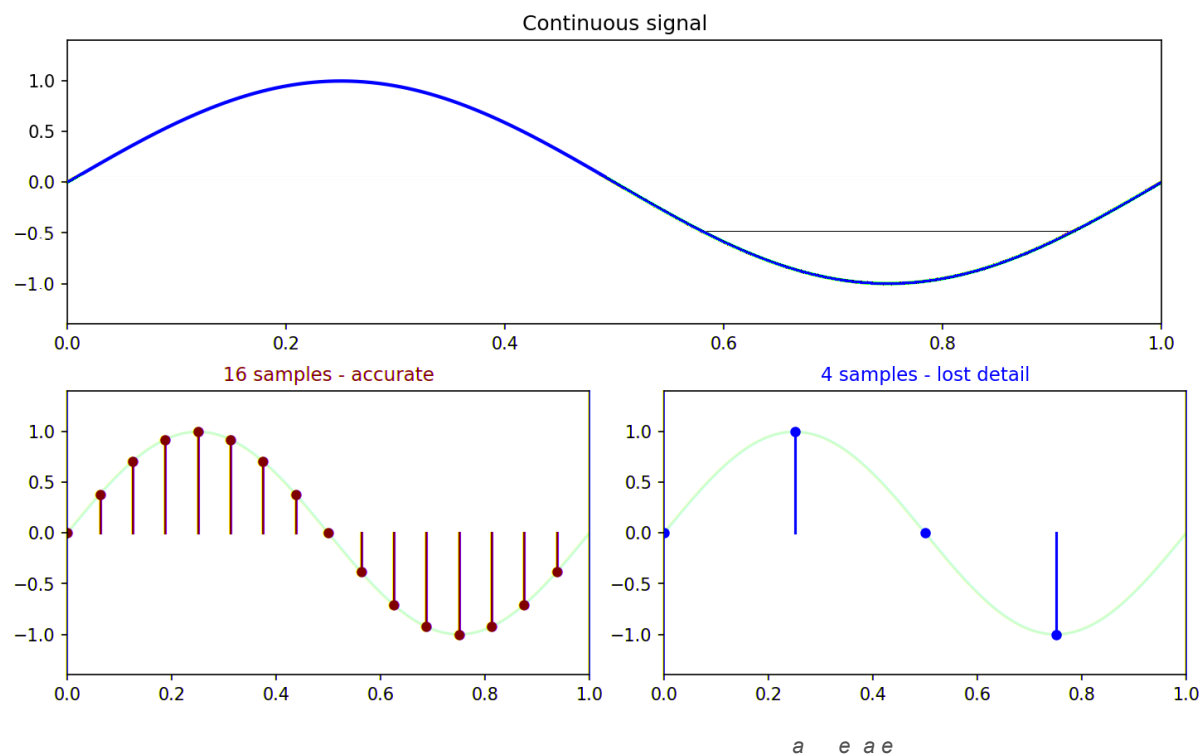
```
def clamp12(v):  
    if v < 0: return 0  
    if v > 4095: return 4095  
    return v
```

: N , 1/N.

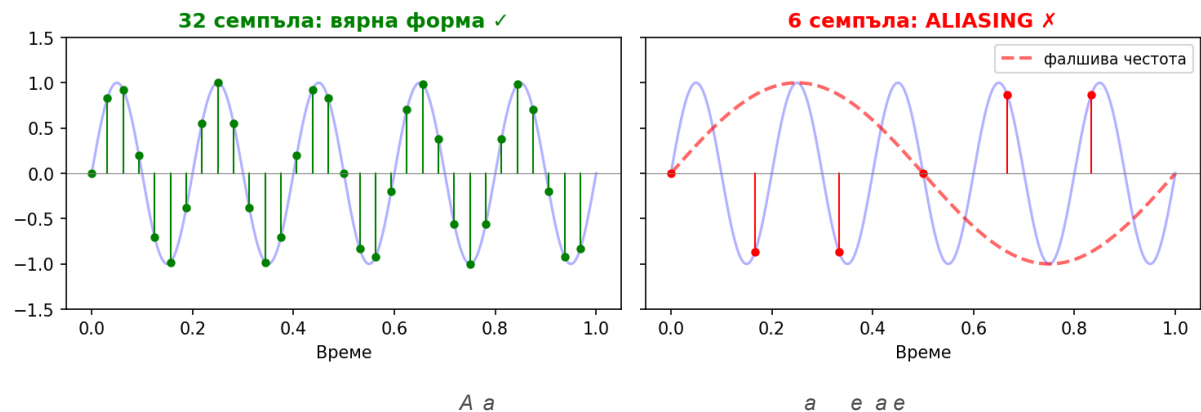


C c

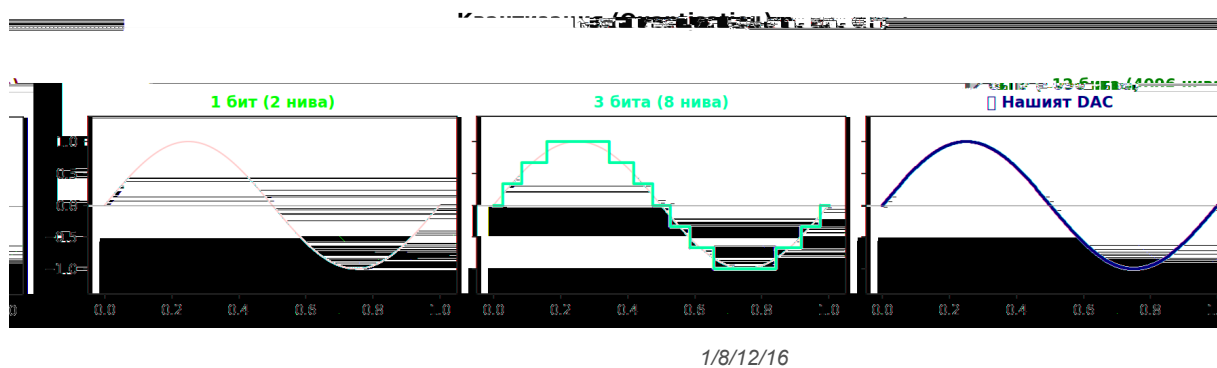
Sampling и Aliasing



Aliasing — фалшиви честоти при нисък sample rate



Квантизация



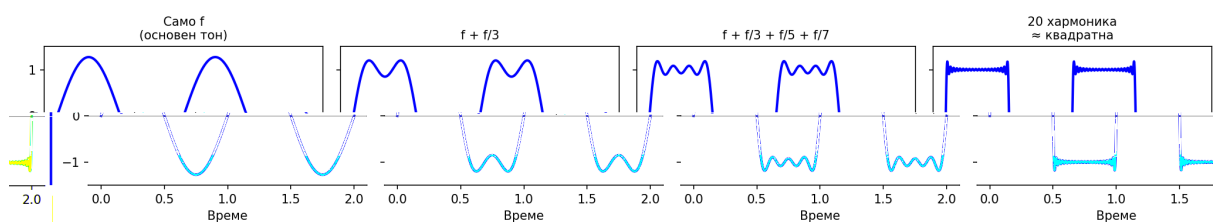
VK_RA4M2

12-битов

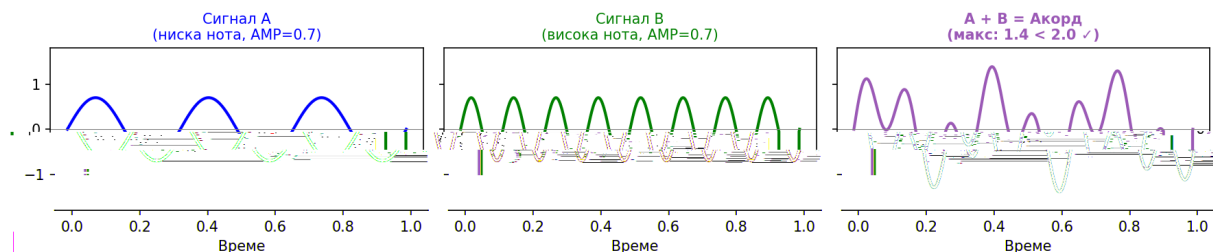
0 4095.

Теоремата на Фурие

Теорема на Фурие — квадратна вълна от синусоиди

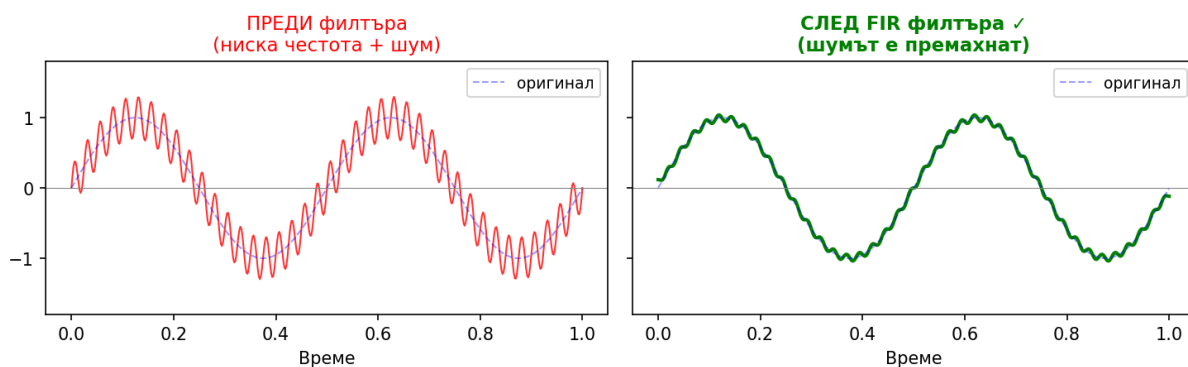


Смесване на два сигнала (адитивен синтез)

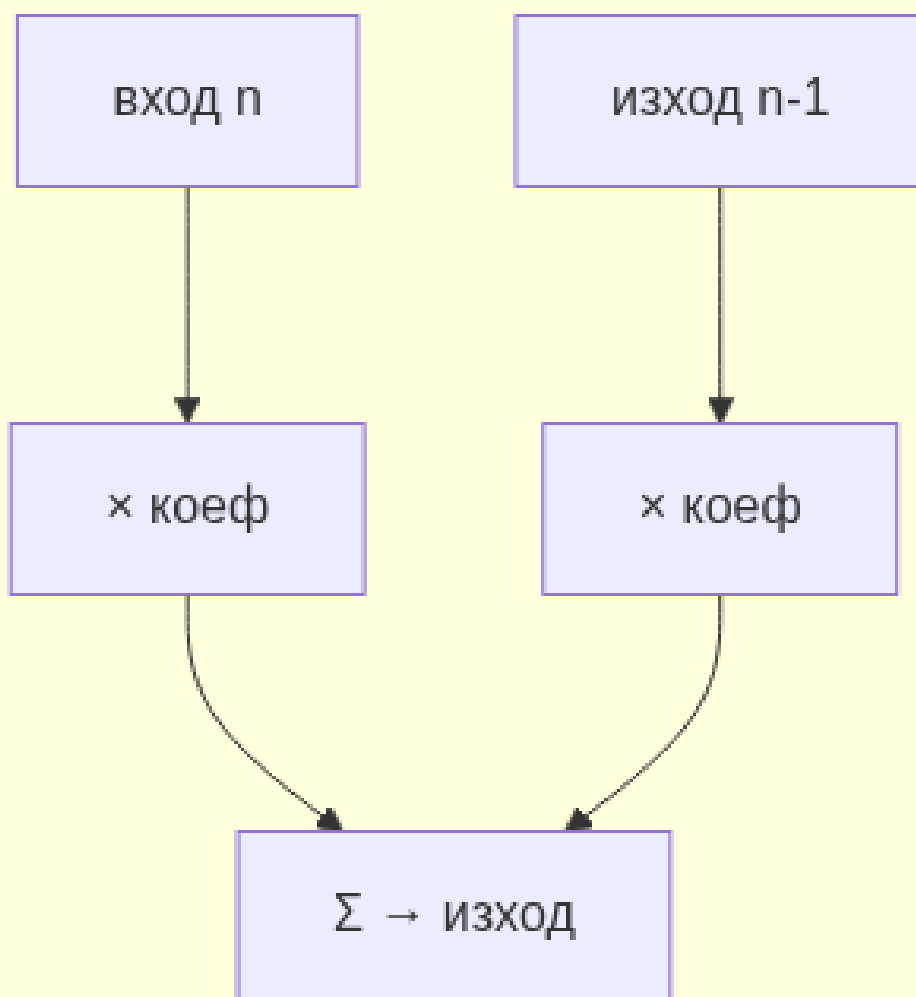


FIR и IIR филтри

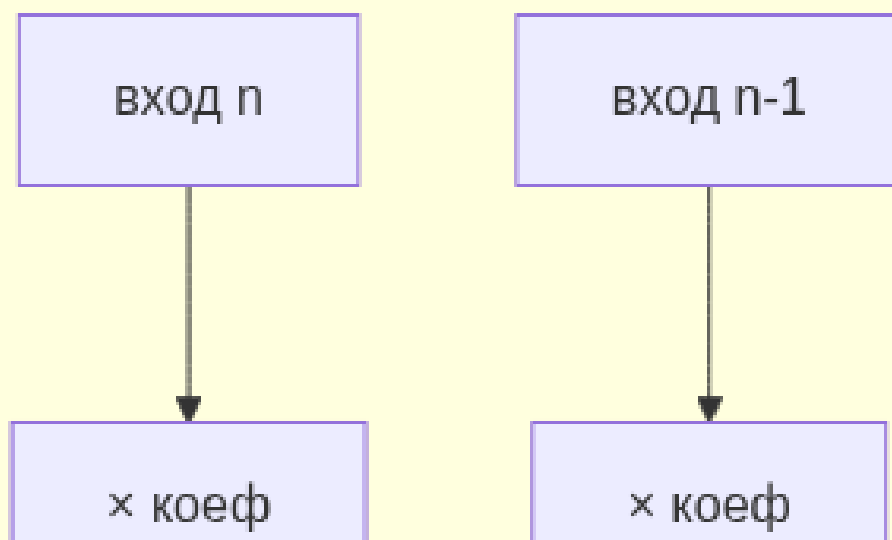
FIR нискочестотен филтър



FIR

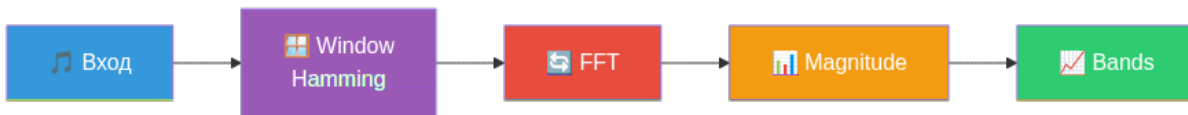
IIR с памет 

FIR без памет



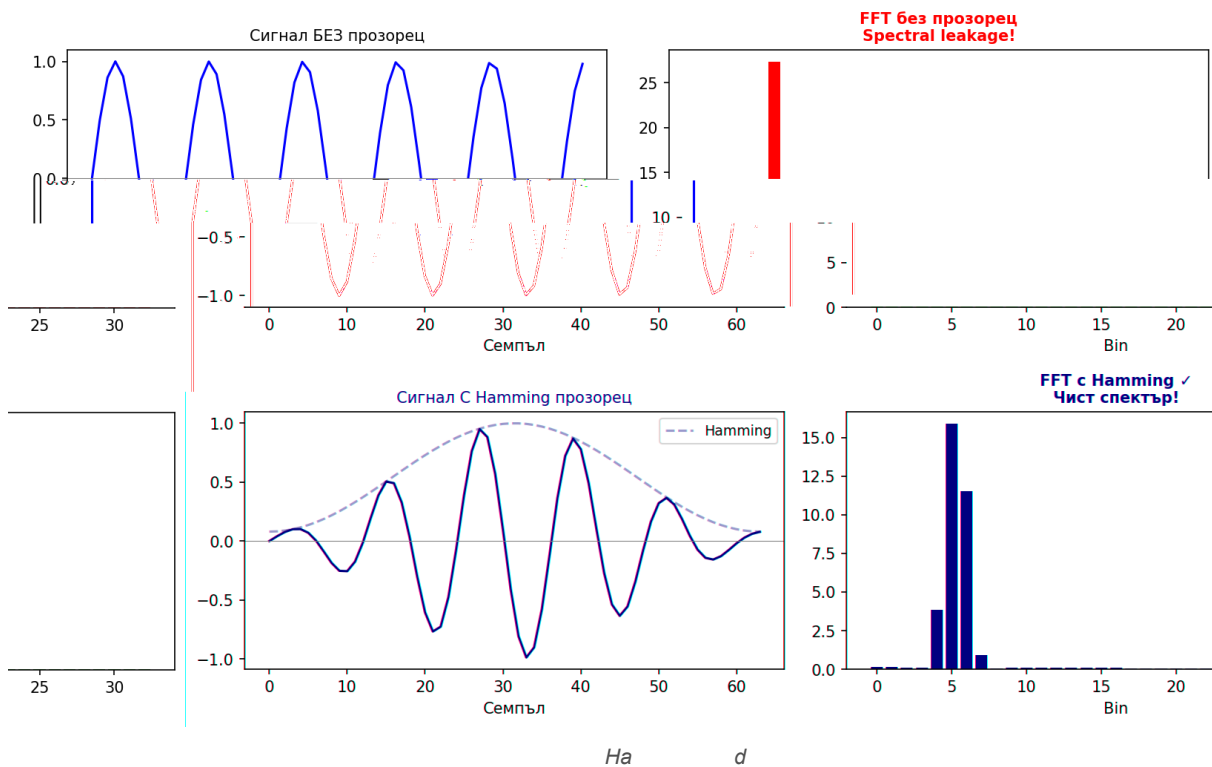
```
import dsp
fir = dsp.FIRFilter([0.2, 0.2, 0.2, 0.2, 0.2]) # 5-tap moving average
fir.run(input_buf, output_buf)
```

FFT спектър



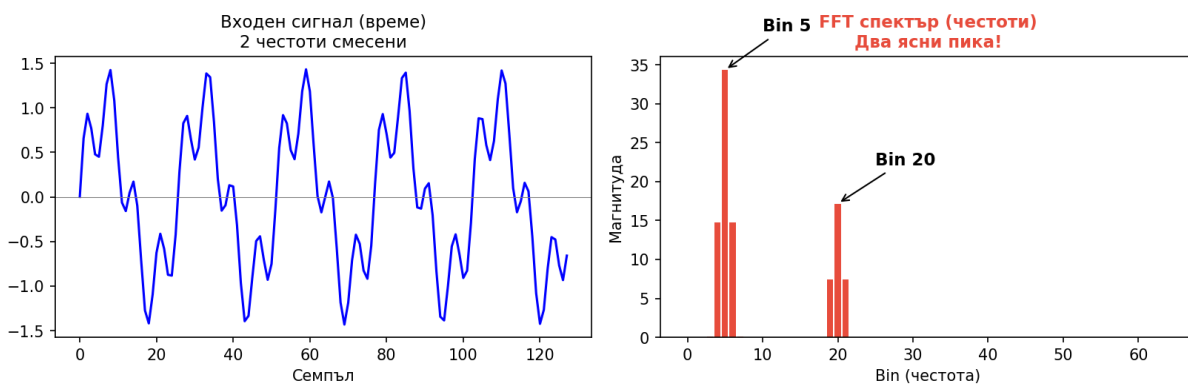
FFT е е

Hamming прозорец — защо е нужен?



На d

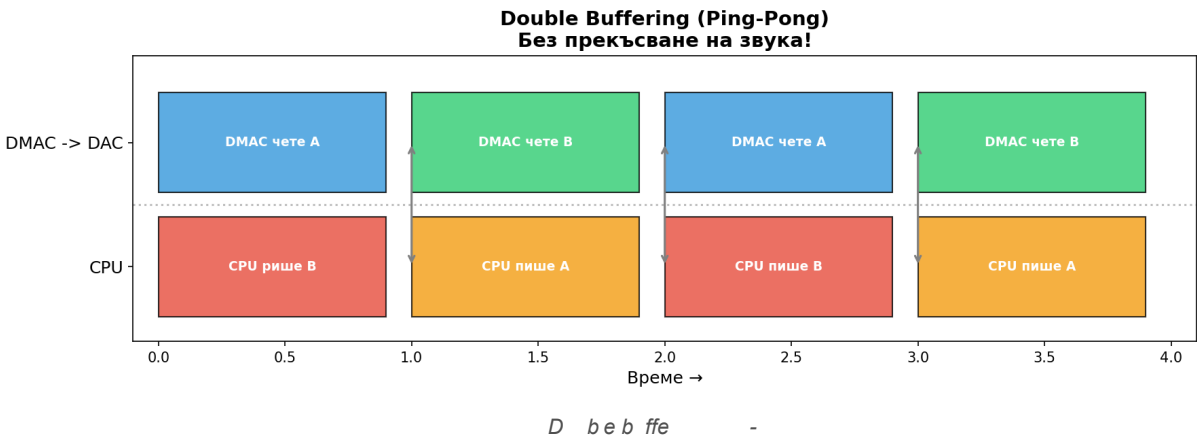
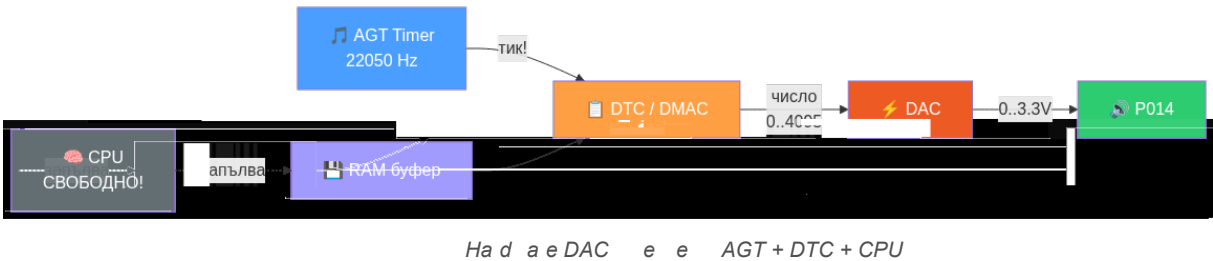
FFT — от време към честота



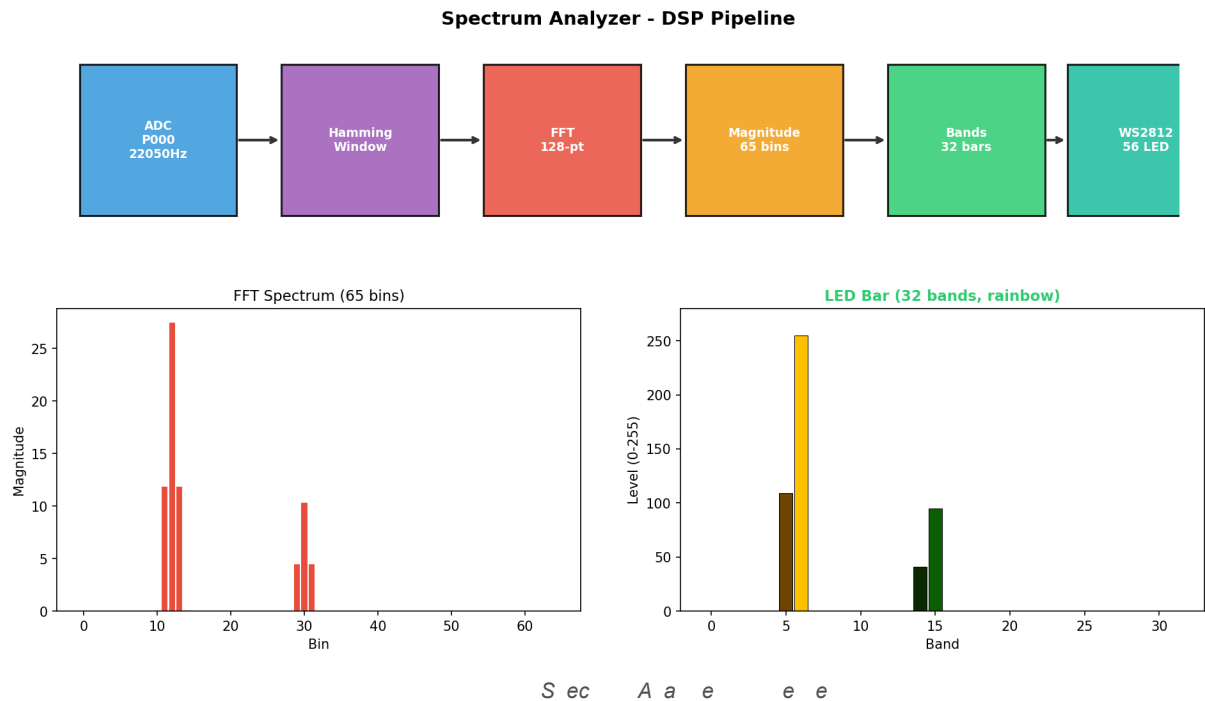
FFT

```
fft = dsp.FFT(128)
fft.run(buf, out)    # out[i] = magnitude на i-тия честотен bin
```

Hardware audio pipeline — DTC/DMAC



Спектрален анализатор с LED лента



```

from machine import AudioADC
import dsp

adc = AudioADC("P000", fs=22050, frame=128)
adc.start()
fft = dsp.FFT(128)
while True:
    if adc.ready():
        adc.read_f32(buf)
        fft.run(buf, mag)
        # → mag[i] = сила на честота i * (22050 / 128) Hz

```

Дванадесетурочен маршрут

#	Урок	Ключова концепция	Файл в dsp_for_kids/
0		, (' '), 20	2
1	,	,	
2		, , ,	
		, . ,	
	/ , ,	12(),	
		. (), . ()	
		,	
		,	
		,	
10		,	
11		+ + 2 12	/

Част XI: Цифрова обработка на аудио сигнали (DSP)

(- ,) () .

Глава 53. Storm ADC — DTC-управлявано аудио семплиране

[T]
ADC

: ADC. ead()
GPT
P

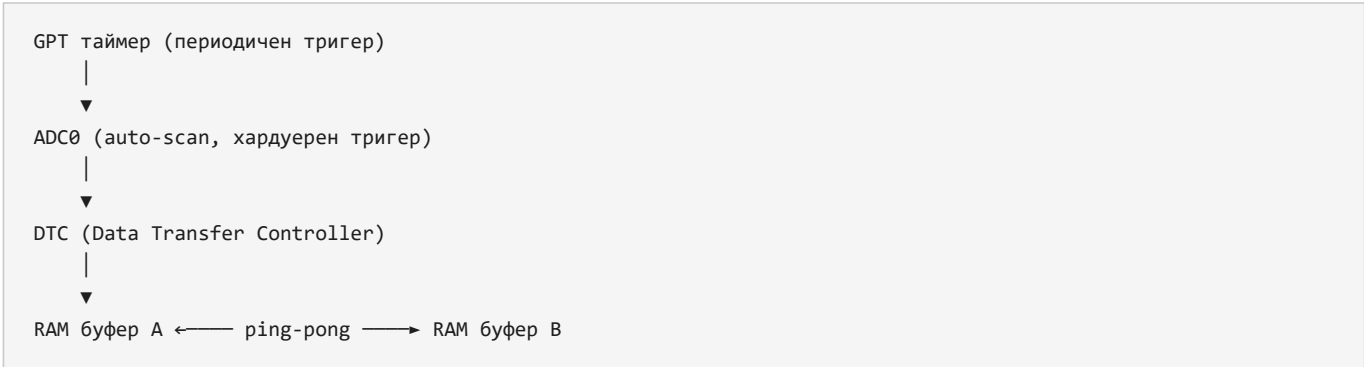
. S
, DTC
, CPU

Метод	Кой взима семпла	Гарантиран timing	Аудио?
. ()	()		
(+ +)			220 0

Какъв е проблемът

ADC.read() една стойност
(. 220 0). 1
, 22

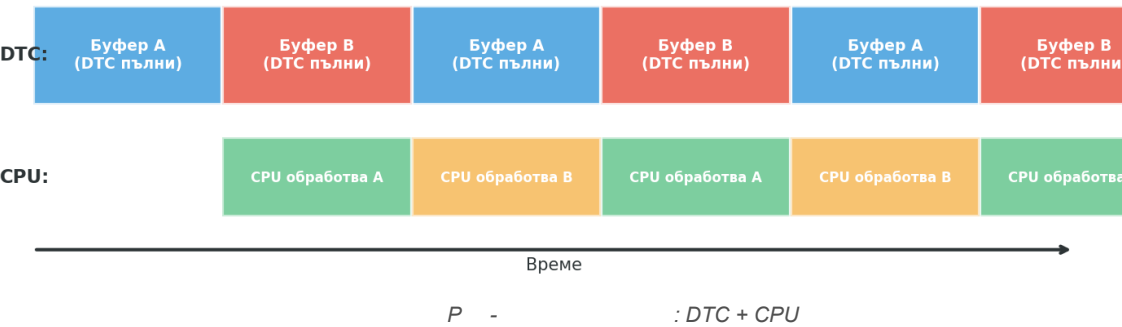
Storm ADC



Архитектура: GPT → ADC → DTC → ping-pong

GPT таймер Fs. 0
ADDR ()
repeat-block frame

Ping-pong буфериране: DTC пълни единия, CPU обработва другия — без загуба на семпли



Регистрова конфигурация (RA4M2)

GPT0 (таймер за тригер):

Регистър	Стойност	Описание
.	0 (-)	
	/ - 1	
		0

$$0 \quad 220 \quad 0 \quad GTPR = 50000000 / 22050 - 1 = 2267.$$

ADC0 (аналогово-цифров преобразувател):

Регистър	Стойност	Описание
.	0 10 ()	
.	1	
.	0	
0		
.	0 (-)	12-

DTC (Data Transfer Controller):

Параметър	Стойност	Описание
	& 0.	(1 -)
	/	-
	-	
	(12)	
	1 -	

MicroPython интерфейс — клас AudioADC

```

from machine import AudioADC

adc = AudioADC("P000", fs=22050, frame=128)
adc.start()          # Стартира GPT + ADC + DTC.

while not adc.ready(): # Проверява дали DTC е запълнил текущия буфер.
    pass

adc.read_f32(buf)     # Копира 128 семпла като float32, нормализирани [-1.0 .. +1.0].
                    # DC offset (2048 при 12-bit) се изважда автоматично.

adc.stop()           # Спира GPT таймера, ADC и DTC.

```

``read_f32(buf)`` (-)

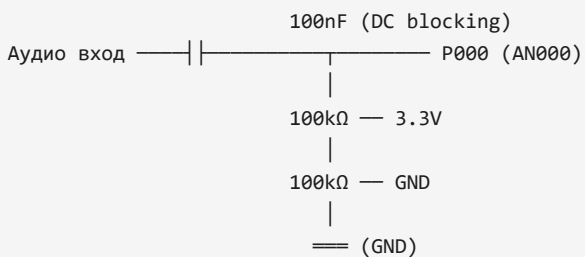
```

for (int i = 0; i < frame; i++) {
    buf[i] = ((float32_t)raw[i] - 2048.0f) / 2048.0f; // [-1.0 .. +1.0]
}

```

20 (/2 1. .).

Входна схема (AC coupling)



- 100nF кондензатор
- Резистивен делител 100kΩ/100kΩ
- 0 .. . 1. .
- 1. .

/2 1. .

Защо е важен ring-pong буферът

1. Спираме ADC → четем → пускаме ADC

+ (2). 220 0

2. Четем от активния буфер

().

Файлове

- ra/ra_storm_adc.c (+ +)
- ra/ra_storm_adc.h storm_adc_config_t

- machine_audioadc.c ()

Резюме

- хардуерна верига без CPU участие , .
- - нула загубени семпли нула race conditions.
- AudioADC start(), ready(), read_f32(), stop().
- 12- 2 -1.0..+1.0 read_f32().

Самопроверка

- 1.
2. - ,
3. 20 read_f32()
4. , (. + 100)

Упражнения

1. 0 .
2. (20)
3. , time.ticks_us() adc.ready().

Ако не работи

- adc.ready() True, , adc.start()
- read_f32() 0.0.
- 1. 0 .
- 100

Нататък

” ,

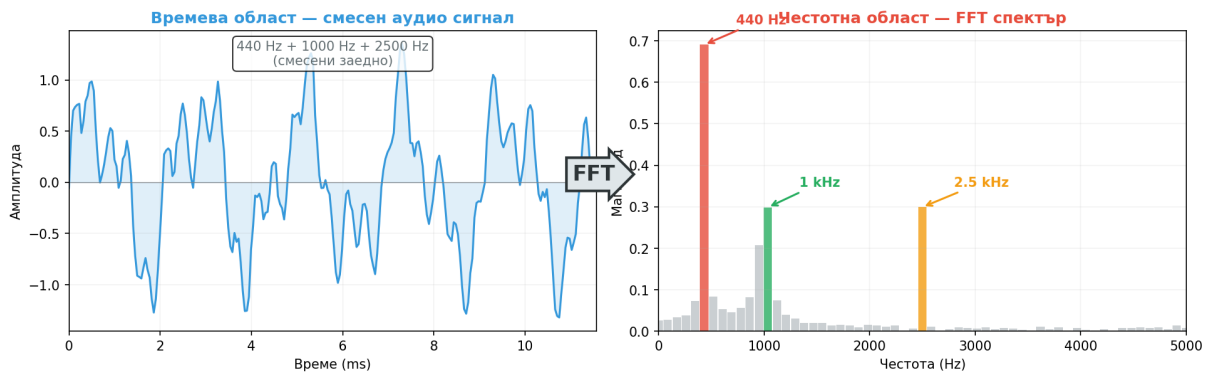
Глава 54. FFT — честотен анализ с CMSIS-DSP

[T] : FFT " "

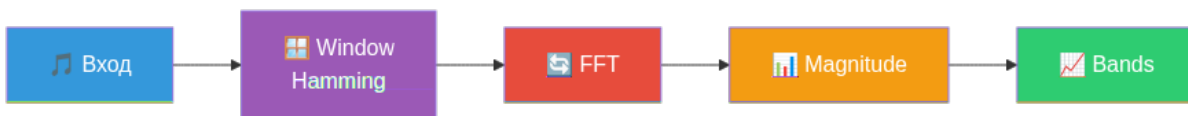
() . 128 ()

64 ().

FFT: от времева област към честотен спектър (CMSIS-DSP)



FFT:



FFT е е

Какъв е проблемът

Честотна област — FFT спектър

12 (-).

100 , 1 , 10 .

FFT ().

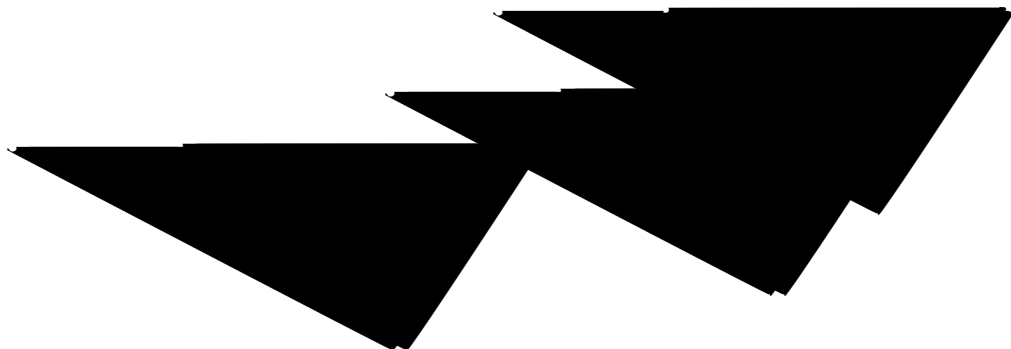
0 + 1 + 2. ,

CMSIS-DSP: защо не пишем FFT сами

CMSIS-DSP,

- `arm_rfft_fast_f32()`
- `( / )`
- `-`
- `2 ( - 100 )`
- `12 - ~35 μs`

DSP pipeline: 4 стъпки



```

PCM буфер (128 float32)
|
▼ fft.window_apply(in, out)
Hamming прозорец
|
▼ fft.run(in, out)      ← arm_rfft_fast_f32()
FFT комплексен резултат
|
▼ fft.magnitude(cplx, mag) ← arm_cmplx_mag_f32()
Магнитуди (65 реални стойности)
|
▼ fft.bands(mag, out)
N ленти (0-255 байтове)

```

Стъпка 1: Hamming прозорец

"", spectral leakage (

).

```
w[n] = 0.54 - 0.46 * cos(2π * n / (N-1))
```

веднъж

apply()

window_

```

for (int i = 0; i < fft_len; i++) {
    out[i] = in[i] * self->window[i];
}

```

Стъпка 2: RFFT (Real FFT)

arm_rfft_fast_f32() 12 12 ,

(,). 0 () ()

.

Вход: [x0, x1, x2, ..., x127] (128 float32)

Изход: [DC, Nyq, Re1, Im1, Re2, Im2, ... Re63, Im63] (128 float32)

/ 220 0 / 12 172.27 Hz .

Bin	Честота	Пример
0	0 ()	
1	1 2	
	10	
2	000	
	10000	
	1102	()

Стъпка 3: Магнитуди

(,)

```
mag[k] = sqrt(Re[k]2 + Im[k]2)
```

- arm_cmplx_mag_f32(),
(0).

Стъпка 4: Лентови магнитуди (bands)

2 .

N ленти ()

(1-2),

(10-20).

веднъж set_bands(n, db_floor)

```
float32_t ratio = powf((float32_t)half, 1.0f / (float32_t)n);
for (b = 0; b < n; ++b) {
    band_lo[b] = (uint16_t)(powf(ratio, b) + 0.5f);
    band_hi[b] = (uint16_t)(powf(ratio, b + 1) + 0.5f);
    if (band_hi[b] <= band_lo[b]) band_hi[b] = band_lo[b] + 1;
}
```

bands()

максималната

0-2

```
for (b = 0; b < n_bands; ++b) {
    // Намиране на максимума в обхвата [band_lo .. band_hi]
    float32_t peak = 0.0f;
    for (k = band_lo[b]; k < band_hi[b]; ++k) {
        if (mag[k] > peak) peak = mag[k];
    }
    // Конверсия в dBFS: 20*log10(peak)
    float32_t db = 20.0f * log10f(peak + 1e-12f);
    // Машабиране: db_floor..-0 → 0..255
    int32_t val = (int32_t)((db - db_floor) * db_scale);
    out[b] = (val < 0) ? 0 : (val > 255) ? 255 : (uint8_t)val;
}
```

db_floor

db_floor	Значение	Използване
- 0	-	
- 0	()	
- 0		,

MicroPython интерфейс — модул dsp

```

import dsp
from array import array

fft = dsp.FFT(128)           # Създава FFT обект за 128 семпла.
fft.set_bands(32, -60)       # 32 ленти, шумов праг -60 dBFS.

frame = array('f', [0.0] * 128) # PCM вход.
fbuf = array('f', [0.0] * 128)  # FFT работен буфер.
mag = array('f', [0.0] * 65)    # Магнитуди (N/2 + 1).
out = bytearray(32)            # 32 ленти × 1 байт.

# В главния цикъл:
fft.window_apply(frame, fbuf)   # Hamming прозорец.
fft.run(fbuf, fbuf)            # RFFT in-place.
fft.magnitude(fbuf, mag)       # Комплексни → магнитуди.
fft.bands(mag, out)            # 65 бина → 32 ленти (0-255).

```

няма

Връзка FFT параметри ↔ реална резолюция

FFT_LEN	Fs (Hz)	Bin spacing	Мин. честота	Макс. честота	Латентност
	220 0	.		1102	2.
12	220 0	1 2.	1 2	1102	.
2	220 0	.1		1102	11.
12	220 0	.1		1102	2 .2

Файлове

- moddsp.c
- -

Резюме

- 12
- ~35 μs
- set_bands(), bands(), powf().
- -

Самопроверка

- 1.
2. 12 -
3. " 1 2 "
4. ,

Упражнения

1. 2 100 .

2.
- ,
-
- .
3.
- db_floor
- 0, - 0 - 0
- .

Ако не работи

- (). db_floor=-40.
- 1-2 .
- " ". , window_apply() run().

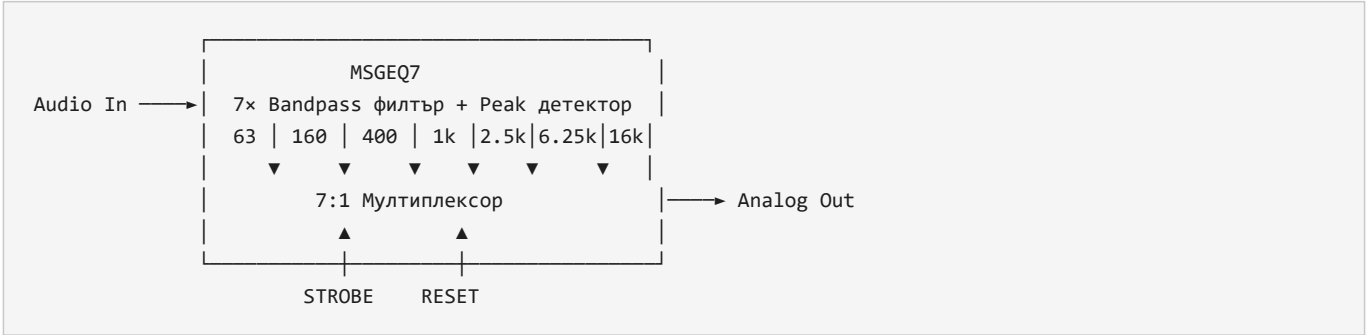
Нататък

,

Глава 55. MSGEQ7 емуляция — 7-лентов спектрален анализатор

Какъв е проблемът

MSGEQ7 (),



Проблемът

Недостатък	Описание
	0 ()
	+

Решението: FFT-базирана емуляция

FFT магнитуди (65 бина)
bin0 bin1 bin2 ... bin64
| | | |
▼ ▼ ▼ ▼

63Hz	160Hz	400Hz	...
------	-------	-------	-----

 ← 7 ленти с MSGEQ7 граници
byte0 byte1 byte2 ... byte6 (0-255)

Метод set_bands_msgeq7(fs, db_floor=-60)

(set_bands), set_bands_msgeq7

геометрични средни

Центрове: 63 160 400 1000 2500 6250 16000
 ↓ ↓ ↓ ↓ ↓ ↓
Граници: 39 100 253 632 1581 3953 10000 Nyquist

$\text{sqrt}(fc[i] * fc[i+1])$

Граница	Изчисление	Честота
	/ 1.	
1 0	(1 0)	100
1 0 00	(1 0 00)	2
00 1000	(00 1000)	2
1000 2 00	(1000 2 00)	1 1
2 00 2 0	(2 00 2 0)	
2 0 1 000	(2 0 1 000)	10000
1		1102

индекси

bin = round(freq / bin_width)
bin_width = Fs / FFT_LEN = 22050 / 128 = 172.27 Hz

moddsp.c

```
static const float32_t msgeq7_fc[7] = {
    63.0f, 160.0f, 400.0f, 1000.0f, 2500.0f, 6250.0f, 16000.0f
};

for (uint8_t b = 0; b < 7; ++b) {
    float32_t f_lo = (b == 0)
        ? msgeq7_fc[0] / 1.6f
        : sqrtf(msgeq7_fc[b-1] * msgeq7_fc[b]);
    float32_t f_hi = (b == 6)
        ? nyquist
        : sqrtf(msgeq7_fc[b] * msgeq7_fc[b+1]);

    uint32_t blo = (uint32_t)(f_lo / bin_w + 0.5f);
    uint32_t bhi = (uint32_t)(f_hi / bin_w + 0.5f);
    // ... clamp & store
}
```

set_bands_msgeq7()

същият bands()

Бинова карта при Fs=22050, FFT_LEN=128

Лента	Център	f_low	f_high	bin_lo	bin_hi	Бинове
0			100	1	1	1
1	1 0	100	2	1	1	1
2	00	2	2	2		
	1	2	1 1			
	2.	1 1			2	1
	.2		10000	2		
	1	10000	1102			

Забележка

Визуализация: 7 × 8 LED bar graph

![7-лентов спектрален анализатор – LED визуализация](img/55_spectrum_7band.png)

Render функция (zero-alloc):

```
( , )
( )

( * + 12 ) # 0..
( )
( LEDS_PER_BAND + )
```

```
( * )
(0, , 0, , ) #
( , , 0, , ) #
( , 0, 0, , ) #
```

+1 +2 0

Сравнение: истински MSGEQ7 vs цифрова емуляция

Параметър	Хардуерен MSGEQ7	FFT емуляция (RA4M2)
Брой ленти	7 (фиксиран)	7 (конфигуруеми)
Динамичен обхват	~40 dB	>90 dB (float32 FFT)
Допълнителен хардуер	MSGEQ7 + clock + capacitors	Нищо (само ADC + LED)
Гъвкавост	Фиксиран	7 или 32 ленти, сменяем
Цена	~\$2 за чип + PCB	\$0 (вградено в софтуера)
Strobe/Reset протокол	Необходим (7 ADC четения)	Не е необходим
Латентност	~1 ms (аналогов)	~6 ms (FFT 128 точки)
CPU натоварване	0% (отделен чип)	~5% (FFT + bands)

Проект от общността: 14-Channel Analyzer (donnersm)

Проектът [14ChannelAnalyzerV2.0](<https://github.com/donnersm/14ChannelAnalyzerV2.0>) от Mark Donners демонст...

MicroPython пример – пълен MSGEQ7 анализатор

, , 2 12,

,

Конфигурация

#

.

0

12

Хардуер

(" 00", . , 1)
 . (0)
 2 12(, (" 112"),)
 (" 000", 220 0, 12)



DSP

. (12)

. (220 0, - 0) # !

Буфери

```

        ('', 0.0 * 12 )
    ('', 0.0 * 12 )
    ('', 0.0 *   )
        (           )
        (           )
        (           * )

.   ()

        .   ()

.   2(           )
.   (           ,           )
.   (           ,           )
.   (           ,           )
.   (           ,           ) #

```

Peak hold + render + WS2812 ...

Файлове

- `moddsp.c` – `set\_bands\_msgeq7()` имплементация (C)
- `examples/spectrum\_analyzer.py` – пълен анализатор с 5 визуални режима

Резюме

- MSGEQ7 е аналогов 7-лентов спектрален анализатор, който може да се замени с FFT + софтуерно групиране на ...
- `set\_bands\_msgeq7(fs)` изчислява граници чрез геометрични средни между 7-те MSGEQ7 центрови честоти.
- След конфигурацията, `bands()` работи с 7-байтов изход – без промяна на API.
- 56 WS2812 (7×8) дават класически bar graph с TriBar цветова схема.
- Цифровата емуляция превъзхожда хардуерния MSGEQ7 по динамичен обхват, гъвкавост и цена.

Самопроверка

1. Защо границите между MSGEQ7 лентите се изчисляват като геометрична средна, а не като аритметична?
2. Колко FFT бина покрива лентата за 6.25 kHz при FFT_LEN=128 и Fs=22050?
3. Какво е предимството на цифровата емуляция пред хардуерния MSGEQ7 чип?
4. Защо ниските ленти (63 Hz, 160 Hz) имат само по 1 бин при FFT_LEN=128?

Упражнения

1. Добавете 8-ма лента с център 20 kHz: модифицирайте масива `msgeq7\_fc[]` и стойността `7` в `moddsp.c`.
2. Сравнете визуално `set\_bands(7)` и `set\_bands\_msgeq7(22050)` – коя версия дава по-естествен спектър?
3. Експериментирайте с FFT_LEN=256: подобрява ли се разделянето на лентите за 63 Hz и 160 Hz?

Ако не работи

- Симптом: само горните ленти (2.5k, 6.25k, 16k) светят. Проверка: нормално е за говор или музика с малко б...
- Симптом: лентата за 63 Hz не реагира на бас. Проверка: при FFT_LEN=128 тази лента има само 1 бин (172 Hz)...
- Симптом: нивата са твърде ниски. Проверка: опитайте `db\_floor=-80` за по-висока чувствителност.

Нататък

С тези три глави – Storm ADC, FFT и MSGEQ7 емуляция – имате пълната DSP верига от аналогов сигнал до визуал...

Глава 56. DSP Synthesis – Петро синтезатор архитектура

Глави 56–58 покриваха аудио ****входа****: ADC → FFT → визуализация. Тази глава обхваща обратната посока – ****из****...

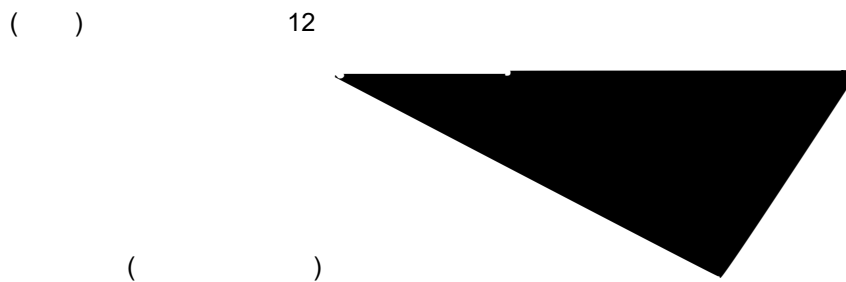
Архитектура на wavetable синтеза

Принципът е прост: един период на вълната се записва в lookup таблица (`array('H', ...)`), а DTC циклично п...

Например за нота A4 (440 Hz) с таблица от 128 семпла:

0 12 20

AGT таймерът генерира прекъсване на 56 320 Hz, DTC прехвърля по 1 семпъл от таблицата към DAC регистъра при...



Вълнови форми

`retro\_synth.py` предизчислява 6 вълнови таблици при създаване на обекта (веднъж, без GC натоварване в горе...

****Square (50% duty cycle)****

```
( , , 1, 2)
( * )//
( )
+ -
```

Резултат: правоъгълна вълна с рязко превключване между +amp и -amp. Богат на нечетни хармоници (1f, 3f, 5f....

****Pulse 25% и Pulse 12.5%****

Същата функция `make\_square()` с `duty\_num=1, duty\_den=4` (25%) или `duty\_den=8` (12.5%). По-тесният pulse ...

****Triangle****

```
( , )
// 2
( )
#
+ *
#
- * ( - )
```

Линеен наклон нагоре, после надолу. Съдържа само нечетни хармоници, но с бързо намаляваща амплитуда ($1/n^2$)....

****Sawtooth****

$$\begin{aligned} & \left(\frac{1}{n^2} \right) \\ & \left(\frac{1}{n^2} \right) \\ & + \left(\frac{1}{n^2} \right) // (n - 1) \end{aligned}$$

Линейно нарастване от -amp до +amp, после рязко падане. Съдържа ****всички**** хармоници (четни + нечетни). Най...

****LFSR шум (noise burst)****

$$\begin{aligned} & \left(\frac{1}{n^2} \right) \\ 0 & \quad 1 \# 1 - \\ & \left(\frac{1}{n^2} \right) \\ ((& 0) (\quad 2) (\quad) (\quad)) \& 1 \\ (& 1) (\quad 1) \\ & * (\quad -) // \quad \# \\ + & (\quad \& 1) - \end{aligned}$$

16-bit LFSR (Linear Feedback Shift Register) генерира псевдослучайна последователност. Полиномът $x^{16} + x^...$

За разлика от периодичните вълни, шумовият буфер се изсвирва еднократно (mode=`NORMAL`), защото цикличното ...

Нотна система и MIDI конверсия

Модулът поддържа три формата на вход:

Формат	Пример	Как работи
Числена честота	`440` или `440.0`	Директно, без конверсия
Честота като текст	`"440Hz"`	Парсва числото преди "Hz"
Научна нотация	`"A4"`, `"C#5"`, `"Eb3"`	MIDI → честота

Конверсията от нота към честота минава през MIDI номер:

$$\begin{aligned} & (\quad + 1) \quad 12 + \\ 0 & \quad 2 ((\quad) / 12) \end{aligned}$$

Полутоново транспониране (за арпеджа и sweep):

$$\begin{aligned} & (\quad , \quad) \\ 2.0 & * (\quad / 12.0) \end{aligned}$$

12 полутона = 1 октава = удвояване на честотата. Темперираната скала е логаритмична – всеки полутон е $\times 2^{1/12}$ (...)

Sweep алгоритъм

Sweep-ът е линейна интерполация на честотата от start до end за определено време:

```

    ( , , , , 2 )
    ( )
    ( )
    //
    ( )
    + ( - ) * / ( - 1)
. ( , ) #
. ( ) # 1

```

Всяка стъпка ****рестартира**** DTC с нов sample rate (нова честота), запазвайки същата wavetable. Резултатът е...

SFX	Start	End	Стъпки	Вълна	Ефект
Jump	A3 (220 Hz)	F5 (698 Hz)	18	square	Sweep нагоре
Laser	A6 (1760 Hz)	A3 (220 Hz)	28	saw	Sweep надолу

Арпеджо архитектура

Арпеджото е повтаряща се последователност от ноти, дефинирана чрез полутонови отмествания от основната:

```

    ( , , (0, , ), 0, )
    ( )
    ( )
. ( ( , ), )

```

По подразбиране `(0, 4, 7)` е мажорен акорд:

- 0 полутона = основна нота (root)
- 4 полутона = голяма терца
- 7 полутона = чиста квинта

Добавянето на `12` (октава) създава пълен октавен арпеджо: `(0, 4, 7, 12)`.

DTC vs DMAC за аудио

Параметър	DTC	DMAC
Макс. repeat	256 семпла	1024+ (ISR re-arm)
Circular mode	Хардуерен (безкраен)	ISR рестарт
CPU натоварване	0% ~0.01% (ISR)	
Каналите	Споделен	8 независими
Предпочитан за	Wavetable loop (≤ 256)	Noise burst (> 256)

RetroSynth автоматично опитва DMAC за noise burst (2048 семпла > 256 DTC лимит), с fallback на DTC ако DMA...

(, 12000)

(,)

(,)

Смесване на сигнали – четири подхода

Смесването (mixing) на аудио сигнали е фундаментална операция. VK_RA4M2 предлага четири различни подхода, в...

****Подход 1: Pre-computed (офлайн) смесване****

Сумираме вълните математически в буфера ****преди**** DTC стартиране. CPU не участва по време на playback.

audiomixer_demo.py — триъгълник + синус в 1 буфер

```

( )
/
.0 * ( - 0. ) - 1.0 # -1..+1
. (2 a . 20 * ) # 20
12( + ( 0.7 + 00* )) # +

```

Правилото за предотвратяване на клипване: ****сумата на амплитудите не трябва да надвишава MID**** (2048):

AMP × 0.7 × tri	+	SINE_AMP × sine	≤ 2048
------------------------	----------	------------------------	---------------

Ако сумата > 2048, `clamp12()` отрязва – звукът се изкривява (hard clipping).

```

| Плюс | Минус |
|-----|-----|
| 0% CPU по време на playback | Вълновата форма е статична |
| Прост код | Не може да се променя в реално време |
| DTC CIRCULAR – безкраен | Трябва да знаем всички сигнали предварително |

```

****Подход 2: Dual DAC (хардуерно паралелно)****

Два DAC канала (DA0 + DA1) генерират независими сигнали едновременно. Смесването е ****електрическо**** – чрез ...

dac_dual_dtc_dmac.py — един буфер, два канала

0. (, , . , .)

1. (, , . , .)

Схема за пасивно смесване с резистори:

0 (01) 10

()

1 (01) 10

Двата канала могат да имат различни буфери, различни SR, различни DTC/DMAC:

Бас на DA0 (ниска честота) + мелодия на DA1 (висока честота)

0. (, , . , .)
1. (, , . , .)

Плюс	Минус
Истински паралелен изход	Само 2 канала (RA4M2 ограничение)
Независими sample rate	Нужна е външна схема за смесване
0% CPU	Двата сигнала не "знаят" един за друг

****Подход 3: Real-time AudioMixer (C модул, полифоничен)****

Модулът `audiomixer` (`modaudiomixer.c`) реализира ****истински real-time полифоничен миксер**** с до 16 гласа....

Създаваме миксер: 4 гласа, 22050 Hz, DA0

. (, 220 0, (" 01 "))

Подготвяме два семпъла (signed 16-bit)

(' ', ...) # 0

(' ', ...) # 0

Пускаме два гласа ЕДНОВРЕМЕННО

. 0. (,) # 0 0
. 1. (,) # 1 0

Контролираме нивата в реално време

```
. 0. 0. # 0 0%
. 1. 0. # 1 0%
```

Алгоритъмът на `audiomixer_render_buffer()` за всеки изходен семпъл:

```
2 0 ( 2- , )

( ) / / 1 / 1 1
1 / 2 - ( 1 - )

+
( , - 2 , 2 )
( + 2 )
```

****Защо Q15?*** Fixed-point аритметика е ~10× по-бърза от float на Cortex-M33 без FPU coprocessor. Q15 формат...

****Защо int32 акумулатор?*** При 4 гласа с level=1.0: max sum = 4 × 32767 = 131068. Това надхвърля int16 (327...

****Как предотвратяваме клипване?*** Намаляваме level на всеки глас:

4 гласа: $1.0/4 = 0.25$ на глас $\rightarrow$ сума ≤ 1.0 (без клипване)

0.2

Или: 2 гласа по 0.5 = 1.0 (граничен случай)

. 0. 0.
 . 1. 0.

Правило: ****сумата на level стойностите на всички едновременно активни гласове трябва да е ≤ 1.0 **** за гарант...

Double buffering осигурява непрекъснат поток:

()

()

... ()

Плюс	Минус
До 16 гласа	~1-5% CPU (render в ISR callback)
Real-time level control	По-сложен код
Supports loop + one-shot	Един DAC канал
S8/U8/S16/U16 формати	Фиксиран sample rate

****Подход 4: Time-division (последователно)****

RetroSynth свири ноти ****една след друга**** на един DAC канал. Не е истинско смесване, но е най-простият подх...

. (" ", 200) # , 200
 . (" ", 200) # , 200 (!)
 . (" ", 200) # , 200

```
| Плюс | Минус |
|-----|-----|
| Най-прост код | Монофоничен (без акорди) |
| 0% CPU при DTC | Паузи между нотите |
| 1 DAC канал | Не може 2 тона едновременно |
```

****Сравнение на четирите подхода****

```
| Подход | Гласове | CPU | Real-time | Код |
|-----|-----|-----|-----|-----|
| Pre-computed | 1 (static mix) | 0% | Не | `audiomixer_demo.py` |
| Dual DAC | 2 (паралелни) | 0% | Частично | `dac_dual_dtc_dmac.py` |
| AudioMixer | 1-16 (полифоничен) | 1-5% | Да | `modaudiomixer.c` |
| Time-division | 1 (монофоничен) | 0% | Не | `retro_synth.py` |
```

Пълен SFX pipeline

Един SFX ефект (например `coin()`) минава през следните стъпки:

```
.    ()

(" ", 0, " 2 ")
(" 2 ") - (' ', 12 )
(" ") 12 2 2
.    ( , 2 2, , )
      2 2
      0..12 12 ( )
      12 ( 01 )
.    ( 0)
.    ()

(" ", 0, " 2 ")
(" ") 1 1 12 1 2
... ( )
```

Времедиаграма:

0 0 0 1 0

20 1 1
2 . 2